

Privacy Analysis of Augmented Reality Glasses

Bachelor Thesis

Natural Science Faculty of the University of Basel
Department of Mathematics and Computer Science
PET Group
<https://pet.dmi.unibas.ch/>

Advisor: Prof. Dr. Isabel Wagner
Supervisor: Nima Akbari

Michelle Jessica Bösigler
michellej.boesiger@unibas.ch

January 23, 2024

Acknowledgment

I would like to express my sincere gratitude to Professor Dr. Isabel Wagner for providing me with the opportunity to explore such an engaging and interesting topic. I am also very thankful to my supervisor, Nima Akbari, whose expertise and guidance were extremely helpful in overcoming the technical challenges and experimental procedures, especially during the more demanding phases of experimentation and decryption.

Abstract

Wearable augmented reality (AR) devices, such as the Ray-Ban Meta glasses, allow users to take pictures, record videos from the wearer's perspective, and share them online with friends and family on social media. The Ray-Ban Meta glasses are also able to make phone calls and stream music and have an AI assistant that allows users to perform these tasks hands-free. However, these capabilities also present unprecedented privacy concerns for users and bystanders. This thesis investigates the privacy implications of the Ray-Ban Meta glasses through static and dynamic analyses. The static analysis focuses on the device and its companion app, while the dynamic analysis examines network traffic and Bluetooth communications captured during the experiments.

The study reveals that, even though the capture has strong encryption protocols protecting the transmitted content, metadata, such as packet sizes and traffic patterns, can still reveal application usage and sensitive information, including media transfers, command execution, and pairing events. Bluetooth communications further allow for device identification and potential user profiling. The exposed information may also include data related to bystanders, who are often unaware of the collection and, therefore, unable to provide informed consent. Although the companion app did not request excessive permissions or include unnecessary third-party libraries, it continuously communicated with Meta's servers. These continuous transmissions raise concerns about potential long-term tracking, analytics, and possible unauthorized access to the data, which could similarly affect bystander privacy. Although the encryption limited a complete analysis of the traffic captured, partial decryption still revealed the existence of detailed user data, reinforcing the privacy concerns made in the metadata analysis.

Table of Contents

Acknowledgment	ii
Abstract	iii
1 Introduction	1
2 Background	3
3 Related Work	5
4 Device Overview	8
4.1 Hardware Specifications	8
4.2 Software Specifications	11
4.3 Network Capabilities	12
4.4 Privacy and Security	12
5 Methodology and Experiments	15
5.1 Experiment Design	15
5.2 Experimental Setup	16
5.3 Data Collection	17
5.4 Experimental Procedure	18
5.5 Analysis Methods	20
5.6 Limitations	24
6 Static App Analysis	27
6.1 Permissions Analysis	27
6.2 Third-Party Usage	29
6.3 Summary	32
7 Network Analysis	33
7.1 General Network Analysis	33
7.2 Active Traffic	36
7.2.1 User interactions - Touch commands	36
7.2.2 User interactions - Voice commands	43
7.2.3 Pairing Sequence	50

7.3 Idle Traffic Analysis	52
7.4 Summary	55
8 Bluetooth Analysis	56
8.1 BLE Analysis	56
8.2 Classic Bluetooth Analysis	58
9 Decryption	61
9.1 Decryption Attempts	61
9.2 Decryption Analysis	64
10 Discussion	68
11 Conclusion and Future Work	71
 Bibliography	 74
Appendix A Appendices	77
Appendix B Code for Bypassing Certificate	82
 Declaration of Scientific Integrity	 85

1

Introduction

Imagine a world where ordinary glasses can discreetly record conversations and capture images or videos without your knowledge or consent. With the rise of augmented reality (AR) glasses, this is the reality we are entering. These devices are part of the development of the Internet of Things (IoT), alongside technologies such as voice assistants, smartwatches, and smart home gadgets. While they offer many conveniences for our daily lives, they also raise privacy concerns. Incidents of baby monitors being turned into spy devices or doorbells sharing video with third parties showcase the potential risks of these technologies. Unlike standard IoT devices, AR glasses introduce unique privacy concerns due to their ability to continuously record surroundings and transmit visual data and audio information from the wearer's perspective to remote servers. For instance, a recent report of Meta Ray-Ban glasses being used for facial recognition without consent Nguyen and Ardayio [26] illustrates how such devices could violate privacy and increase surveillance risks.

While existing research has explored privacy risks, focusing on a range of IoT devices, there remains a scarcity of studies examining the privacy implications of specific wearable AR devices. This thesis addresses this gap by analyzing the Meta Ray-Ban glasses, an AR wearable IoT device developed by Meta Platforms, Inc. (formerly Facebook) from Menlo Park, CA, USA.

Studying the Ray-Ban Meta glasses presented several challenges. The proprietary nature of the glasses, in which manufacturers limit access to internal components, software, and data processing mechanisms, makes it difficult to understand and examine their data collection and processing mechanisms. This lack of transparency made it challenging to evaluate privacy risks effectively. Furthermore, the encryption protocols used in the network traffic, such as Transport Layer Security (TLS) and security techniques such as certificate pinning, protect the content of transmitted data. TLS encrypts the transmitted content, ensuring that unauthorized parties cannot read it, while certificate pinning prevents interception of adversaries by rejecting connections to unverified servers. Although these measures enhance security, they also make it challenging to analyze the specific information being sent to external servers or third parties.

In light of these challenges, this thesis applies a combination of traffic and app analysis to examine the privacy implications of the Ray-Ban Meta glasses. By analyzing network

traffic captured under a range of usage scenarios and investigating its companion app, Meta View, this thesis aims to identify and understand what data is transmitted or possibly collected, its destinations, and its potential effects on user and bystander privacy. This focused approach contributes to a more complete understanding of the unique privacy risks of wearable AR devices and provides a basis for developing effective mitigation strategies.

The remainder of this thesis is organized as follows. Chapter 2 presents the background and motivation for studying AR glasses, including an overview of relevant privacy and security concepts. Chapter 3 reviews related work in the field, focusing on prior works on AR devices and their privacy concerns. Chapter 4 outlines the hardware and software components of the Ray-Ban Meta glasses. Chapter 5 describes the methodology for data capture and analysis as well as their limitations. Chapter 6 provides a static analysis of the companion app, Meta View, exploring permission requests and potential data flows. Chapter 7 discusses the network-level analysis for traffic captures from the experiments, and Chapter 8 discusses Bluetooth communication. Chapter 9 details the decryption attempts made and the analysis of the endpoints successfully decrypted. Chapter 10 discusses the key findings of the privacy risks and challenges encountered. Finally, chapter 11 presents the conclusion and proposes future work in this field.

2

Background

AR wearables introduce unique privacy challenges for both the wearer and bystanders. This chapter explains how AR devices operate and the network communication protocols involved, establishing the concepts needed to understand the privacy implications of the Ray-Ban Meta glasses. It also introduces the scope of this thesis.

Augmented reality (AR) combines digital information with the real-time user environment, meaning that it provides a real-world environment with generated perceptual information overlaid on top of it. AR wearables are becoming increasingly sophisticated; for instance, the Ray-Ban Meta glasses have built-in cameras, microphones, and allow for hands-free interactions and real-time analytics. However, these components also allow for continuous data capture from the wearer's perspective, raising concerns about how and when data is being collected, especially for those individuals who are unaware and, hence, do not have the ability to give consent to being recorded.

While traditional Internet of Things (IoT) devices may gather metrics in fixed environments, the Ray-Ban Meta glasses and other AR wearables introduce unique challenges to private and communal privacy. The ability of the Ray-Ban Meta to continuously collect data from the wearer's perspective as they are moving and forward that information to its servers or external services, along with a lack of clarity about the information being gathered, raises ethical concerns about user privacy.

Moreover, most AR wearables, including the Ray-Ban Meta glasses, rely on a companion app for functionalities, including initial set-up, authentication, firmware updates, and communication with cloud providers or other services. The Meta View app, for instance, can pair with the glasses, import photos or video recordings from the glasses, manage preferences, and potentially share that data with its servers, which leads to a violation of confidentiality as data might be used for analytics beyond the user's awareness. Many devices also rely on external servers, which raises additional concerns, as they may process and retain data for extended periods, making it unclear who has access to user information in the long term or whether it is shared with advertisers. Whether or not the information is shared with advertisers or only with internal analysts, they expand the attack surface of users to a broader network.

When compromised and intercepted, these communications could expose personal in-

formation about the user or bystanders to adversaries. Many AR devices use encryption protocols, such as Transport Layer Security (TLS) and Quick UDP Internet Connections (QUIC), to protect the payload content in transit. However, the accompanying metadata, including packet sizes, timing, and connection endpoints, can still offer insights into user behavior and application usages, allowing attackers to deduce application usage when the user is transmitting data and infer user activities. A blog post, Schwendner [31], demonstrated vulnerabilities in the previous generation of the glasses, the Ray-Ban Stories, which allowed interception and decryption of transmitted data. Since then, Meta has implemented improvements to fix those vulnerabilities and secure the communication channels. The blog illustrates how, even with encryption, it is possible to disclose information due to vulnerable or insecure communication channels.

In addition to network traffic, the Ray-Ban Meta glasses use Bluetooth Low Energy (BLE) and Classic Bluetooth for pairing and peripheral management with the companion app. While BLE provides functionalities, attackers monitoring the local environment can capture BLE advertisements, which can expose device names, MAC addresses, and other identifiers, which could link specific wearables to an individual, correlating activities that enhance user tracking. Bluetooth traffic may also contain partial handshakes that inadvertently reveal device capabilities or usage patterns.

Privacy implications extend beyond the wearer but also to those who inadvertently have their images or details collected by the glasses. While the wearer may have consciously decided to use the glasses, bystanders are rarely notified that audio or video recordings are being made. From a bystander’s perspective, continuous recording in public spaces confronts conventional notions of informed consent. In addition, bystanders may be subjected to their image or information being sent or shared to cloud providers or advertisers for purposes, including analytics, without any knowledge. The data being sent or shared could also be used in machine learning classifiers or face recognition engines, thus creating more privacy concerns, such as user profiling. Therefore, it is important to have more explicit user and bystander controls, along with stronger privacy protections for both users and bystanders.

This thesis investigates the potential privacy implications of these factors on the Ray-Ban Meta on users’ glasses and bystanders by analyzing the device’s hardware design, companion app architecture, third-party integrations, and communication protocols. The study covers the static analysis of the app, exploring permissions and code structures that may indicate data flow to external services, network traffic examinations, identifying what is transmitted under different usage scenarios, Bluetooth communications, evaluating device pairing and peripheral interactions for possible vulnerabilities, and implications for bystanders, addressing the privacy concerns for individuals who may not have consented to AR data collection.

3

Related Work

Privacy and security challenges in augmented reality (AR) devices have gained increasing attention in recent years, with some challenges identified by Roesner et al. [29]. These challenges of AR technologies raise concerns about the potential misuse of data collection and sharing and the privacy of both users and bystanders. AR glasses, in particular, have the ability to continuously record and collect data, thereby blurring the lines between personal and public spaces. Prior work has explored and addressed these challenges from various angles, including user concerns and expectations about privacy, network traffic analysis, companion app analyses, and bystander implications. The following sections detail existing research to provide context for the privacy analysis of the Ray-Ban Meta glasses within the broad scope of AR privacy research.

User Privacy Concerns in Augmented Reality

There is extensive prior research on AR privacy that explored user perceptions of privacy and data collection of AR devices. When designing effective privacy protections, it is crucial to understand user perceptions and expectations about privacy. For instance, Gallardo et al. [21] explored speculative privacy concerns regarding AR glasses data collection. Through the interviews, they found that people expressed discomfort with capturing specific data types (for example, facial images), certain subjects whose data was captured (for example, children), or having data sharing that involves advertisers or unknown third parties. Hadan et al. [22] broadened this scope by exploring user coping strategies in immersive extended reality, emphasizing that individuals often feel compelled to develop personal or technical measures to mitigate privacy risks when devices or services do not offer sufficient protection. Another paper, from De Guzman et al. [17], examined privacy and security challenges in mixed reality, identifying vulnerabilities such as location tracking, biometrics data exposure, and inadequate user consent mechanisms. These findings emphasize the need for secure data collection, storage, and sharing that align with user expectations of privacy in AR devices.

Companion App Privacy Risks

Many Internet-of-Things (IoT) devices, including AR wearable devices, rely on a dedicated companion application to function fully. In the case of the Ray-Ban Meta glasses, the Meta View app is the companion app for the glasses. It plays a central role in device management, configuration, and data retrieval. These apps can introduce additional privacy risks. Nan et al. [25] conducted a large-scale analysis of data exposures through the companion apps of IoT devices. They found that IoT companion apps often include third-party libraries and extensive permissions that can result in unintended data sharing with advertising and analytics platforms. This concern is particularly relevant for AR glasses, where the companion app configures the glasses, controls media and data transfer, and may similarly request sensitive permissions and share data with external entities. FTC [12] criticized major social media platforms, including Meta, for their broad surveillance practices involving collecting and retaining large amounts of user and non-user data. Such reports raise questions about how the companion app may facilitate extensive data gathering and retention. These issues are applicable to this thesis, which evaluates the Meta View app in terms of permissions usage, third-party integrations, and potential data transmission to external entities.

Network Traffic and Data Exposure

Analyzing network traffic is a critical step in evaluating IoT device security. Fereidouni et al. [20] provided an overview of IoT and discussed the risks posed by Man-in-the-Middle attacks in IoT systems, showing how such attacks can intercept and manipulate data exchanges. To mitigate such risks, most AR devices use encryption to keep the data unreadable to potential attackers. However, Ren et al. [28] demonstrated that encryption alone does not necessarily protect user privacy. They investigated information exposure through IoT device traffic and demonstrated that, encrypted or not, the traffic can leak metadata, allowing third parties to identify patterns and infer user behavior. Furthermore, Acar et al. [13] also revealed that even with encrypted traffic, adversaries could exploit traffic patterns, timing, and metadata to figure out user activities and behavioral patterns.

Focusing on wearable IoT devices, D'Mello et al. [19] reviewed technical and policy perspectives on wearable IoT security. They accentuated that encryption, while crucial, is often inadequate on its own due to issues like improper key management and weaknesses in IoT communication standards. Therefore, concluding that encryption must be complemented by secure key management, rigorous testing for protocol weaknesses, and robust privacy policies that limit data retention.

Beyond network transmissions, many AR wearables, including the Ray-Ban Meta glasses, use Bluetooth connections for pairing or device management. Studies of Bluetooth communication, such as that by Nagrae et al. [24], focused on uncovering vulnerabilities in Classic Bluetooth and Bluetooth Low Energy (BLE), including insufficient encryption and inadequate authentication mechanisms.

The current thesis builds upon these findings by examining the security and privacy of the Ray-Ban Meta glasses' network and Bluetooth transmission. In particular, the thesis addresses how data related to user interactions may be sent through network and Bluetooth

communications and the security of such communication channels.

Bystander Privacy Implications

AR devices raise unique ethical and privacy concerns for bystanders, who may be inadvertently recorded, often without consent. Gallardo et al. [21] found that transparency and consent measures for bystanders remain insufficient in many commercially available AR products. Similarly, studies from Denning et al. [18] investigated bystander reactions to AR glasses and showed that bystanders are often uncomfortable with the lack of transparency about when data is being captured or recorded in shared spaces. Insecure communication protocols increase these risks by potentially exposing captured bystander data to interception or misuse, as Nagrale et al. [24] discussed in their analysis of BLE threats. These insights align with one of the goals of this thesis, which is to determine whether the Ray-Ban Meta glasses adequately protect not only the user's information but also that of bystanders whom the glasses may have recorded.

Privacy in Wearable AR Devices: A Comparative Perspective

A paper by Kotsios [23] explored the privacy challenges in AR technologies, specifically Google Glasses, which included its ability for continuous recording, facial recognition, and data access, and the impacts on bystanders. It argued that while AR technologies have great potential, they also require a framework that protects both users and bystanders from unauthorized data collection and surveillance by governments and adversaries or external parties. Another study, by Safavi and Shukur [30] on Google Glasses showed that Google could not reassure its users that the data collected and stored is kept safe and therefore susceptible to unauthorized access, leading to exposure of sensitive data. However, most studies on AR privacy have taken a broader approach, where they analyze a variety of devices in order to provide a general pattern of data collection, storage, and transmission. Examples include works mentioned above by Ren et al. [28], Denning et al. [18], and Acar et al. [13], which illustrate the privacy risks in AR devices, whether it be user consent or network vulnerabilities. However, device-specific investigations remain relatively rare, potentially resulting in an incomplete understanding of that device's privacy risks. This thesis narrows the focus to the Ray-Ban Meta glasses to analyze this specific device's privacy and security considerations. In doing so, the thesis complements the broader studies by bridging the gap between general discussions of AR privacy and the in-depth exploration of privacy for a particular device.

4

Device Overview

This chapter provides an overview of the Meta Ray-Ban glasses, detailing their hardware and software specifications and network capabilities. A preliminary review of the device's hardware and software specifications was conducted using official documentation, such as that from Meta [5] or from Ray-Ban [7], online resources, and hands-on usage of the glasses. The purpose is to gain practical insight into their functionalities, such as media capture and voice assistant interactions. This informed the design of experiments introduced in Chapter 5, which focused on evaluating the privacy and security of the glasses.

4.1 Hardware Specifications

This section details the key hardware features of the glasses, including its camera system, audio capabilities, controls, and more. These components enable core functionalities such as media capture, audio playback, and voice interaction.

Camera and LEDs

This subsection covers the integrated high-resolution camera with visual indicators in the Meta Ray-Ban glasses, which supports media capture while providing feedback on the device's status.

The glasses feature an ultra-wide 12 MP camera, capable of capturing high-resolution photos (3024 x 4032 px) and videos in 1080p. Located in the upper left corner of the front frame, as shown in Figure 4.1, the camera supports taking photos and recording videos, which is one of the core features of the glasses. The maximum length of the video is adjustable depending on user preference, with the length ranging from 15 seconds to 3 minutes. The captured media can be uploaded to the phone via the Meta View app for storage or sharing. The camera also supports live streaming to social media platforms such as Instagram, with a maximum stream duration of 30 minutes per stream.

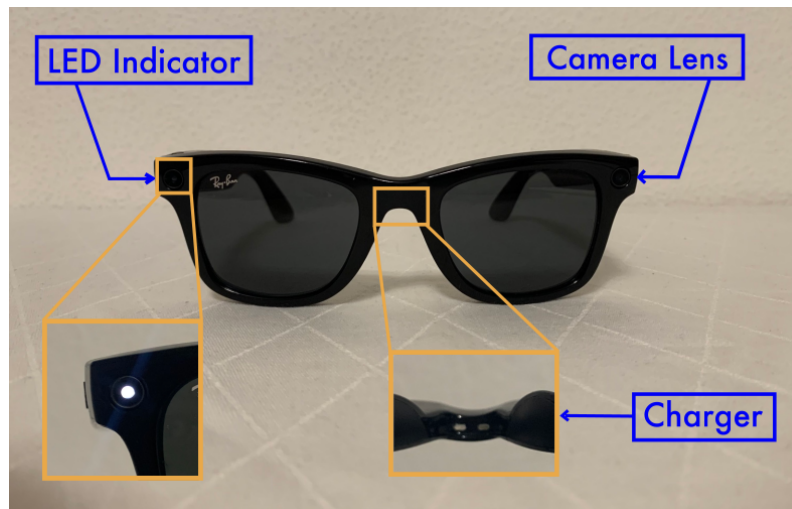


Figure 4.1: Front view of the Meta glasses

A white indicator LED in the upper right corner of the front frame of the glasses lights up during photo or video capture. Its sole purpose is to inform bystanders so that they are aware of when a photo is being taken or when there is an active recording. If covered, the glasses will not allow the user to initiate any capture, emitting an audio alert to the wearer to remove the obstacle that covers the indicator LED before trying to capture. The glasses also have a second notification LED in the inner right corner (Figure 4.2), giving the user visual feedback on various device statuses, including power, pairing, battery level, and more, using different colors and blinking patterns.

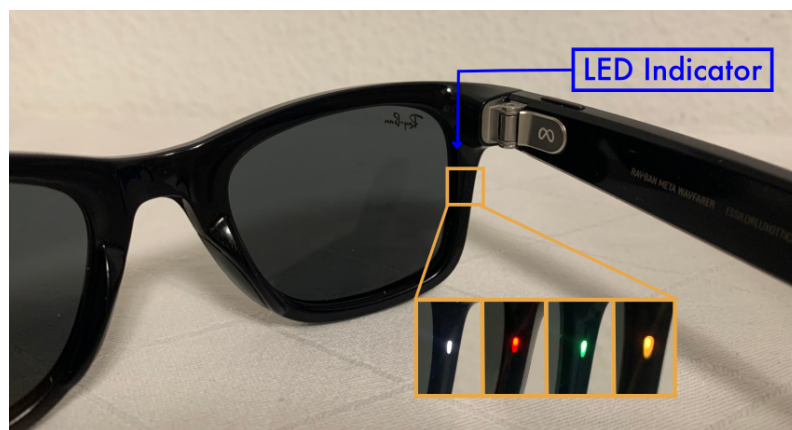


Figure 4.2: Right inner view of the Meta glasses

Interaction Features

This subsection looks at the key interaction capabilities of the Meta Ray-Ban glasses, including audio playback, intuitive controls, and sensor-driven functionalities with responsive design.

The glasses feature integrated speakers at the temples, as seen in Figure 4.3. These speakers support platforms like Spotify for streaming audio. A five-microphone array, which

improves audio quality and reduce white noise. Combined, they support interactions, including talking to the AI assistant, audio capture during video recordings, and hands-free communication on platforms such as WhatsApp or Messenger.

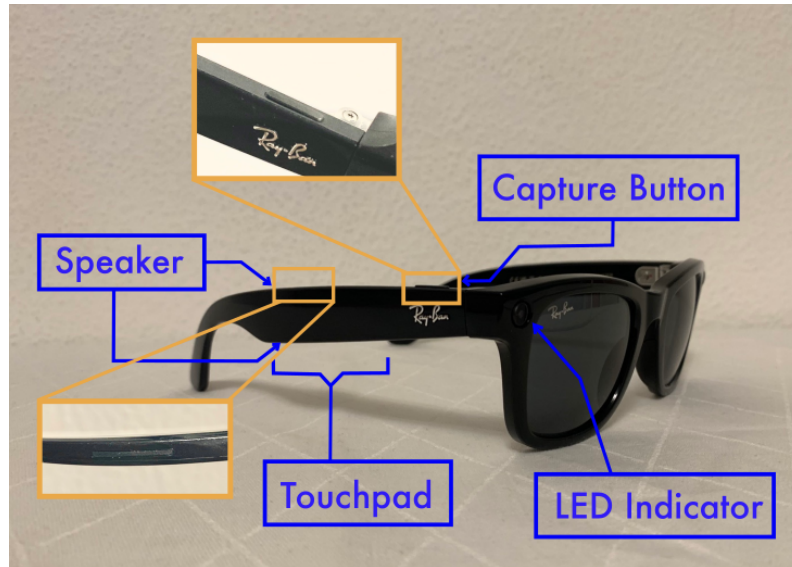


Figure 4.3: Right outer view of the Meta glasses

The touch features of the glasses, seen in Figure 4.3, include a touchpad on the outer side of the right arm. This allows the wearer to control functionalities such as music playback through gestures such as swiping or tapping. When these gestures are performed, there will be a slight vibration and audio feedback, which is part of the responsive design of the glasses. A capture button on the top right arm allows the wearer to take photos by short-clicking the button or video recordings with a long press on the button. Furthermore, a power toggle on the inner corner of the left arm (Figure 4.4) allows users to power the device on and off, with a visible red dot on the toggle indicating the off state.



Figure 4.4: Left inner view of the Meta glasses

The glasses are equipped with sensors positioned along the inner arms that detect

when they are being worn or removed. These sensors enable features such as automatically pausing music when the glasses are taken off. Small vibrations and audio feedback notify the wearer when the glasses are being worn.

Performance and Power

This subsection describes the glasses' processing power, storage capabilities, and power management features, which ensure and enhance usability for tasks such as media capture, voice interactions, and real-time data processing.

The core of the glasses is the Qualcomm Snapdragon AR1 Gen 1 processor, which manages the sensors, handles image processing, and provides AI capabilities. It also has 32 GB of built-in storage, allowing the glasses to save photos and videos locally without requiring immediate synchronization with the companion app. In order to support these capabilities, the glasses have an accompanying charging case. The case charges the glasses through the nose bridge. The case includes a USB-C port for recharging its lithium-ion battery. An LED indicator on the back of the case signals the pairing and battery status, while a pairing button helps the pairing process with the phone. The compact, portable charging case ensures that the glasses remain powered throughout the day.

4.2 Software Specifications

The Ray-Ban Meta's software capabilities include features provided by the companion app, firmware updates, and device compatibility. This section introduces the software specifications that support the glasses, their usability, and some limitations.

Device Compatibility

The glasses rely on the companion app Meta View, developed by Meta Platforms. It is available for iOS (14.4 or later) and Android (10 or later). Officially compatible devices are limited to a certain selection of Apple, Google, and Samsung phone models. While the app may still function on non-supported phones, certain functionalities may not be feasible, or certain features may not work as intended. The exact limitations experienced during the thesis are detailed in the later chapters.

Functional Capabilities

The Meta View app provides media synchronization, allowing the user to take photos and videos at any moment and sync them to the phone whenever they want, as well as customization of preferences, including audio controls, camera options, voice assistant settings, and more. The glasses include a voice assistant, activated by the phrase "Hey Meta," which is also customizable. The voice assistant supports hands-free tasks such as checking battery status, messaging, and making or receiving calls. Furthermore, the app manages firmware updates over the air (OTA), ensuring the glasses are up-to-date with the latest security patches and features.

Software Limitations

The initial experiments were performed with the companion app on a Xiaomi Redmi Note 8 running Android 14. This phone was able to perform most functions with no issues. However, live streaming did not work as the app would crash when trying to live stream with glasses. This phone model also had an extremely unstable connection, with the glasses often having problems where it could not transfer the captured media to the phone. Fixes to resolve these issues include unpairing and repairing the glasses, factory resetting the glasses, and deleting and reinstalling the app.

The decryption efforts on network traffic were made on a Xiaomi POCO X5 5G phone running Android 14 with the security update dated October 2024. The limitation of this phone was much more noticeable as a simultaneous connection of Bluetooth and the app was not possible. As a result, the user can pair the glasses with Bluetooth in Settings or connect to the app, but not both simultaneously. This limitation was much worse and impeded the ability to perform the experiments. Therefore, the final experimentation was conducted on a Samsung Galaxy S24 running Android 14.

4.3 Network Capabilities

The glasses have Wi-Fi 6 (802.11ax) technology, operating on both the 2.4 GHz and 5GHz frequency bands. It facilitates media transfer from the glasses to the companion app. Bluetooth 5.2, operating in the 2.4 GHz frequency bands, is used for the initial pairing and other controls such as music playback. More specifically, BLE is only used for the initial pairing of the devices, while Classic Bluetooth is used for audio streaming and other functions such as device management.

4.4 Privacy and Security

This section addresses the privacy risks, data-sharing practices, and security measures implemented to protect users and bystanders. The issues mentioned are only based on the physical device and are discovered from hands-on usage with the glasses.

Privacy Concerns for Users

The glasses allow users to listen to messages, send messages, and make calls through the AI assistant. When a message is sent to the user, the glasses allow the wearer to ask the Meta AI to read it aloud directly. It is not required to receive a message to use this functionality; users can command the AI to read all recently received messages. In order to send messages or initiate calls through the AI assistant, the user only needs to know the contact's name.

By default, these functions work when the paired phone is locked. Since the glasses do not incorporate any voice authentication or verification, anyone with physical access to the glasses can exploit these functionalities. For example, an unauthorized individual could command the AI to read messages, revealing the sender's name and the content of the message. This not only exposes sensitive information but also allows the individual to reply

or call the sender, posing a privacy risk to both users and others. Having recognized these concerns, Meta implemented new privacy measures in June 2024 to protect the privacy of the users, where users now have the option to turn on the restriction that requires the phone to be unlocked to use these functions.

Privacy Concerns for Bystanders

The LED indicator is intended to notify and warn bystanders during media capture. However, there are some notable issues with this bystander privacy measure. First, in a bright outdoor environment, such as under the sun, the light is not very visible, which reduces its effectiveness as a visual warning to bystanders. Additionally, there exists a method to bypass this mechanism. This privacy feature is only checked during the start of a media capture, meaning if a user covers the LED and tries to start a video recording, the glasses will refuse to record. However, if the user starts a video recording with the light left uncovered and then, during capture, obscures the LED indicator, the glasses will continue to record without interruption as if the indicator is not covered for up to three minutes. This loophole reduces the effectiveness of the LED indicator as a privacy mechanism for bystanders.

Data Sharing and Management

The companion app acts as an intermediary, collecting user data such as interactions and voice commands from the AI assistant and transmitting it to Meta servers for processing. The new AI-driven features include asking the AI to identify objects, translate text, or upload captures made immediately onto Instagram or Facebook. All of which rely on cloud-based processing. This means the data is analyzed remotely, and the results are then sent back to the glasses.

On the app, users can manage their data-sharing preferences. However, the selection is limited, and users can only choose to either share or not share additional data. Meta stated that any images sent to the cloud and processed with AI and the captures made and sent via the glasses onto Instagram are automatically stored on Meta servers, meaning they are not considered additional data. The company claims that these data help Meta to improve their products and train their AI models. This data sharing is particularly concerning for users in the United States (US), where opting out of such data sharing is currently not an option. In contrast, users in the UK and the European Union (EU) are theoretically allowed to opt-out, given a justification. However, Meta retains the discretion to reject these requests, which raises questions about the fairness and transparency of their data policies.

Security Measure

On a more positive note, the glasses are designed to pair with a single device, regardless of the user account. This means that even with the same user account for the companion app login, pairing the glasses with a new phone requires a factory reset before connecting to the new device. Factory resetting the glasses erases all information stored on the glasses. This is a safety measure that ensures users that if their glasses are stolen or lost, others

cannot directly access locally stored data by pairing them to another device with a Meta account. When attempting to connect the glasses to a new device, the glasses require a full factory reset. This process erases all images and data locally stored in the glasses, providing a layer of security for the user. The glasses also have end-to-end encryption for secure data transmission between the glasses and the companion app on the paired phone.

5

Methodology and Experiments

This chapter describes the experimental design, setup, and execution for capturing, analyzing, and understanding the traffic generated by the Ray-Ban Meta glasses and its companion app. It also addresses the limitations encountered during the process.

5.1 Experiment Design

The experiments aim to identify the privacy risks associated with the Ray-Ban Meta glasses and its companion app. The goal of the study includes identifying the types of data generated by the glasses and app, examining the encryption practices, determining the destinations of the transmitted data (geological and corporational), and interactions with third parties, such as advertisers and trackers, and assessing whether these transmissions include personal information about users or bystanders. An additional focus is paid to the permissions requested by the companion app and third-party libraries used by the app for its potential privacy risks as it may expand the data sharing beyond user awareness.

The experiments were designed to prioritize interactions between the Ray-Ban Meta glasses and the companion app on the phone, with efforts to minimize external interference and ensure the capture of relevant traffic. However, it is important to acknowledge that this controlled environment may not fully replicate the complexity of real-world scenarios.

The glasses rely on both network and Bluetooth connectivity, and upon that, each communication serves primarily different purposes. Therefore, the final experiment design has separate captures and analysis methods for the different communications. Finally, all captures were also made with bystander privacy in mind, where the photos and video recordings were taken facing a white wall with no third-party voice other than the audio streaming from Spotify captured during any voice recordings.

The research initially aimed to analyze the decrypted traffic for sensitive data such as raw readings. A man-in-the-middle (MitM) setup using mitmproxy was used to intercept and decrypt HTTPS streams for network traffic. As outlined in [20], IoT devices often lack solid security measures to protect transmitted data. However, the Meta View app uses certificate pinning, which limits this approach. Other tools, like Frida and Objection, were then used to try and bypass the pinning, but the strong certificate pinning remained.

These technical limitations prevented full decryption, resulting in the analysis focusing on metadata, traffic patterns, and destinations to assess privacy risks.

5.2 Experimental Setup

The environmental setup used to conduct the experiments are detailed in this section. It outlines the hardware and software components involved, including the testbed and tools used to capture the traffic generated by the glasses and companion app.

Tools Used

Wireshark [9], an app that captures data packets in real time, was the primary tool for monitoring network traffic and Bluetooth communications. It provides a packet-level view of the data exchanged to examine the traffic patterns, protocols, and information transmitted between the glasses and the phone. An ALFA network adapter (Long-Range USB Adapter of model AWUS036NHA) was used to create a Wi-Fi hotspot, allowing the laptop to act as an intermediary to route network traffic and capture it for analysis. Classic Bluetooth captures were obtained using the Ubertooth One [8], a sniffer device with firmware version 2024.10. Meanwhile, the nRF52840 Dongle [10], another sniffer device, was used to monitor BLE traffic. A man-in-the-middle proxy (mitmproxy [16]) was used to intercept and decrypt network traffic to analyze requests, responses, and payloads, though certificate pinning restricted much analysis. An additional phone was used as a recording device to have an auditory record of the experiments.

For the analysis of the captured traffic, visualizations were generated post-capture using custom Python scripts. The analysis of the companion app was done using Mobile Security Framework (MobSF) [14], an automated security assessment tool for mobile applications. It identifies permissions and third-party integrations embedded within the Android Application Package (APK), a file format that is used to install an application on an Android operating system.

Testbed

The primary components in the testbed for the experiments included a laptop running Ubuntu Linux version 24.04, a hotspot created using an ALFA network adapter connected to the laptop, the Ray-Ban Meta glasses, and a Samsung S24 smartphone running on Android 14 as the main device. The hotspot only had the Samsung S24 connected to isolate the traffic captured through Wireshark. For Bluetooth communications, a sniffer was plugged into the laptop via USB to capture the traffic. The Bluetooth sniffer nRF52840 Dongle was used for BLE experiments, and the Ubertooth One was used for Classic Bluetooth experiments. For network traffic, the capture option for the experiments was set to the hotspot, and for BLE traffic, the interface was set to the dongle to capture traffic on those interfaces.

Three phones were used throughout the thesis. During the earlier phases, two unsupported phones, a Xiaomi Redmi Note 8 and a Xiaomi POCO X5 5G, were used to explore device functionalities, test preliminary setups, and attempt traffic decryption. However, due

to the instability and compatibility problems, a third phone was introduced, a Samsung S24, which was an officially supported phone model. All final experiments were performed on this phone, ensuring consistency and comparability in the results.

5.3 Data Collection

This section covers the categorization of traffic data generated by the Ray-Ban Meta glasses and companion app. The collected data was split into two main categories: network traffic and Bluetooth traffic. The data in each main category was then further divided based on the need for analysis and experimentation, as seen in Figure 5.1.

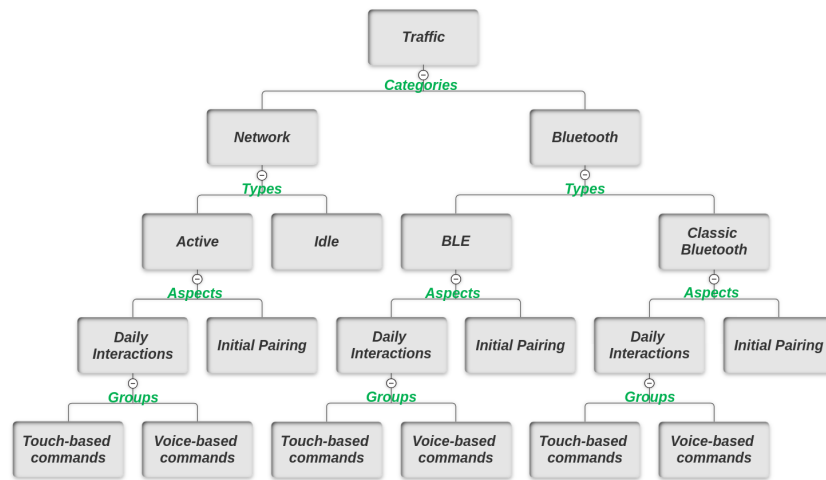


Figure 5.1: Categorization of the types of Traffic generated by the Ray-Ban Meta glasses and the companion app

Network Traffic

For Network Traffic, data was separated into Idle and Active traffic. Idle traffic involves no direct user interaction and was captured while the glasses remained idle for an extended period of time. It is a controlled test that helps identify persistent background data flows. The second type is active traffic, which refers to the traffic captured whilst interacting with the glasses and companion app.

From initial experimentation, it was noted that the Meta View app does not allow pairing when there is no internet connection. Therefore, active traffic is divided into two subcategories: traffic from daily interactions, such as taking a photo or asking the Meta AI about the time, and initial pairing traffic, which is exchanged between the phone and glasses when establishing the connection between the phone and glasses. Furthermore, the preliminary overview and hands-on usage of the glasses identified two types of commonly performed daily interactions. Thus, daily interactions are further grouped into two types of commands: touch-based and voice-based commands. Touch commands execute common user interactions through manual inputs, while voice commands are carried out through the AI voice assistant, allowing for a broader range of interactions during experimentation.

Bluetooth Traffic

Bluetooth Traffic was split based on its type. The first is Bluetooth Low Energy (BLE), which is designed for low-bandwidth, energy-efficient communication and is typically used for the initial pairing of devices. The second is Classic Bluetooth, which supports higher-bandwidth communication and is used for continuous data streaming, such as audio playback.

Bluetooth traffic is event-driven, meaning that the traffic captures are primarily during active interactions with the glasses and phone. Unlike network traffic, which may include background traffic even when idle, Bluetooth traffic is less likely to have meaningful data when kept idle; therefore, there is no idle traffic experiment for Bluetooth. Nonetheless, the data for both types of Bluetooth traffic was divided into user interaction and initial pairing of the devices, of which user interactions are then grouped based on the type of interaction, touch-based or voice-based commands.

5.4 Experimental Procedure

This section describes the process of setting up the environment and capturing the generated traffic. The experiments were repeated multiple times to ensure better reliability. Each round of experiment capture is referred to as a trial in the analyses. The following subsections first introduce the experimental procedure for different traffic types (network and Bluetooth) before discussing further details for each experiment in their respective subsections.

Network Traffic

The environment was set up with only the phone connected to the hotspot. The Ray-Ban Meta glasses were then powered off, and the companion app was forced to quit to kill its process. The experiment began by simultaneously starting a Wireshark capture on the laptop and the voice recording on the extra phone. The Wireshark was set only to capture traffic going through the hotspot. The procedure for the remaining process of the different experiments, including active interactions and initial pairing, will be discussed in the following subsections. The experiments for active interactions with the network traffic were repeated five times each.

Bluetooth Traffic

The only difference between the setup for Classic Bluetooth and the BLE was the sniffer used to capture the nearby Bluetooth traffic. Classic Bluetooth used the Ubertooth one, whilst the nRF52840 was used for BLE. The respective Bluetooth sniffers were plugged into the laptop via USB. The Ray-Ban Meta glasses were powered off and the companion app was force quitted. The Ubertooth one is started via a command on the terminal, whereas the nRF52840 Dongle was started by selecting its interface in Wireshark for the capture. The experiment began by simultaneously starting the Bluetooth capture using the sniffers and voice recording. The following subsections will discuss the procedure for the remaining process of the different experiments for active interactions and initial pairing. All

experiments for Bluetooth traffic were repeated twice.

Touch-based commands

The touch-based commands mainly focused on Media Capture and Music Playback. After the setup of the environment, a predefined list of commands was executed manually. This list can be found in Table A.4. The execution order started with Media capture, which was then imported onto the phone. Next, the commands for Music Playback were performed, starting with playing music. Up until this interaction, each command was given at thirty-second intervals. The time interval allows for sufficient time for the traffic to stabilize between each interaction. Afterwards, the rest of the Music Playback commands were given in fifteen-second intervals. The duration of each touch-command experiment was roughly five minutes.

Voice-based commands

Voice commands have a wider range of functionalities than touch commands. The different functionalities of voice commands include General Questions, Music Playback, Timers, Media Capture, Messaging, and Phone Calls. Similarly, before the start of the experiment, the environment was set up. The specific set of commands executed can be found in Table A.5. All voice commands were performed in thirty-second intervals, ensuring consistency. Thereby making it easier to compare and observe patterns. An automated voice recording delivered the initial series of voice commands.

The specific sequence of functionalities that used the automated recording includes General Questions, Music Playback, Timers, and Media Capture. The remaining commands of the experiment were performed manually with the same thirty-second interval per command. The transfer of captured media from the glasses to the phone was the first manual command executed, followed by the commands for Messaging and Phone Calls. The duration of each voice command experiment was roughly eleven and a half minutes.

Initial Pairing

Pairing Sequence handles traffic during the initial pairing of the glasses and the companion app. This experiment had one more prerequisite in the setup before starting the experiment. The glasses needed to be unpaired and forgotten on the phone in Bluetooth settings. The experiment then began when the glasses were powered on and the app opened. The interaction then continues with the glasses being paired with the phone. The glasses are then unpaired and forgotten again before re-pairing. Each pairing round took roughly one minute. For this experiment, the unpairing and re-pairing actions were repeated five times but in the same traffic capture. This means that the five repetitions of pairing between the devices were done in one go, whereas the experiments for the touch-based and voice-based commands had five separate captures, one for each repetition. The duration of the initial pairing experiment was around five and a half minutes.

Idle Traffic

A single capture of approximately eight hours took place throughout the night to observe whether the glasses communicated with the Meta servers or other endpoints. No interactions were done on the phone or the glasses. The experiment started with the phone and glasses being powered on and left in proximity while running the Wireshark capture.

5.5 Analysis Methods

The captured traffic was analyzed offline after all experiments were completed. This section describes the process and methods used to analyze each type of traffic. Wireshark was used primarily for packet inspection, while the Python scripts generated visualizations for the traffic volume, packet sizes, protocol usage, and destination IP mapping for each experiment. The captured data was exported as a CSV file, aggregated in Python, saved as a CSV file, and subsequently visualized using standard Python libraries for analysis.

App Analysis

The companion app for the Ray-Ban Meta glasses was analyzed using the static Mobile Security Framework (MobSF), a tool specifically designed for analyzing mobile applications. It decompiles Android applications' APK files to inspect their source code, resources, and configuration files. This tool detects hardcoded credentials, requested permissions, and integration of third-party libraries. For the app analysis, the components found were divided into four types. The first are first parties, which refers to the components or elements provided by Meta. Support parties refer to those that come from the same parent organization, Meta, and support functionalities and features of the app but are external to the app itself (e.g., WhatsApp). Finally, third parties are those that are not provided by Meta Platforms but instead by an external, independent organization (e.g., Google). The fourth is platform-provided, which are components provided as part of the Android development framework or operating system (e.g., AndroidX).

Network Traffic

In general, the network traffic captured is divided into four different parties in the analysis. The first is private traffic, which includes local broadcasts or traffic directed to the Samsung S24 phone. The second is first-party traffic, which refers to the destination addresses owned by Meta and is directly related to the app (e.g., the Meta servers). Support-party traffic refers to those that are sent to addresses that are owned by Meta but are external to the app (e.g., WhatsApp). Finally, third-party traffic refers to those that are not owned by Meta Platforms and are instead routed to an external, independent organization (e.g., Google). In order to distinguish whether traffic went to private, first-party, support-party, or third-party domains, the packets were grouped based on IP ownership and DNS resolution and then categorized accordingly.

Network traffic was also grouped based on its readability to understand the extent of information exposure better. The protocol was found to be an effective distinguisher

for assessing readability, making it the primary criterion for grouping the data. A packet is considered readable if most of the packet content is human-readable. It is considered partially readable if a minor part of the payload can be understood. Finally, it is considered unreadable if only the metadata is exposed and the payload is completely not human-readable.

The captured traffic showed some third-party hostnames that were labeled as Google. These are part of Google's global network, including `googleapis.com`. At the same time, some other hostnames were labeled as Google Cloud, which corresponds to the use of Google Cloud Platforms (GCP). These are often used by other applications and services for hosting or storage. This brings attention to a crucial differentiation. While some IP addresses may point to Google servers, the actual organization being communicated with may not be Google itself but rather a third-party service using Google's infrastructure. These addresses were reverse DNS resolved and analyzed within the captures to determine their associated services. In the captures, the main application found relying on Google Cloud for hosting and delivering services was Spotify.

Bluetooth Low Energy Traffic

The BLE captures revealed only a few MAC addresses, which is a positive indicator for the isolated environment created for the experiment. However, none matched the MAC address of the glasses or the phone. Assuming that the BLE communication between the phone and glasses was captured, this suggests that the devices are using randomized MAC addresses for communication. In order to find which addresses belong to our devices, the Advertising Data for each packet was extracted. In Advertising Data, there is a field in the packet called Manufacturer Specific, which includes a company ID that provides the manufacturer of the device sending that packet. The Company ID was used as a filter, and the following two IDs, Samsung Electronics Co. Ltd (0x0075) and Meta Platforms, Inc. (0x01ab), were found. These are the two manufacturing companies of the devices involved in the experiments. These IDs were used as a filter, for example, `btcommon.eir_ad.entry.company_id == 0x0075` to determine the number of MAC addresses involved for each ID. Only one MAC address was associated with each company, `5f:64:8e:90:8d:44` for the glasses and `SamsungElect_98:02:66` for the phone.

Classic Bluetooth Traffic

An overview of captures shows that packets lack a source and destination, which are typically used to identify communication endpoints. Therefore, in order to identify which Classic Bluetooth communication was from the glasses or the phone, the Lower Address Part (LAP) was extracted from the Bluetooth Pseudoheader in each packet. Bluetooth device addresses are unique identifiers assigned to a device by its manufacturer, typically 48 bits in length. The addresses consist of a 16-bit Non-significant Address Part (NAP) and an 8-bit Upper Address Part (UAP); these bits usually make up the identifier assigned to the organization. The last 24 bits are the Lower Address Part (LAP), a unique identifier the manufacturer assigns to a device.

The glasses are powered on at minute 0.5 for each experiment conducted, meaning that any traffic occurring before that time could not have originated from the glasses. For instance, Figure 5.2 displays the packet count for all addresses captured during the touch-based commands experiment. The address 0x0021c4f4, represented with the blue line, was captured within the first thirty seconds of the experiment. Since the glasses were not yet powered on, this LAP could not belong to them and was therefore eliminated as a candidate.

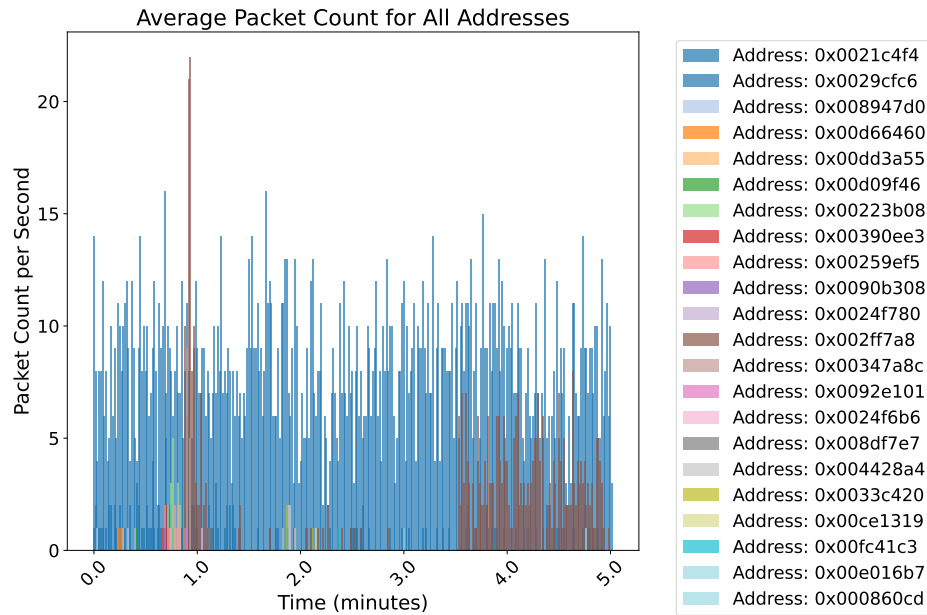


Figure 5.2: Average packet count for all captured addresses in the touch-based command experiment.

Another method used to filter the addresses involved the signal strength. Given that the glasses and phone were placed right next to the sniffer, they should have high signal strength. However, there was no RSSI value (Received Signal Strength Indicator) in the capture, as it is not provided by the Ubertooth one. This was a hardware limitation, which otherwise would have indicated the strength of the received signal at the receiver's end. Instead, a signal power (SP) value was available, measuring a relative signal level on a channel, which could be used to infer signal quality in place of RSSI. The signal power is examined to determine what addresses would make the most sense. The average signal strength for the experiment environment should be below -60 dBm due to the placement of the sniffer and the devices. The average signal strength for the LAPs, after filtering out those found within the first thirty seconds of the experiment, was extracted and depicted in Figure 5.3, which showed only two possible candidates for the devices.

In order to check which of the LAPs belonged to the glasses, the traffic for each LAP over time was tracked to determine whether it aligned with the interaction timings where a command was issued. Two LAPs were found to align well with the commands and each other, those being 0x00347a8c with an average SP of -23 and 0x002ff7a8 with an average SP of -38 dBm, as seen in Figure 5.3. These LAPs were then cross-referenced between

the different experiments and the trials within each experiment using, for example, `bt-bredr_rf.lower_address_part == 0x00347a8c` as a filter in Wireshark. There was a possibility that the devices used Bluetooth address randomization, which would allow a device to regularly change its Bluetooth addresses to prevent long-term tracking by others. Fortunately, all classic Bluetooth experiments performed were done in close periods to one another, and both LAPs were present in all captures.

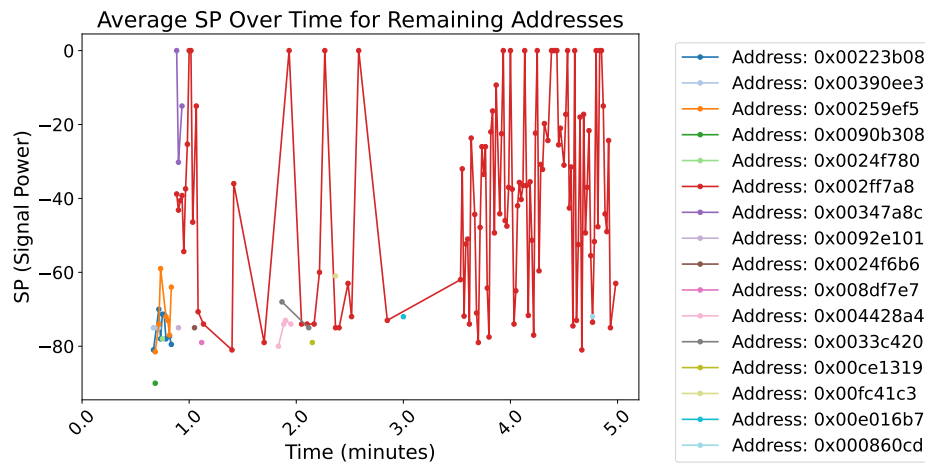


Figure 5.3: Average signal power (SP) over time for remaining addresses in the touch-based commands experiment after filtering.

The next question was to identify which LAP belongs to the phone and which to the glasses. However, note that the correctness of this assumption does not affect the analysis. Here, the traffic frequency for each LAP was used, which identified one LAP to be active only for five seconds, roughly when the glasses are paired, and the other LAP to be active a bit before the first LAP and continue to be active throughout the experiment.

Classic Bluetooth typically has a master-worker relationship in which one device coordinates the communication, and the other device responds to its commands and requests; therefore, the second identified LAP would be the master device. A phone would usually be considered a master device due to its controls and ability to connect to multiple devices. A worker device can only connect to its master device and cannot communicate with any other worker devices. However, this is only a general premise; therefore, the sequence of packets during pairing was examined to identify which LAP was the first to begin communication.

Since the master scans the surrounding devices and selects the worker device to connect to, it would make sense that the master device is the one to initiate communication. The captures confirmed that the second LAP is the master device, which is believed to be the phone since the phone initiates the pairing with the glasses. For certain operations that might need the glasses to be the master, it is important to know the design of Classic Bluetooth, where the master controls the communication timing and surveys the worker for data. Even if the glasses initiate commands or stream data, the phone could technically still be the master that is coordinating all transmissions.

Classic Bluetooth also allows for dynamic role swapping, which allows the worker to

have temporary control and may explain some commands where the glasses appear to be the ones giving the command. Therefore, it is the assumption that the phone is of the address 0x002ff7a8 and is the master device controlling the link and the glasses, originally on the address 0x00347a8c, to use the link of the phone for further communication.

5.6 Limitations

Several limitations affected the scope and progression of this thesis. First, the unsupported phone models proved to be much more of a problem than initially predicted. Second, the process of capturing the Bluetooth packets proved to be challenging. Lastly, the inability to decrypt the encrypted traffic restricted the analysis to mainly metadata and destination mappings. The following sections go into detail about the limitations of each aspect.

Phone

The initial experiments were performed with the companion app on a Redmi Note 8 running Android 14. This phone was able to perform most functions. However, live streaming did not work as the app would crash when trying to live stream with glasses. The Bluetooth connection between the glasses and the phone was also relatively unstable, with the phone having problems finding the glasses. This phone model also had constant problems with transferring media from the glasses to the phone. The transfer process involved the glasses establishing their own Wi-Fi network, acting as a Wi-Fi access point to which the phone connects for the duration of the transfer. As a result, fixes, including unpairing and re-pairing the glasses, factory resetting the glasses, and deleting and reinstalling the app, were common occurrences.

The decryption efforts on network traffic were made on a POCO X5 5G phone running Android 14 with the security update dated October 2024. The limitation of this phone was much more noticeable as a simultaneous connection of Bluetooth and the app was not possible. As a result, the user can choose to either pair the glasses with Bluetooth in Settings, which allows music playback, or connect to the app, which allows for media transfer, but not both at the same time. This limitation was far more detrimental to the ability to perform the experiments. The Bluetooth connection between the glasses and the phone was also relatively unstable as the glasses attempted to connect via Bluetooth but failed repeatedly due to the limitation that prevents simultaneous pairing of the devices over both Bluetooth and the network.

The third phone used was a Samsung S24 on Android 14. This phone is one of the phone models officially supported by the Meta View app and had none of the previous connection problems, which allowed for a much smoother experience. This phone model was selected to be used for the final experiments.

The final limitation of the phone was the geographical location. The Ray-Ban Meta glasses are available and supported in multiple countries, including the US, UK, Germany, and others. However, Switzerland is not one of those countries. Therefore, some features of the Meta AI were restricted. These features include asking the Meta AI to translate text,

set reminders, ask the AI to look at an object where it would try to identify it or ask the AI to scan QR codes or call a number placed within the camera view of the glasses. This limited the number of interactions that could be included in the experiment design.

Bluetooth Sniffer

Multiple sniffers were considered in the initial experimentation for Bluetooth sniffing. However, some could not reliably capture the communications, resulting in the decision to use the Ubertooth One and the nRF52840 Dongle for the final experiments. These devices were able to sniff the packets exchanged between the glasses and the phone reliably.

Initially, the Micro:bit V2.20/21 [6], a programmable microcontroller board, flashed with a sniffer firmware, btlejack [15], was used to capture BLE packets. However, the Micro:bit was unsuitable for this experiment as it could not reliably sniff the connection between the phone and the glasses. The way for the Micro:bit to target a connection requires the Micro:bit to first sniff for existing connections and retrieve the access address of active BLE devices, which is then output into the terminal as a list. Next, it is necessary to determine which access address corresponds to the BLE device and then use that access address to target the connection between the devices. The problem was that the sniffed addresses had RSSI values that were too low to be from the glasses. Given that the glasses were positioned directly beside the sniffer, such low signal strength was unlikely. Furthermore, interactions with the glasses did not affect the packet count for these addresses. The Micro:bit, in general, was not able to sniff many connections during initial experimentations.

Another sniffer tried was the Adafruit Bluefruit nRF51822 [2], a board that can monitor BLE traffic between devices when flashed with sniffer firmware [1]. However, the Adafruit was also found to be unsuitable for this experiment as it was also unable to sniff the connection between the phone and the glasses. The way for the Adafruit to target a connection requires it to first sniff for existing connections and retrieve the MAC address of active BLE devices, which is then output into the terminal as a list alongside an incrementing number assigned to each MAC address. Next, the number associated with the device's MAC address must be used to target the connection between the devices. When testing the Adafruit, a list of at least 20 different MAC addresses was presented, which was more than the Micro:bits. However, the MAC address of the glasses does not appear in the list.

The third sniffer considered for sniffing BLE traffic was the nRF52840 Dongle. It is a USB device that can monitor BLE traffic and integrates with Wireshark directly, allowing for better real-time, on-screen visualization of the captured packets. Although it required more setup for the flashing, requiring a programmer app on the desktop to flash the firmware [11]. The nRF52840 Dongle targets connections by scanning for nearby BLE devices, identifying their MAC addresses, and allowing the user to select a target connection to monitor. However, this feature was not needed as the on-screen visualization on Wireshark was enough to identify the Ray-Ban Meta glasses and the phone. During experimentation with the device, the nRF52840 performed much better than the other devices in sniffing BLE traffic, as it was able to consistently detect the glasses and phone during pairing. These advantages made it the final choice for the BLE sniffing experiment.

The fourth and final sniffer considered was the Ubertooth One, a USB device that is able to scan, capture, and monitor Bluetooth traffic across channels. Unlike the previous sniffers, it comes pre-flashed with firmware, allowing it to function as a Bluetooth and BLE sniffer right out of the box. This means that there is no need to flash additional firmware. It can also sniff Classic Bluetooth, which none of the previous devices supports. This means that it was the only choice for Classic Bluetooth sniffing. Initially, the Ubertooth One was chosen for BLE sniffing as well. The reason was that its process did not require sniffing existing connections in order to target a specific connection. For the Ubertooth One, only the MAC address of the device was needed to target a device. However, the resulting capture would sometimes have random time jumps during traffic capture, therefore resulting in the nRF52840 Dongle being used for BLE sniffing.

6

Static App Analysis

This chapter presents a static analysis of the Meta View app, the companion app for the Ray-Ban Meta glasses. The analysis focuses on examining important aspects of the app that would aid in understanding its role in managing user data and its privacy implications. These aspects include the app's requested permissions, the integration of third-party libraries, and the potential transmission and recipient of user information.

The app serves as the primary interface between the glasses and Meta's backend services. Since it manages core functionalities, such as pairing, media import, and notifications, the permissions requested by the app and its libraries can significantly impact user privacy. Therefore, examining the permissions and dependencies is important to understand the potential risks related to data collection, user tracking, or unauthorized access. The static analysis was performed using MobSF to inspect the app's APK, providing details about permissions, native libraries, manifest components, and other relevant information.

6.1 Permissions Analysis

This section covers the permissions identified in the app as well as the privacy and security risks when granted. Android applications must declare which system resources they require through permissions. The Meta View app requests several permissions that are considered dangerous by the Android operating system. Dangerous permissions grant access to sensitive user data or critical features, making their presence noteworthy when evaluating potential security and privacy risks. The following dives into these permissions and their potential implications based on the permission types.

Location

The app requests multiple location-related permissions, including background, coarse (network-based), and fine (GPS). While these location permissions are necessary for functionalities such as pairing with the Ray-Ban Meta glasses or providing location-based services, they also raise some concerns. Background location permission allows the app to track the user's whereabouts even whilst the app is not actively in use, which creates the possibility of con-

stant user tracking. Coarse location access uses network data, for example, cell towers, to determine an approximate location of the user, whilst fine location uses GPS to determine a precise location, thereby allowing for higher accuracy in the mapping of user movements. These permissions could allow third parties to track the precise movement of a user by continuously collecting GPS data, essentially like physical surveillance. Furthermore, when combined with other permissions, such as network access, these location permissions could lead to detailed profiling of user activities and movements.

Bluetooth

Permissions to access Bluetooth-related functionalities include scanning and connection, which allows scanning for, discovering, and pairing with nearby devices. While necessary for pairing with the Ray-Ban glasses, these permissions present a risk where malicious devices within range could attempt to establish unauthorized pairing and access to the glasses. Furthermore, if the connections are not properly secured, data exchanged via Bluetooth could be intercepted or tampered with by a third party.

Account and Device Connectivity

In order to access user accounts and manage the device's network connections, the app requires permissions such as `GET_ACCOUNTS` and access to nearby Wi-Fi devices. These permissions support importing images from the glasses to the app. However, if misused, there is a risk that an attacker or the app itself could gain unauthorized access. The `Get Accounts` permission allows the app to access a list of accounts in the Accounts Service, which could be used for unauthorized data access or data gathering. The permission for the connection to nearby Wi-Fi devices allows the app to advertise and connect to nearby devices, which could expose the user to malicious devices within range attempting to initiate connections over Wi-Fi, potentially leading to eavesdropping or data interceptions by third parties.

Storage and Media

By requesting read, write, and delete access to data from external storage and media files, the app is able to manage photos and videos captured by the glasses. However, the problem with these permissions is that the storage includes personal data, including photos, videos, or even documents, which, if exploited, could be accessed or modified without user consent. Another concern is that the app could theoretically upload these files to remote servers without explicit user consent, which infringes on data privacy and user autonomy. Furthermore, if this permission is combined with location permissions, photo metadata could be used to track user activity and movements.

Phone and Call

The app requests permission to read the call log and contact data, allowing the app to read and access the user's contact information and call history. The app also requests

permission to read the phone state and identity, allowing the app to access the phone identifiers, including the phone number, serial number, and whether or not the user is on a call. These permissions allow the app to access and collect sensitive information, including details on their communications and social circles. The ability to access phone identifiers raises concerns about user and device fingerprinting across multiple services. If exploited, these permissions could reveal personal information such as the user's communication patterns, call history, relationships, and private contact information, which could be used in targeted attacks.

Communication and Messaging

The app requested several communication-related permissions, including receiving, reading, and sending messages. Although necessary for features such as communicating on messaging platforms, these permissions allow access to personal user communications. This access introduces the possibility of unauthorized retrieval of messages or intercepted one-time passwords, which could be used for phishing attacks where the data are used for identity theft or unauthorized messaging. The app also requests permission to post notifications, which allows the app to post notifications. This is a valid request for the posting functionality on social media platforms; however, it could be exploited for deceptive or phishing attempts via fake notifications.

The list of permissions found in the Meta View app is provided in Table A.1. The permissions in Malware are the most widely abused permissions that allow access to highly sensitive data, or critical system features such as storage, location, SMS, and phone state. If abused, it can expose users to threats such as unauthorized data collection or surveillance. Although not as extensive, the permissions in Common are also commonly abused and pose privacy or security risks. The classification between dangerous and normal depends on the level of risk associated with granting these permissions to an app. The Android permission system defines this classification. Permissions classified as dangerous grant access to sensitive information and, therefore, have a higher potential to affect user privacy or security. On the other hand, permissions classified as normal allow access to less sensitive data and are considered low risks to user privacy and security.

In summary, the permissions requested by the Meta View app, as summarized in Table A.1, are necessary for certain app functionality, including managing connectivity with the Ray-Ban glasses and enabling features like photo sharing, notifications, and voice commands. Nevertheless, granting these permissions exposes users to potential privacy violations, such as continuous precise location tracking, data harvesting, or interception of personal communications.

6.2 Third-Party Usage

This section examines whether the app includes any third-party libraries, determines whether personal information is being transmitted, and, if so, who the recipient is. The investigation focuses on components of the Meta View app that may involve third-party usage or data

transmission, including native libraries, trackers, manifest components, and databases, to identify relevant data flows and privacy implications.

The libraries and components are also grouped into different party types for the static app analysis. Similar to traffic analysis, first-party refers to those owned by Meta and directly linked to the app. Support parties refer to the components that Meta owns but are external to the app. Third-party are libraries, services, or components that Meta does not own and instead provided by an external organization. Additionally, there is an additional party included here, which is platform-provided parties. They are components provided as part of the Android development framework or operating system (e.g., AndroidX).

Native Libraries

Native libraries are compiled binaries (.so files), usually written in C or C++, that potentially integrate code from external sources. Thirteen native libraries were identified in the app. A complete and detailed table is provided in A.2. Among these, only two libraries, libbreakpad and libbreakpad.cpp.helper, are third-party libraries. Google developed both libraries for crash reporting. They are typically used for debugging and improving app stability, allowing developers to collect and analyze crash data. The remaining libraries appear to be first-party or platform-provided, determined through the human-readable strings embedded within the APK file. Their functionalities, such as dynamic linking and signal handling, also align closely with tasks typically associated with internal development.

Trackers

Trackers are mechanisms in an app used to monitor user behavior, usually for analytics or advertising purposes. Therefore, analyzing the trackers integrated into the app can identify potential data collection and transmissions. The Meta View app has the Facebook Flipper tracker, categorized as an analytics tracker. Analytics tracking is designed to collect data on how users interact with the app, such as which features are used the most, how long users stay engaged, or where they encounter problems.

Facebook Flipper is used in the app to transmit user data to Meta or Facebook for analytics purposes. However, although the stated purpose is to help improve the user experience or to find bugs, it raises potential privacy concerns as analytics trackers may collect data that could include potentially sensitive information, especially without sufficient user consent. Although there is no solid evidence that the tracker shares data with external organizations or advertisers, this analytics tracker transmits user engagement information to the internal servers of Meta, which may be combined with other user data for better user profiling.

Manifest Components

The AndroidManifest.xml file serves as a blueprint for the app, providing metadata that defines the structure, permission, and components of the app. By analyzing the list of components, including services, activities, and receivers, the aim is to identify which components

are tied to third-party libraries. The app includes 20 services, receivers, and activities that are exported. These components are accessible not only to the Meta View app but also to other apps on the device. As a result, it is a potential entry point for interactions with third-party libraries. A complete detailed table of these components can be found in A.3. The majority of exported components are first-party or support-party. While these do not involve third-party organizations, they may transmit user data internally within the Meta ecosystem. Only four components were found to be tied to third-party components.

The first component, `FirebaseInstanceIdReceiver`, is owned by Google and supports push notifications by registering device tokens. It aids in the communication between the app and Firebase Cloud Messaging. This component transmits device tokens and potential notification metadata to Google's Firebase Cloud Messaging servers. The second component, `PushLiteGCMJobService`, manages lightweight push notifications. Although the service is implemented by Meta, it relies on Google's infrastructure, specifically Google Cloud Messaging, and therefore is classified as third-party. As with the first component, it transmits device tokens and potential notification metadata to Google servers.

For both components, if the app sends sensitive information in the payload, such as account updates regarding password changes, this data could be exposed during transmission or logged on Firebase. Although notifications are usually encrypted, metadata accompanying them could be accessible, including the device model or timestamps of when the notification was sent. Similarly, like the Meta View app, Firebase may also collect additional data for its analytics, which could include the frequency of push notifications, resulting in potential breaches of user privacy.

The third component, `RevocationBoundService`, is part of a library that allows users to authenticate with their Google accounts in the app. This service is part of Google Play Services, specifically, Google Sign-In API. This library is used for managing sign-in tokens and other related authentication features, meaning that authentication tokens and user account data necessary for authentication and session management are transmitted to Google, most likely including email address and user ID. Authentication tokens and user account details are sensitive and, if intercepted, could allow unauthorized access to Google accounts.

The last third-party component, `RedirectUriReceiverActivity`, is part of the Spotify Authentication SDK, a library allowing users to authenticate their Spotify account through open authorization (OAuth) tokens. Open authorization tokens and possibly user account details may be transmitted to Spotify servers as part of the authentication process. Similarly with Google, authentication tokens and user account details are sensitive and, if intercepted, could allow unauthorized access to Spotify accounts.

In summary, the app integrates several third-party libraries, including Firebase (Google), Google Cloud Messaging, Google Sign-In, and Spotify Authentication SDK. These libraries are necessary for features such as push notifications and user authentication. However, they may transmit sensitive data, such as personal information, device tokens, open authorization tokens, user credentials, and account data, to their respective servers. If these transmissions are intercepted by external entities or used by their respective platforms for unintended purposes, there is a risk of unauthorized access.

Firestore Database

Firestore, developed by Google, provides cloud-based, backend, and analytics services. The services it provides mean that Firestore databases may handle sensitive user data. The app communicates with the Firestore endpoint `fb-messaging-b8df1.firebaseio.com`. On a positive note, the Firestore Remote Config is disabled in the app, which is a feature that allows dynamic modifications of the app's behavior and appearance directly from the server side without requiring a new version or update of the app to be launched. Firestore Remote Config could expose unauthorized users to sensitive parameters such as API keys. Therefore, with it being disabled, the risk of exposing sensitive user data or security vulnerabilities from unauthorized parameter changes of the app is reduced. However, since the app still communicates with Firestore and the exact data transmitted and stored remain unclear, the possibility that sensitive user-related data could be shared in a third-party environment remains a concern.

6.3 Summary

The Meta View app's analysis reveals that it requests a wide range of permissions, including those classified as dangerous. Although these permissions, such as location access and read or write privileges, are required for the app's functionalities, they expose users to potential risks, including continuous location tracking or unauthorized access to personal media and files.

The Meta View app identified various third-party libraries and potential data flows to third-party organizations, particularly from Google, including Firestore, Breakpad, and Google Sign-In, and Spotify, including Authentication SDK. Although these components provide important functionalities such as crash reporting, push notifications, and user authentication, transmitting sensitive data, such as device tokens and user credentials, to third-party organizations raises privacy concerns. Furthermore, first-party trackers, such as Facebook Flipper and other native libraries and components, are first-party or platform-provided and most likely collect user behavior data and transmit it to Meta analytics servers. Although its existence reflects that the app mainly relies on its internal infrastructure and the foundational tools of the Android development framework, it raises privacy concerns about the scope of internal data sharing within the Meta ecosystem.

The analysis details the importance of clearly communicating these data flows and permission requirements to users for a more informed market and the necessity for robust data protection policies and security measures to minimize the risks. Overall, the findings suggest that the Meta View app functionalities rely on external parties, namely Google and Spotify, and practice data collection and sharing, which potentially risks user privacy.

7

Network Analysis

This chapter examines the network traffic generated by the Ray-Ban Meta glasses and the Meta View app. Chapter 5 describes the traffic analysis methods and the testbed for capturing the traffic, which includes both active usage scenarios of the glasses (touch-based commands, voice-based commands, and pairing) and idle traffic. It also explains the different types of parties in the analysis. The focus is to assess the privacy implications of network traffic captured during different interactions, including user profiling, location inference, and bystander exposure. Specifically, to identify what patterns can be observed, determine whether any personal or bystander information is exposed, and pinpoint the potential privacy and security risks. Due to most traffic being encrypted, the analysis focuses on metadata, such as domain names, packet sizes, destinations, and protocols, to infer such information.

7.1 General Network Analysis

The captured network traffic from interactions with the glasses reveals several potential concerns related to user data and privacy worth mentioning. For example, a general analysis across all experiments showed that an obvious pattern can be found in the traffic volume and protocols used for media transfer, which will be detailed later. This section examines three prevalent trends observed across the entire data collection. The first is protocol usage, which analyzes the types of protocols captured in the experiments. The second trend explores the general flow of traffic, including the size distribution of network packets and the sequence of packets in the traffic. The last aspect focuses on the destination of the packets.

Protocols

Figure 7.1 shows all protocols captured in the active interactions experiments and their readability with examples based on the payload. A protocol type is considered unreadable if the packet's content is not human-readable. A packet is partially readable if the payload consists of readable elements, usually the domain names, with the rest being unreadable. Readable is when the majority of the payload is human-readable.

Packets using the User Datagram Protocol (UDP) are an example of partially readable

traffic. Within the payload of some UDP packets, a readable string was found, SIGPATH-SREPCERTINDEX. A dissected interpretation of this string could be SIG(nature) PATHS REP CERT(ificate) IN(de)X, which indicates that the packet was involved as part of a certificate exchange. However, the captured packets, as seen in the figure, predominately relied on Transmission Control Protocol (TCP), Transport Layer Security (TLS) versions 1.2 and 1.3, and Quick UDP Internet Connections (QUIC). Their payloads are typically encrypted, making them unreadable in raw form. Interestingly, there was an observable difference in the frequency of TLS handshakes between the experiments for touch-based and voice-based commands. In touch-based commands, TLS handshakes were consistent throughout the capture, whereas in voice-based commands, they only really appeared in the first two minutes of the capture or the last two minutes.

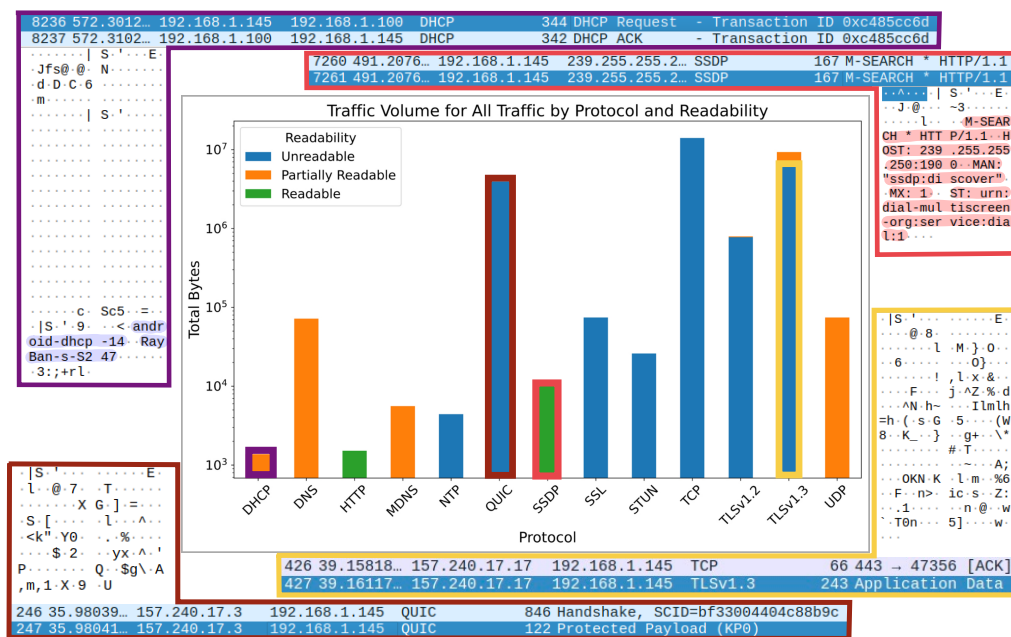


Figure 7.1: First-party traffic volume categorized based on the packet's protocol and readability. The accompanying Wireshark snapshots show example protocols for each type of readability, along with what can be read in their payloads.

Nevertheless, the accompanying metadata can still lead to information exposure. For example, in the information column, the Server Name Indication (SNI) fields in TLS 'Client Hello' messages revealed repeated connections to domains such as graph.facebook.com as well as Spotify and WhatsApp APIs. Similarly, recurring destination domain names, including graph.facebook.com and spclient.wg.spotify.com, are visible in Domain Name System (DNS) queries. Despite the encryption of the payload, repeated connections to particular domains, especially under a certain time interval, can disclose details about user activities and application usage.

Additional protocols, such as Address Resolution Protocol (ARP) and Dynamic Host Configuration Protocol (DHCP), revealed device identifiers and MAC and IP addresses. More specifically, from ARP, the IP addresses of the phone and network were exposed,

and the MAC addresses of the phone, network, and glasses were exposed. The exposed IP addresses of the network connection provide a general location of the user, and the MAC addresses of the devices were consistent throughout the experiments, which allows potential user fingerprinting. In all captures, the ALFA network adapter appeared as Alfa_b2:e8:9a, showing how unique device identifiers appear regularly in network traffic. Another protocol, Dynamic Host Configuration Protocol (DHCP), used for assigning IP addresses, has DHCP requests packets that expose additional information, including the phone as android-dhcp-14 and the glasses as Ray Ban-s-S247. Such information could also aid in fingerprinting connected devices.

Furthermore, protocols such as Simple Service Discover Protocol (SSDP) and Multicast DNS (MDNS), which are for service or device discovery, are visible to all devices on the same network. As a result, local network information and the phone's discovery and connectivity attempts were on full display. In the captures, these protocols often had queries for the service name `_spotify-connect._tcp.local`, which shows that the phone was running Spotify and tried to use this service name to find other devices that could play Spotify on the same network. This visibility of service-discovery traffic increases the risk of passive monitoring by exposing available services and device identities.

Traffic Volume and Packet Size

Another aspect of observation pertains to the overall traffic flow, where traffic peaks or fluctuations often correlate with specific application usage. For example, the largest spikes typically occur during media transfers, although experiment-specific patterns are discussed in detail in the subsequent sections. In order to have a better understanding of these patterns, the protocol-specific traffic volumes were analyzed and found to have recurring sequences that can also be tied to specific interactions. For instance, the interaction for media transfer typically involves four packets of Internet Control Message Protocol version 6 (ICMPv6) packets, followed by DNS queries to Facebook, which is then continued with

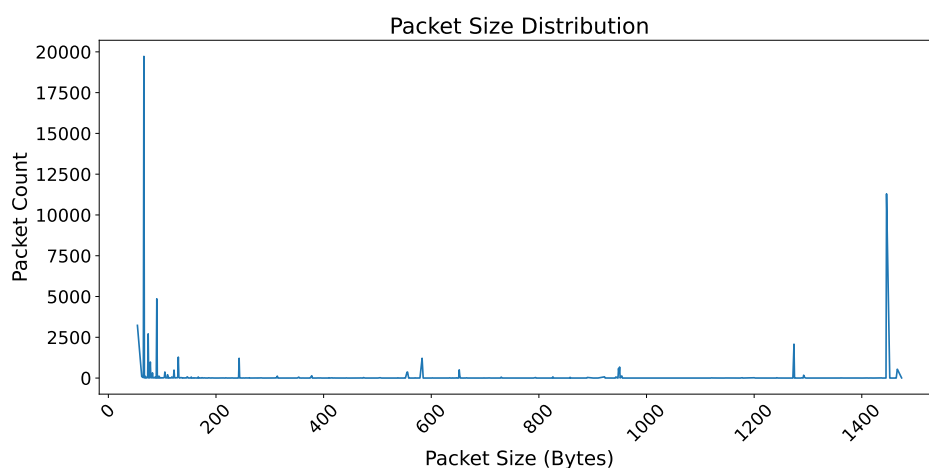


Figure 7.2: The packet distribution, which shows a comparison of the number of packets that are of a specific size for all traffic captured in the five rounds of voice-based commands experiments.

TCP and TLS protocols with large packet sizes. Typically, most packets are relatively small, fewer than 200 bytes.

In contrast, media transfers frequently involve packet sizes above 1000 bytes, often around 1446 bytes. Figure 7.2 illustrates the packet size distribution for the voice-based command experiments. The two highest peaks in the figure are packets with sizes less than 200 and packets with sizes greater than 1400 bytes. Even without decryption, this difference in traffic volume allows an observer to infer user activity. It should be noted that these ICMPv6 packets appeared exclusively during media transfer, which simplifies the identification of this specific interaction, even if the actual content remains encrypted.

Traffic Destination

Throughout the experiment, there were continuous interactions with public servers. First-party traffic dominates the traffic volume, communicating with Facebook APIs (e.g., `graph.facebook.com`). Furthermore, there was frequent activity involving Spotify APIs, such as `spclient.wg.spotify.com`. While some Google APIs, such as `android.googleapis.com`, were detected, they appeared to be related either to Google Cloud services supporting Spotify or system services for the device. Many of the destinations of these addresses were located in Switzerland, allowing adversaries to get an idea of the user's general location.

It is worth noting that none of the identified third-party IP addresses were detected to be from prominent advertising or tracking domains. However, the Meta View app analysis in Chapter 6 of the glasses' permission requests and the glasses' data collection policies [3] indicate that Meta does collect user activity data. Therefore, the absence of explicit tracking does not eliminate the presence of tracking. However, this does imply that users have less control over their data, as they lack full transparency, making it difficult to know which activities are monitored, what data is being gathered for analytics, and how they may be used.

7.2 Active Traffic

Active traffic refers to the traffic generated during user interactions through touch-based commands, voice-based commands, and pairing sequences between the device and the companion app. The following subsections dive into each category of user interaction experiments, explaining unique traffic patterns and potential privacy risks for each experiment.

7.2.1 User interactions - Touch commands

The experiment for touch-based commands is primarily comprised of commands for Media Capture and Music Playback. Across all five repeated trials, packets sent to public destinations account for over 80% of the total traffic volume. Among these public addresses, first-party traffic dominates the contribution to the traffic volume, as seen in Figure 7.3, exceeding 2.8 MB on average per trial and contributing more than 98% of the public traffic volume.

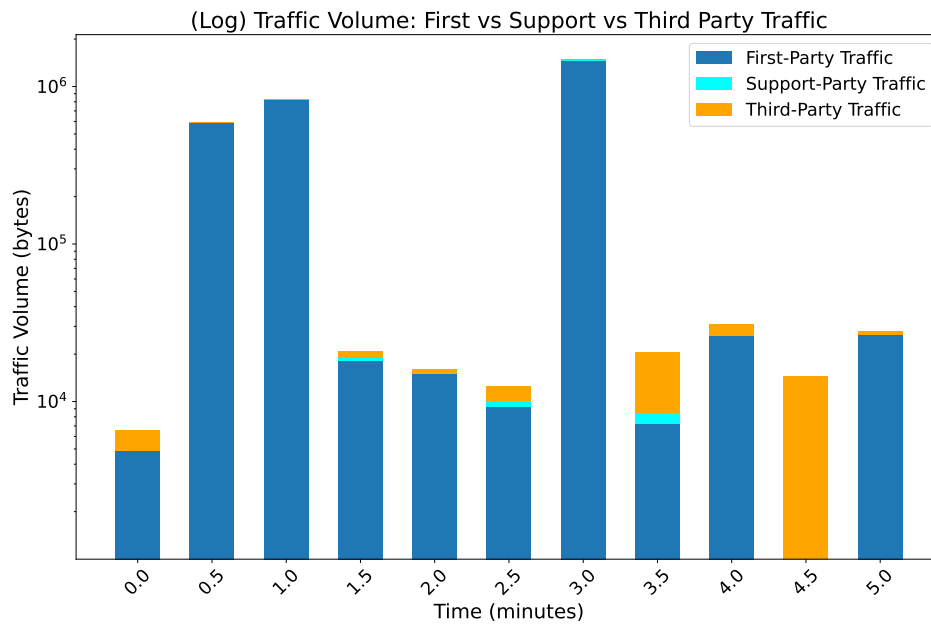


Figure 7.3: The mean public network traffic captured across the five repeated trials of the touch-based commands experiments, with the data categorized by party type.

The bar graph in Figure 7.3 shows that there are two prominent spikes in the traffic, which appear even more clearly in Figure 7.4. From the figure, the experiments have a relatively consistent pattern throughout the timeline, as indicated by the similar general behavior of the plotted lines. Due to the majority of traffic being first-party traffic, the patterns in the total traffic volume are analogous to that of first-party traffic and its volume

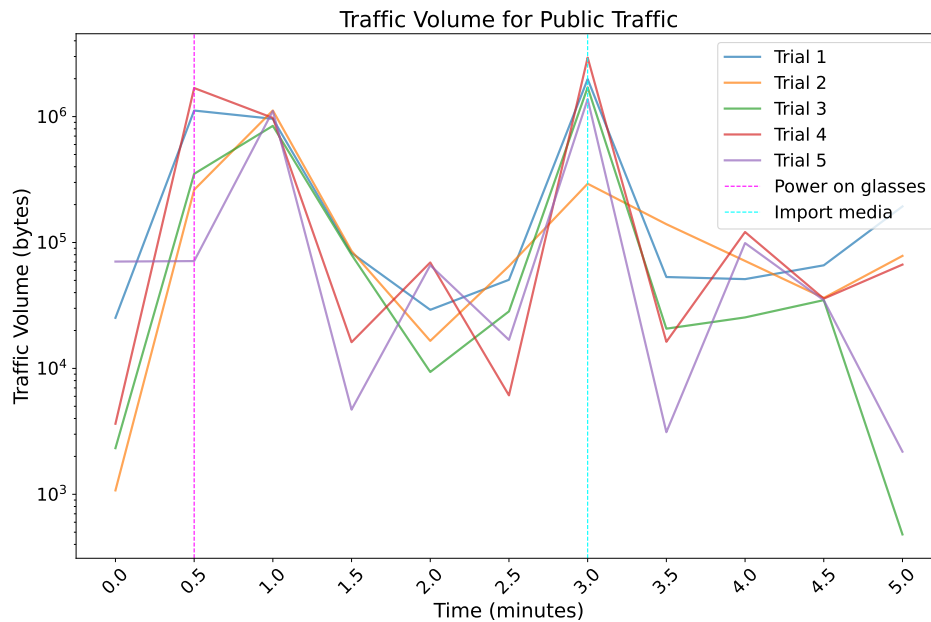


Figure 7.4: Log line plot of public traffic captured during the five repeated trials of the touch-based commands experiments, with the data separated by trial.

trends. The first spike, as indicated with the magenta line, occurs during the initial power on of the glasses and opening of the Meta View app. The duration of this interaction typically spans from minute 0.5 to 1.5. The second prominent peak, as indicated by the cyan line, is during the transfer of the captured media from the glasses to the phone, which is expected as the media transport is through the network and involves bulk data transfer. In comparison, capturing media and playing music did not generate large traffic bursts. This striking difference in traffic volume demonstrates that even encrypted payloads could expose an approximate sense of user activity and that external entities could still determine certain user activities based on the traffic volume.

On the other hand, support-party traffic was nearly negligible at under 5 KB on average per trial, which is less than 0.2% of traffic volume. Figure 7.5 shows that Trial 5 was somewhat inconsistent with the other trials. Upon inspection, it was noticed that there was an absence of Continuous Data SSL packets, which were present in all other trials. This inconsistency was likely due to network factors or interrupted connection, as the packets were highlighted in black and red, indicating failed connections and packet losses.

The destination of all traffic captured was a single WhatsApp service provider. These packets consisted mainly of TCP ACK packets and some SSL packets. This finding is unexpected, given that touch commands do not interact with nor involve WhatsApp. Looking at the figure, the interaction with support-party traffic appears consistently during the initial pairing, media capture, and when playing music on Spotify. None of these interactions require WhatsApp. As shown in Figure 7.5, this behavior was consistent across all trials, each with similar traffic volume. Focusing on the existence of this address during media capture, the figure suggests that the app, for some reason, initiates a connection to the WhatsApp API whenever the wearer takes a photo or video, even though such a connection would typically be expected only when attempting to upload or share the media on WhatsApp. According to Meta’s Terms of Privacy [4], images captured by the glasses or interactions

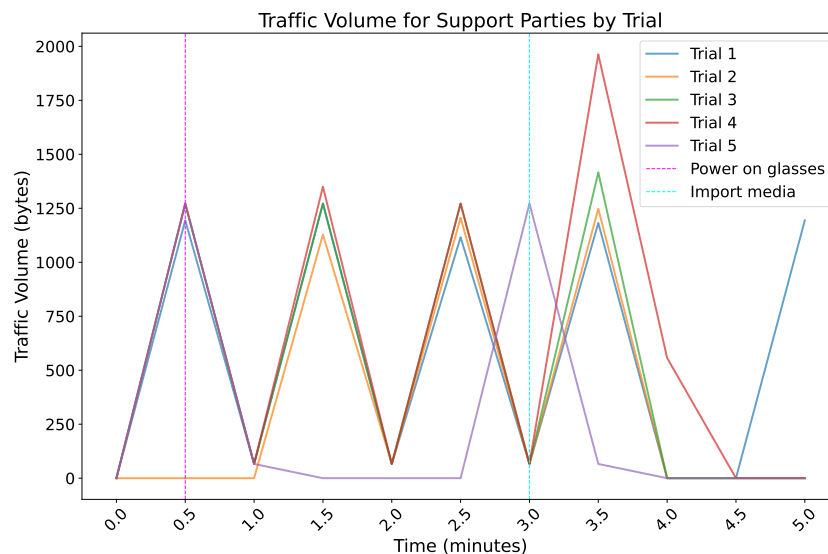


Figure 7.5: Support-party traffic captured during the five repeated trials of the touch-based commands experiments, with the data separated by trial.

made with the glasses are not stored unless uploaded onto one of their social media platforms or sent to their servers during the interaction. However, it is plausible that, by leveraging WhatsApp as an intermediary, these images could then be sent to their servers for analytics or other purposes without violating their terms of service, which would be a complete privacy invasion for users. This behavior was observed exclusively during physical interactions with the glasses and was absent during the voice-command experiments. A possible explanation could be Meta’s policy, which states that they already collect the information from voice-enabled features, reducing the need for additional transmissions during voice interactions. This unexpected behavior raises questions about whether the app unnecessarily transmits data to servers during media capture without the user’s awareness.

There was minimal third-party traffic, below 60 KB on average per trial, contributing to less than 2% of the traffic volume. This indicates that there are only a few external dependencies when physically interacting with the glasses. As shown in Figure 7.3, minutes 3.5 and 4.5, corresponding to the start of music playback and change of the music track, had the most notable peaks in third-party traffic occurrence. Unsurprisingly, these surges involve communication with Spotify servers. During these time intervals, there was also nearly no first-party traffic, suggesting that music playback does not rely on Meta servers.

In order to analyze the extent of information exposure, the traffic was categorized by protocol to assess the readability of the network traffic. This is illustrated in Figure 7.6, which shows that most packets were not human-readable. Few of the partially readable or readable traffic included local protocols, such as DHCP, MDNS, and SSDP.

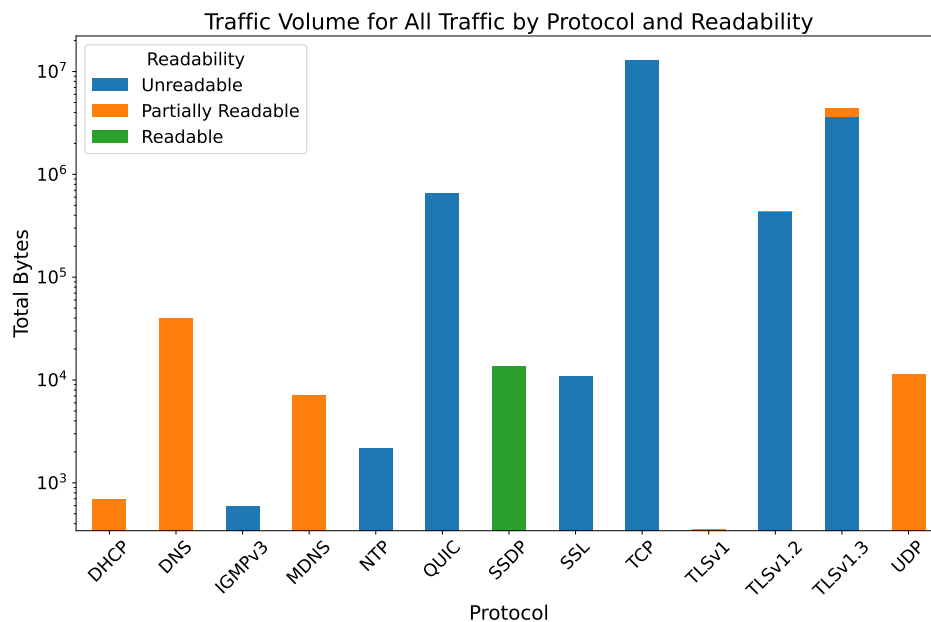


Figure 7.6: All network traffic captured across the five repeated trials of the touch-based commands experiments, with the data categorized by readability and protocol.

Protocols for local networks can still reveal private details. MDNS and DHCP protocols revealed information about the services and devices in the network. For example, `_spotify-connect.tcp.local` indicates that Spotify Connect is being queried, allowing direct music streaming from Spotify. This protocol only reveals the IP address of the phone trying to find other devices compatible with running Spotify. However, this still informs third parties that the phone is running Spotify. There is one protocol, SSDP, with readable payloads. They are packets with (Hypertext Transfer Protocol) HTTP content. The readable content includes MAN: "ssdp:discover," specifying it as a discovery message. The type of service being searched is also visible from search target (ST), ST: urn:dial-multiscreen-org:service:dial:1. Discovery and launch (DIAL) is a protocol used by devices to allow apps to be discovered and launched, meaning this packet is a discovery query. Both MDNS and SSDP packets in the captures are in two-minute intervals and can expose information about the types of devices and services available on the local network to unauthorized devices.

The rest of the protocols handle public traffic. DNS protocols are one of the two partially readable protocols. The queries revealed the hostnames of the targeted domains. There were frequent queries for Facebook domains, such as `graph.facebook.com` or `gateway.facebook.com`. The consistent DNS throughout the capture not only reveals which application is active but can also indicate server communication patterns that could be used to infer user behavior. The other partially readable public protocol is UDP. The UDP packets had a payload that started with the tag `SpotUdp0`, which appears to be an application-specific identifier, which allow observers to infer application usage.

There was an interesting singular packet with protocol TLSv1 was directed to `gateway.facebook.com`. This was in Trial 1 around minute 5.0, near the capture's end. Although no sensitive user data is revealed, metadata, including the hostname, indicated in the SNI or

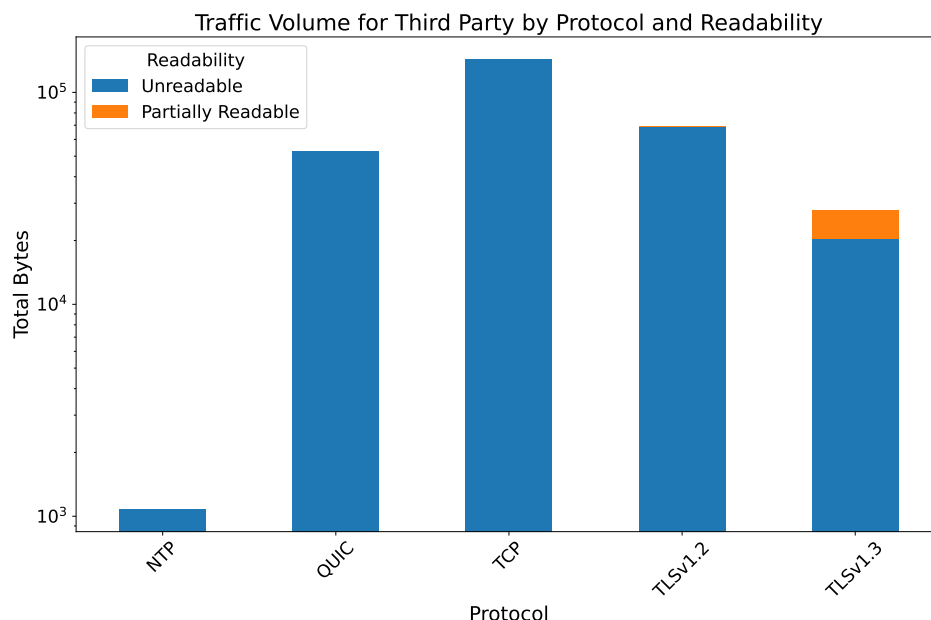


Figure 7.7: Third-party traffic captured across the five repeated trials of the touch-based commands experiments, with the data categorized by readability and protocol.

the cryptographic details, such as the supported cryptographic algorithms or TLS versions, were clearly revealed. However, its presence in the capture raises questions about potential server configurations that would result from the deprecated version of the protocol.

Focusing on third-party traffic, only five protocols were identified: NTP, QUIC, TCP, TLSv1.2, and TLSv1.3. These protocols are visualized in Figure 7.7. Most of the traffic, as illustrated in the figure, is unreadable, reflecting the substantial encryption in these communications. The partially readable traffic in TLSv1.3 only consisted of ‘Client Hello’ messages that revealed the hostnames of the phone that wanted to establish a secure connection. More than half of these hostnames were directed to Spotify, such as `clienttoken.spotify.com` and `login5.spotify.com`, which are related to authentication tokens or the login procedure for Spotify. The rest of the partially readable traffic was mainly related to Google, which had queries to hostnames such as `www.google.com`. As a result, although the exact content is unknown, the payload still exposed the references to the domains. Spotify and Google both had packets with protocols TCP, TLSv1.2, and TLSv1.3. Interestingly, Google was the sole contributor to QUIC packets.

To better illustrate the third-party traffic destinations, Figure 7.8 shows the total traffic volume for third parties for different IP addresses that contributed over 200 bytes. While many of the addresses are directed to Google, Spotify produced the largest amount of third-party traffic in this experiment. Spotify had more than 40 KB on average per trial, meaning that it accounted for more than 70% of all third-party traffic. The reason is that the average traffic volume of Spotify addresses was much higher than that of Google. It is noted that although the IP addresses of the traffic for the orange lines are registered to Google, the traffic is directed to Spotify services, as Spotify is using Google Cloud to host its services. This concept was explained in 5.5. The orange lines in Figure 7.8 show

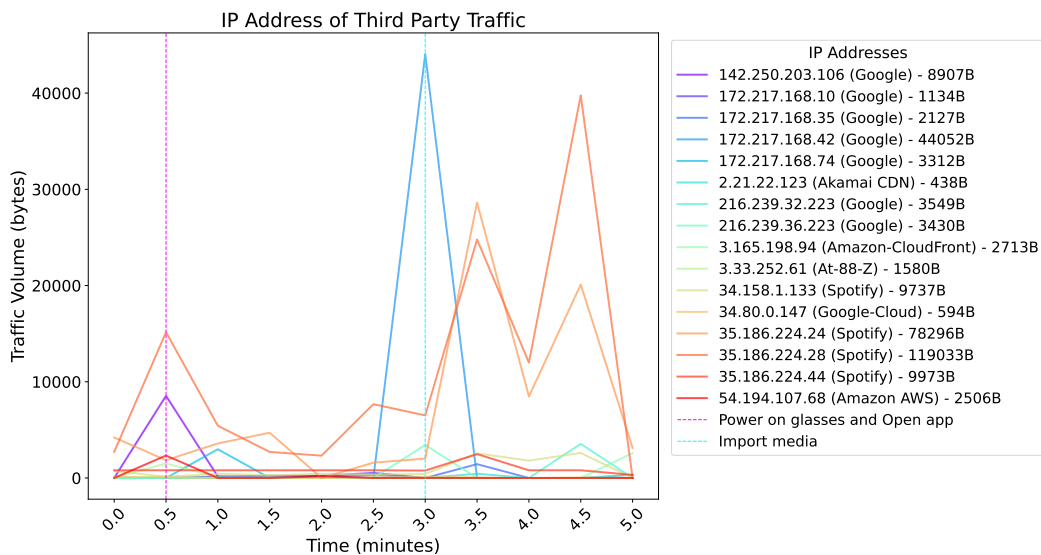


Figure 7.8: Third-party traffic that contributed more than 300 bytes captured across the five repeated trials of the touch-based commands experiments, with the data categorized by IP addresses. The legend displays each IP address along with its corresponding ISP or Service if known, and the Total Number of Bytes.

that Spotify was more active during the last two minutes of the experiment during music playback. Google was the next largest third-party destination; however, the packets were either related to Google Play services or were there for connectivity. The presence of each of those IP addresses was also inconsistent throughout the trials, which likely indicates that their existence was not from the app. For example, the blue spike belonging to Google at minute 3.0 was solely from Trial 3; no other trials had this specific IP address and did not display such a spike at that minute. The protocol for this Google address was QUIC, and, as illustrated in Figure 7.7, all QUIC payloads are unreadable. It is notable that no destination addresses were linked to any trackers or advertisers, though this does not exclude their existence; only none were found in the domains and readable content.

The geological location of the third-party traffic directed towards Spotify is in CA, US, as seen in Figure 7.9. The traffic to Google is also sent to the US, although Google does have some addresses in Switzerland. There were numerous destination addresses that pointed to Switzerland, the same country where the experiment took place. Although these IPs were minor in terms of traffic volume, the location of these addresses can still be used by an adversary to determine the approximate geological location of the user. Third-party entities can also identify the services or applications the user uses based on the IP addresses. Unexpectedly, there is a destination in the figure shown to be in Taiwan, Taipei, which is from Trial 3. The IP address of this destination belongs to Google Cloud services. There is no data payload in the four packets captured; they are purely acknowledgment packets. There is another notable cloud service in the US, where the address is registered to Amazon. The few packets to this address were only found in Trial 2 and were all acknowledgment packets as well.

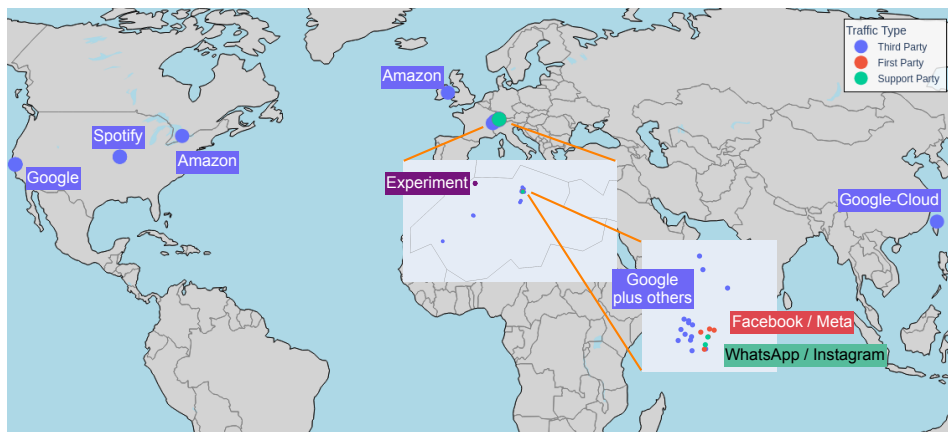


Figure 7.9: Destination addresses of all traffic captured across the five repeated trials of the touch-based commands experiments.

An additional unique destination was found in Ireland. This was from a singular packet that is part of a TLS handshake from Trial 1. Specifically, it was a TLS server certificate message. Its purpose is to prove the server's identity to the client to establish trust and contain the public key. Unlike others, it was partially readable. In the packet, the root CA in the certificate chain was identified as Amazon Root CA, meaning

it is managed by Amazon. There was another CA, Starfield Services Certificate Authority, from Starfield Technologies, Inc., which is most likely an intermediate CA in the certificate chain that signed the server's certificate. References to certificate authority (CA) endpoints could be identified, including <http://ocsop.rootg2.amazontrust.com/rootg2.cert0=> and <http://crl.rootg2.amazontrust.com/rootg2.crl0>, which are link points to the certificate of and Certificate Revocation List (CRL) for the root CA, Amazon Root CA. The location where the CA is registered was also shown as Scottsdale, Arizona, US. Third-party entities can infer the server's identity from this packet using the IP address from the metadata, `ec2-54-194-107-68.eu-west-1.compute.amazonaws.com`. Based on the domain, this server appears to be part of Amazon's computing services in the West EU. Based on the Client Hello in the Information column of the packet, `sdk.push.samsung.com`, this packet is most likely related to push notifications for the Samsung phone. Nevertheless, attackers could use the information from the packing, including the server's hostname, the IP address, and certificate details, which include links to CA validation endpoints and information about the root and intermediate CAs, to map the validation infrastructure used by the CA, providing attackers the ability to target certificate validation servers.

7.2.2 User interactions - Voice commands

From the combined capture of all five repeated trials of the experiment, nearly 70% of the traffic volume for the voice-based commands experiment was directed to public destinations. As illustrated in Figure 7.10, most of this public traffic is first-party traffic, with an average of more than 3.5 MB on average per trial and comprising over 95% of the total traffic. Although support-party traffic rose in terms of volume, with 50 KB on average per trial, it still contributes to less

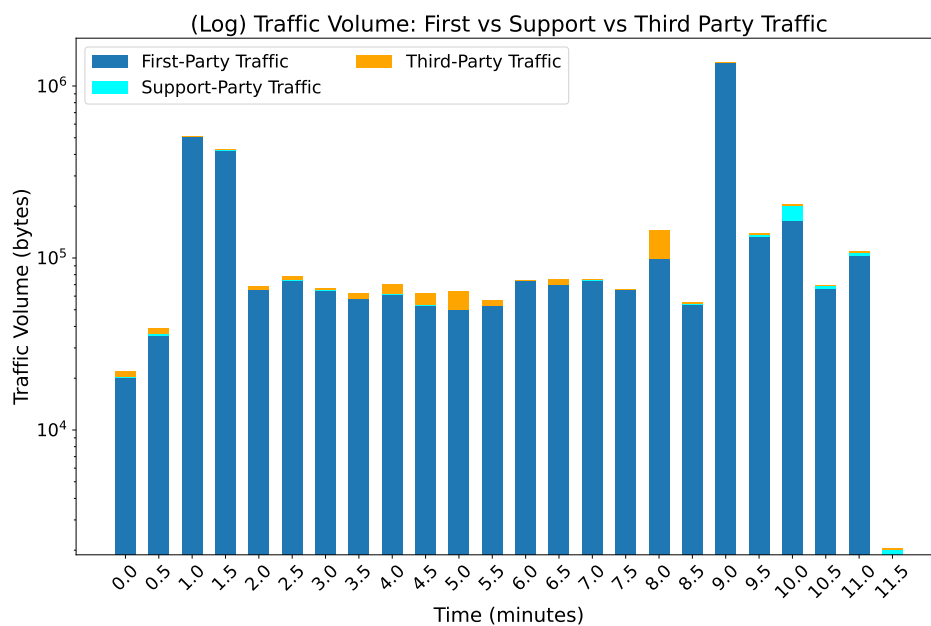


Figure 7.10: The mean public network traffic captured across the five repeated trials of the voice-based commands experiments, with the data categorized by party type.

than 1% of the public traffic volume. The figure shows a slight increase in support-party traffic during the last 2 minutes, though it is mostly around minute 10. This high frequency of traffic reflects the voice command that sends a voice message to a person. There was also an increase in third-party traffic, averaging 120 KB per trial. However, considering that the duration of the experiment is twice as long as touch-based commands and the average traffic volume also doubles the amount of touch-based commands, no extra third-party traffic is being generated from the different command types. In essence, the similar proportion of third-party traffic volume suggests that the traffic is driven more by user interaction than by the specific input method.

The voice command experiments execute a command every thirty seconds, starting from minute 0.5, which is reflected in Figure 7.11. The scatter plot displays the traffic volume aggregated every ten seconds, with the cyan lines indicating the thirty-second intervals. Most dots with higher traffic volume are located on the cyan lines. This means that an adversary could presume that when the traffic volume is roughly 10^5 , it is likely that the user is executing a voice command on the glasses. For many of the voice commands, the data points across different trials are relatively close, which suggests a higher reliability for the adversary to infer that traffic volume corresponds to user interaction. This pattern was unique to voice-based commands and was not observed in touch-based commands. This adds to the ability of the adversary to determine which method of interaction was used, voice-based or touch-based.

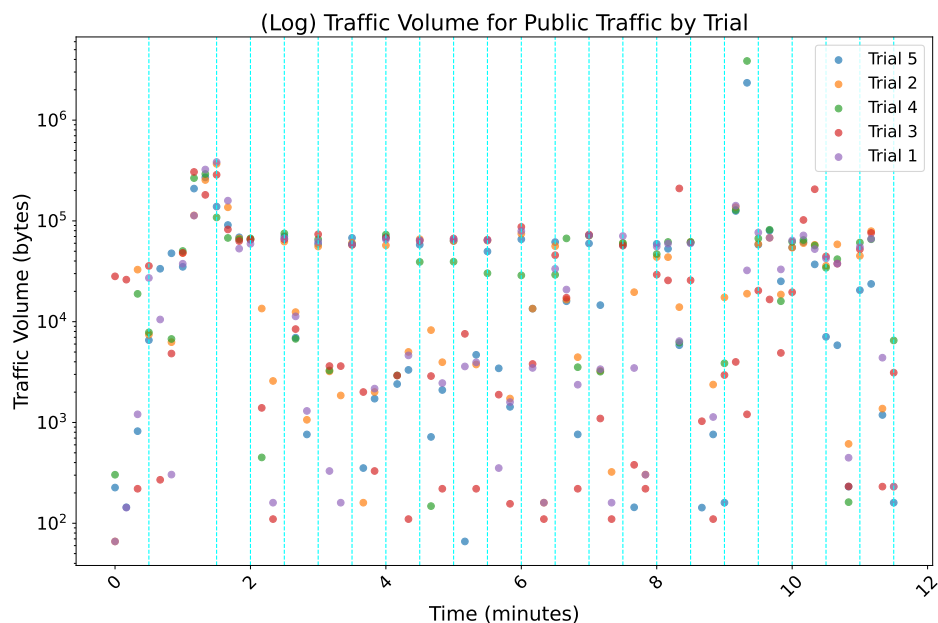


Figure 7.11: Public network traffic captured across the five repeated trials of the voice-based commands experiments, with the data grouped in ten-second intervals.

A more specific traffic pattern noted during the experiments was based on the protocol used. There is a visible pattern from the QUIC protocol between the phone and the Meta servers. There are consistent, consecutive 5 to 7 packets, but usually 6 packets, with the QUIC protocol, which contains a payload that is transmitted immediately when a command

is executed. This sequence of traffic can be easily identified because, in idle capture, there were nearly no consecutive 6 QUIC packets across the 8-hour period. This identifiable traffic pattern allows for better tracking of user interactions with the glasses. It is also possible to have an idea of what command the user provided. Figure 7.12 shows a side-by-side comparison of two example commands in Wireshark from Trial 5, one asking for the AI to send a message and the other asking for the date. The images in the figure only include packets with the protocol QUIC, and the two columns displayed are the time of the packet capture and the information of the packets. The command to ask the AI to send a message is at minute 9.5 (570 seconds), as shown in the image on the left, while asking for the date is at minute 2.5 (150 seconds), which is the image on the right.

As seen in the right of Figure 7.12, asking for the date usually results in the 6 consecutive QUIC packets that contain payload followed by 4 empty QUIC packets, which is then continued with nearly 20 QUIC packets with payload. The sequence is then separable from when asking Meta to send a message to a person, as seen in the left of Figure 7.12, which is due to the back and forths of the glasses needing to know who the receiver of the message is, the content of the message, and the final confirmation to send. There would be 6 to 8 consecutive QUIC packets with payload followed by 4 without, then once again 6 to 8 with and 4 without, forming a cycle until the interaction is complete. This sequence is then also different in comparison to asking for the weather, which usually results in the 6 consecutive QUIC packets containing payload followed by 3 to 4 empty QUIC packets, which are continued with 8 or 9 consecutive QUIC with payload. These are general patterns, so there could be a couple of empty QUIC packets within the group of QUIC packets with payload. However, by observing the number of packets with specific protocols transmitted, a third party could roughly estimate the user's interaction. However, this depends on the interaction as multiple other commands have similar traffic sequences as when asking for the date, whereas asking for the weather and asking to send a message result in more unique traffic patterns.

Figure 7.13 provides a better overview of the general traffic flow across different trials,

573.8045... Protected Payload (KP0), DCID=9533002302735b9c	152.7262... Protected Payload (KP0), DCID=9533002302735b9c
573.8047... Protected Payload (KP0), DCID=9533002302735b9c	152.7505... Protected Payload (KP0), DCID=9533002302735b9c
573.8055... Protected Payload (KP0), DCID=9533002302735b9c	152.7505... Protected Payload (KP0), DCID=9533002302735b9c
573.8055... Protected Payload (KP0), DCID=9533002302735b9c	152.7505... Protected Payload (KP0), DCID=9533002302735b9c
573.8055... Protected Payload (KP0), DCID=9533002302735b9c	152.7505... Protected Payload (KP0), DCID=9533002302735b9c
573.8407... Protected Payload (KP0)	152.7505... Protected Payload (KP0), DCID=9533002302735b9c
573.8587... Protected Payload (KP0)	152.7624... Protected Payload (KP0)
573.8587... Protected Payload (KP0)	152.7799... Protected Payload (KP0)
573.8587... Protected Payload (KP0)	152.8027... Protected Payload (KP0)
573.8874... Protected Payload (KP0), DCID=9533002302735b9c	152.8046... Protected Payload (KP0)
577.3005... Protected Payload (KP0), DCID=9533002302735b9c	152.8305... Protected Payload (KP0), DCID=9533002302735b9c
577.3005... Protected Payload (KP0), DCID=9533002302735b9c	155.5810... Protected Payload (KP0), DCID=9533002302735b9c
577.3031... Protected Payload (KP0), DCID=9533002302735b9c	155.5962... Protected Payload (KP0), DCID=9533002302735b9c
577.3031... Protected Payload (KP0), DCID=9533002302735b9c	155.5962... Protected Payload (KP0), DCID=9533002302735b9c
577.3031... Protected Payload (KP0), DCID=9533002302735b9c	155.5962... Protected Payload (KP0), DCID=9533002302735b9c
577.3031... Protected Payload (KP0), DCID=9533002302735b9c	155.5964... Protected Payload (KP0), DCID=9533002302735b9c
577.3345... Protected Payload (KP0), DCID=9533002302735b9c	155.5967... Protected Payload (KP0), DCID=9533002302735b9c
577.3484... Protected Payload (KP0)	155.6002... Protected Payload (KP0), DCID=9533002302735b9c
577.3549... Protected Payload (KP0)	155.6002... Protected Payload (KP0), DCID=9533002302735b9c
577.3625... Protected Payload (KP0)	155.6002... Protected Payload (KP0), DCID=9533002302735b9c
577.3718... Protected Payload (KP0), DCID=9533002302735b9c	155.6002... Protected Payload (KP0), DCID=9533002302735b9c
579.7541... Protected Payload (KP0), DCID=9533002302735b9c	155.6002... Protected Payload (KP0), DCID=9533002302735b9c
579.7652... Protected Payload (KP0), DCID=9533002302735b9c	155.6009... Protected Payload (KP0), DCID=9533002302735b9c
579.7652... Protected Payload (KP0), DCID=9533002302735b9c	155.6009... Protected Payload (KP0), DCID=9533002302735b9c
579.7652... Protected Payload (KP0), DCID=9533002302735b9c	155.6010... Protected Payload (KP0), DCID=9533002302735b9c
579.7652... Protected Payload (KP0), DCID=9533002302735b9c	155.6010... Protected Payload (KP0), DCID=9533002302735b9c
579.7917... Protected Payload (KP0)	155.6196... Protected Payload (KP0)
	155.6339... Protected Payload (KP0)

Figure 7.12: Wireshark capture of QUIC packets when asking for the weather vs. date in Trial 5 of the voice-command experiment.

illustrating the timeline of all public traffic. The figure shows that the trials were rather consistent with one another, with the only discrepancy during minute 9.0, where Trial 4, the red line, and Trial 5, the blue line, experienced a large increase, while Trial 3, the green line, had a drop in traffic volume. This is also reflected in the average traffic volume for each trial, with a median of 4 MB per trial for trials without the spike, 8 MB for Trial 4, and 6 MB for Trial 5. The spikes are mainly due to first-party traffic from the glasses to the phone during media import, as seen in the surge in minute 9.0. There was no observable reason as to why the trials had such differences in volume during this period. There are two time periods in the figure with a noticeable surge in traffic volume. The first, smaller increase between minutes 0.5 and 2, is the interaction to perform the initial powering on the glasses and opening of the Meta View app. This increase was consistent throughout the trials. The second larger but less consistent increase is at minute 9.0, which coincides with the transfer of media captures from the glasses to the phone.

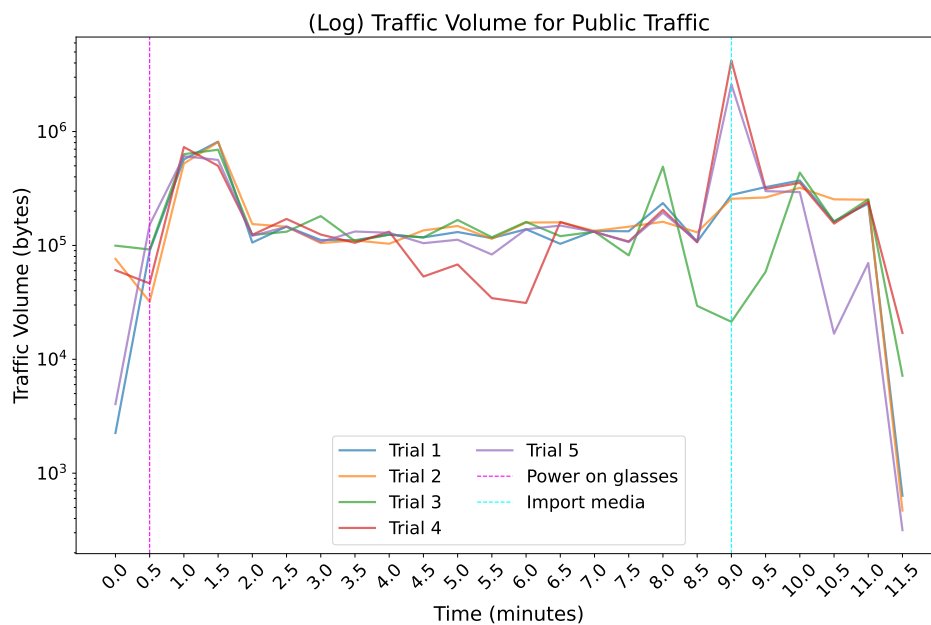


Figure 7.13: Public network traffic captured during the five repeated trials of the voice-based commands experiments, with the data separated by trial.

Diving into the captured support-party traffic, one IP address pointed to Instagram, and two were to WhatsApp. WhatsApp was relatively more consistent throughout the trials, while Instagram's IP address was only in three of the five trials. Both of the traffic destinations were largely unreadable. However, the same packet with the same IP address was present for each trial, with the Info column Client Hello (SNI=media-zrh1-1.cdn.whatsapp.net). This specific packet was only captured once during each trial at the exact time of minute 10.33. The interaction during this time was when receiving a message on WhatsApp. Throughout the trials, there were no other packets with this exact Info column or IP address other than during this interaction. As a result, a malicious person would be able to precisely identify when a person received a message on WhatsApp and, if able to

gain physical access to the glasses (with the phone lock feature off), be able to obtain the message sent to the user.

Figure 7.14 illustrates the third-party traffic volume for different trials. The traffic between the experiment trials aligns surprisingly well with one another, with only some inconsistencies, suggesting that the traffic generated is predictable and closely tied to the commands being executed. As a result, it can be used by attackers to infer the type of interactions the user performs. The first inconsistency is during the initial powering on and opening of the Meta View app, which is between minutes 0.5 and 1.5. The other inconsistency is that in Trial 3, where a much larger spike is observed during the command to make a video recording in minute 8. However, it seems that Trial 3 was an outlier for that interaction. The specific spike in Trial 3 is roughly 200 KB. However, of those, they are all unreadable traffic, with QUIC having roughly 160KB, followed by TCP with a little more than 30KB and TLS with 13KB bytes, as seen in Figure 7.15, which shows the third-party traffic grouped by protocol and readability. The time between minutes 3.5 and 6.5 has the closest alignment of the trials, which starts at the beginning of music playback and ends during the timer interaction. The consistency in the traffic volume can be seen as a positive indicator that the traffic being generated for Spotify is most likely essential communication and is considered necessary for the app’s functionalities, although analytics or advertisers could still be included.

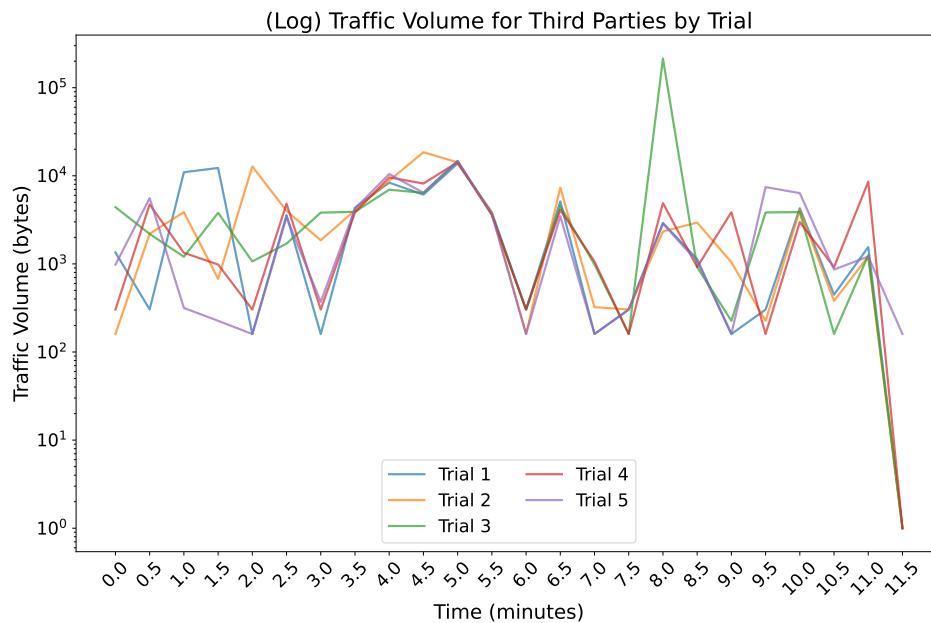


Figure 7.14: Third-party traffic captured during the five repeated trials of the voice-based commands experiments, with the data separated by trial.

The majority of all traffic captured was encrypted and, hence, not human-readable. This is the same for third-party traffic volume, as displayed in Figure 7.15. The partially readable traffic from the protocol TLSv1.3 is roughly the same as in touch-based traffic, where it is ‘Client Hello messages,’ which reveal the domain name the client is requesting a connection to.

Besides the five protocols in touch-based commands, the figure also included some HTTP traffic. This HTTP traffic captured was not present in the experiment for touch-based commands. Out of the five trials conducted, this traffic was present in three of them. Each trial only had two HTTP packets, an HTTP GET request and an HTTP response, at different time periods. The phone sends a GET request to `/generate_204 HTTP/1.1`, which is an endpoint for connectivity checks. The packet completely reveals the software making the request, in this case, Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/60.0.3112.32 Safari/537.36, which shows that Chrome on a Linux device is making the request. The server responds with HTTP/1.1 204 No Content, meaning that network connectivity is confirmed, but no content is returned in the response. Nevertheless, the exact date and time are included in the response. The purpose of this traffic is solely for the device to do a connectivity check. Although no sensitive information is exposed, the software details of the device and the exact timestamps in the packet can still contribute to user profiling.

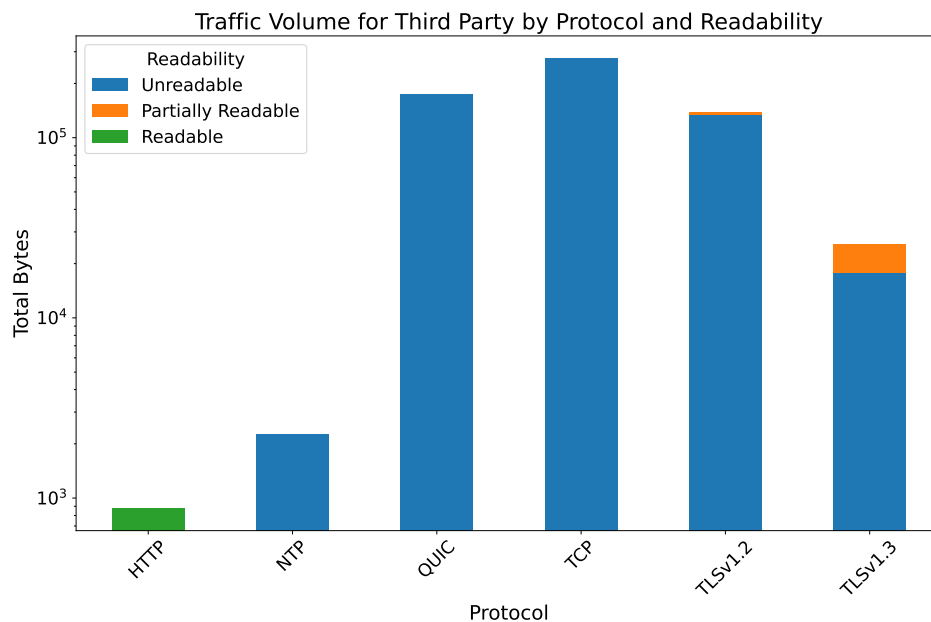


Figure 7.15: Third-party traffic captured across the five repeated trials of the voice-based commands experiments, with the data categorized by readability and protocol.

The third-party traffic volume still had the same main contributor, Spotify, as in touch-based commands, with Google being the second largest destination. Figure 7.16 shows the timeline of third-party traffic based on the IP address. This figure shows all third-party traffic that contributed more than 300 bytes of traffic. It shows that the main third-party destination belongs to Spotify, which has almost 70 KB per trial, nearly 60% of third-party traffic volume, while Google had an average of 50 KB per trial. The traffic for Spotify occurred mostly during Music Playback, as expected, though it was consistently present throughout the whole experiment. Once again, the peak appears to be an anomaly in Trial 3. As mentioned, the minimal third-party interactions show that the app avoids

unnecessary connections to other third parties, which is favorable for reducing attack vectors and protecting user data. However, the predictable reliance on Google and Spotify could be combined with other traffic patterns tied to user interactions, which could help attackers in traffic fingerprinting.

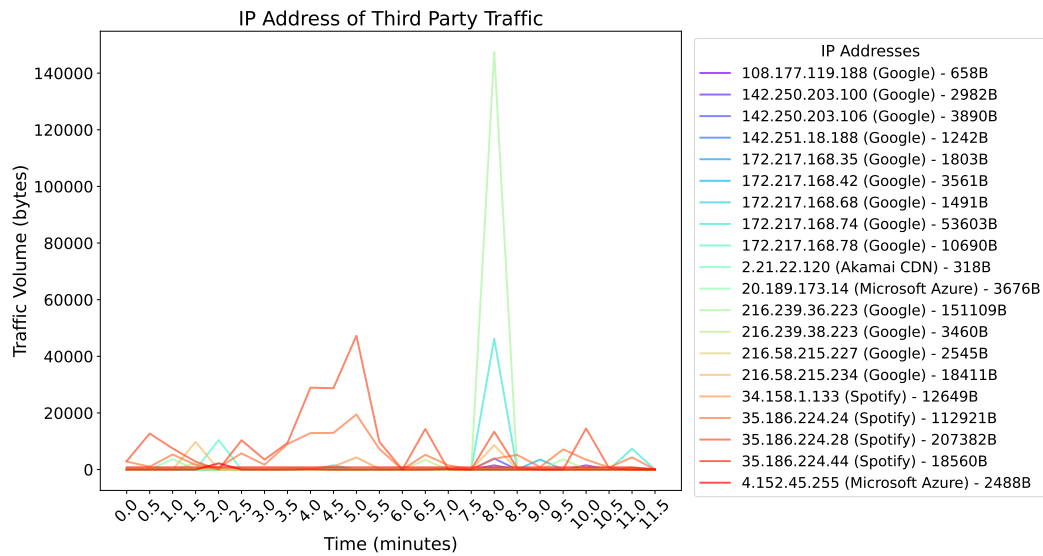


Figure 7.16: Third-party traffic that contributed more than 300 bytes captured across the five repeated trials of the voice-based commands experiments, with the data categorized by IP addresses. The legend displays each IP address along with its corresponding ISP or Service if known, and the Total Number of Bytes

The geological location of most third-party traffic volume is routed to U.S.-based servers, as displayed in Figure 7.17. It is interesting that despite a Google server being located in Switzerland, most of the traffic to Google IP addresses is routed to the US, which shows that Google’s data centers are primarily located and operating in the US. During

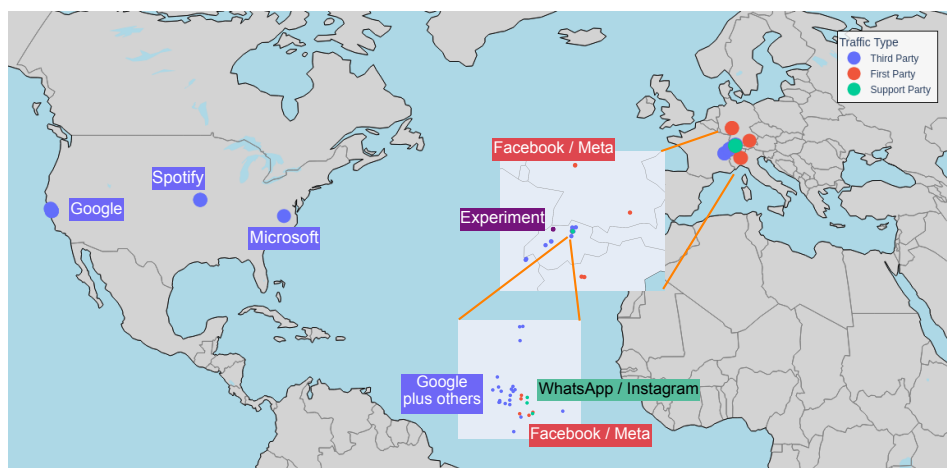


Figure 7.17: Destination addresses for all traffic captured across the five repeated trials of the voice-based commands experiments.

this experiment, most third-party addresses in Switzerland, illustrated in Figure 7.17, were third parties that contributed negligibly in terms of traffic volume. However, its location still exposes the general location of the user.

An interesting observation about the destination of first-party traffic is that the IP addresses of these traffic point to locations in Germany and Italy, compared to the destination for touch-based commands being in Switzerland. A deeper analysis of the packets of these IP addresses show that they are part of a process called STUN/TURN, which helps devices connect to each other over the internet behind firewalls or routers. The TURN server for those IP addresses acts as a middleman when direct connections between the devices are not possible.

In this case, Meta is routing the traffic through its TURN servers in Germany and Italy. As a result, most first-party locations are directed to Meta servers in Europe, more specifically Germany and Italy. The additional addresses only occur during the last minute of the experiment when there is an incoming phone call. This means that for most user interactions, the servers in Switzerland are able to handle the commands. However, phone calls require a more distributed system to handle communication more efficiently, causing the need for servers in Germany and Italy. As a result, based on the destination servers of first-party traffic, a third-party entity could also infer when the user is on a phone call. The map also shows that the location for support-party traffic and most third-party traffic destined for Zürich, Switzerland, which allows for an approximation of the user's location.

7.2.3 Pairing Sequence

From initial hands-on usage of the glasses, it is revealed that the device would not allow pairing without an internet connection. The pairing process starts with the initial discovery and establishing a connection between the Ray-Ban Meta and the phone. Then, data is exchanged to authenticate and validate the devices through secured channels. The reason for this authentication is to prevent two devices from connecting to the glasses at the same time. Due to the fact that the glasses store images locally, if another person were to be directly able to connect to the glasses, they would be able to see those images after pairing them with the glasses, meaning a breach of user privacy.

From the traffic capture, the split between public and private traffic volume was relatively even, with public network traffic representing more than 47% of the total traffic volume, an estimate of 1.25 MB. Most public traffic volume stems from first-party traffic, contributing more than 98% of the traffic volume, nearly 1.25 MB on its own. No support-party traffic was captured at all, and only about 20 KB of third-party traffic was recorded. Some communication with external cloud servers, mainly Google, was observed in the traffic. Figure 7.18 shows the drastic difference in traffic volume of the teal and green IP addresses compared to the rest of the addresses in the figure. Each cyan line in the plot indicates a new round of pairing attempts. From the initial overview, the traffic appeared to have no relative pattern due to the inconsistencies and differences in traffic volume. However, a deeper look into specific packets shows a certain sequence of traffic that can be observed during pairing between the devices.

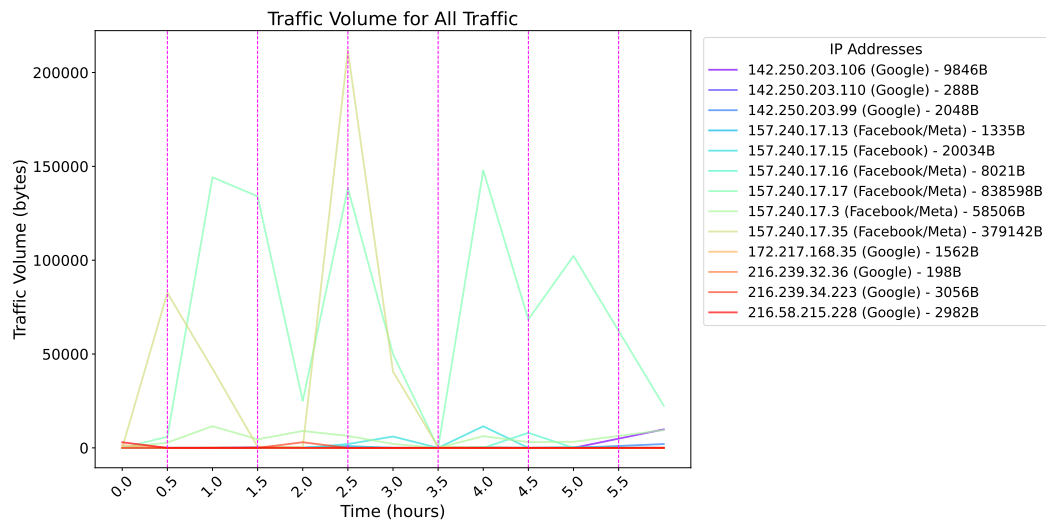


Figure 7.18: Public network traffic that contributed more than 100 bytes captured in the initial pairing experiment, with the data categorized by IP addresses. The legend displays each IP address along with its corresponding ISP or Service if known, and the Total Number of Bytes.

It begins with a sequence of DNS queries. DNS lookups involved include `ar.graph.meta.com` and `graph.facebook.com`. these domains were also present in the captures of the experiments for touch-based and voice-based commands during the initial powering on of the glasses. This is then followed by a sequence of TCP handshakes, including packets with SYN or SYN-ACK, which is for the establishment of secure communication channels and TLS handshakes to ensure the integrity of the data exchanged. Afterwards, when the secure channel is established, packets with TLS protocols with application data are sent, which most likely include

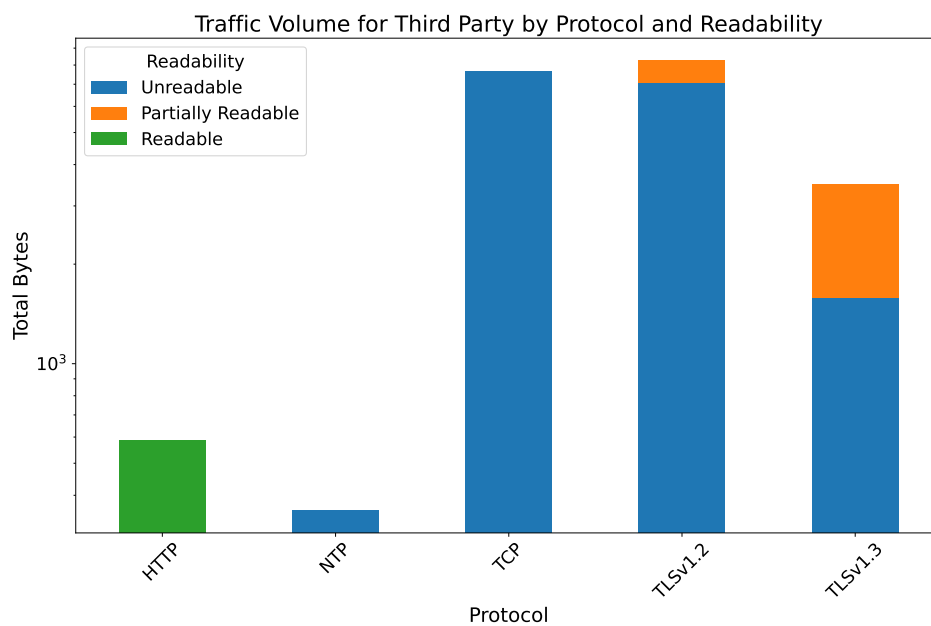


Figure 7.19: Third-party traffic captured during the initial pairing experiment, with the data categorized by readability and protocol.

sensitive data related to pairing, including authentication credentials. This process mainly used TLSv1.3, with a couple of packets using TLSv1.2.

The traffic with TLS traffic also included some Client Hello messages that reveal the domain name being queried, including `graph.facebook.com`, therefore are considered as partially readable, as seen in Figure 7.19. However, in the figure, there is a similar amount of TCP, TLSv1.2, and TLSv1.3 traffic, most of which remains unreadable. Most of the third-party traffic was unreadable, as seen in the figure. The third-party traffic was mainly to Google, while none was to Spotify. However, the only packets captured for the packets to Google were TCP protocols used for connectivity checks and roughly 10 packets of secure communication. This suggests that communication is more likely part of general internet connectivity rather than a dependency on the pairing process for services like authentication. The content of the readable HTTP traffic was the same as in voice-based commands, but here, four packets of HTTP traffic were captured, meaning two sets of the same HTTP GET request and HTTP response packets. The sequence of packets, as well as the domain names revealed in the pairing process, would allow external observers to deduce that it is from the initial pairing of the devices and then monitor the pairing process or further interactions, which allows for better tracking and user profiling.

7.3 Idle Traffic Analysis

An 8-hour capture was conducted while the glasses and phone were idle, aiming to identify any background activity that could pose potential privacy concerns. Public and private network traffic was once again quite balanced, with public traffic having roughly 48% and about 0.83 MB of traffic.

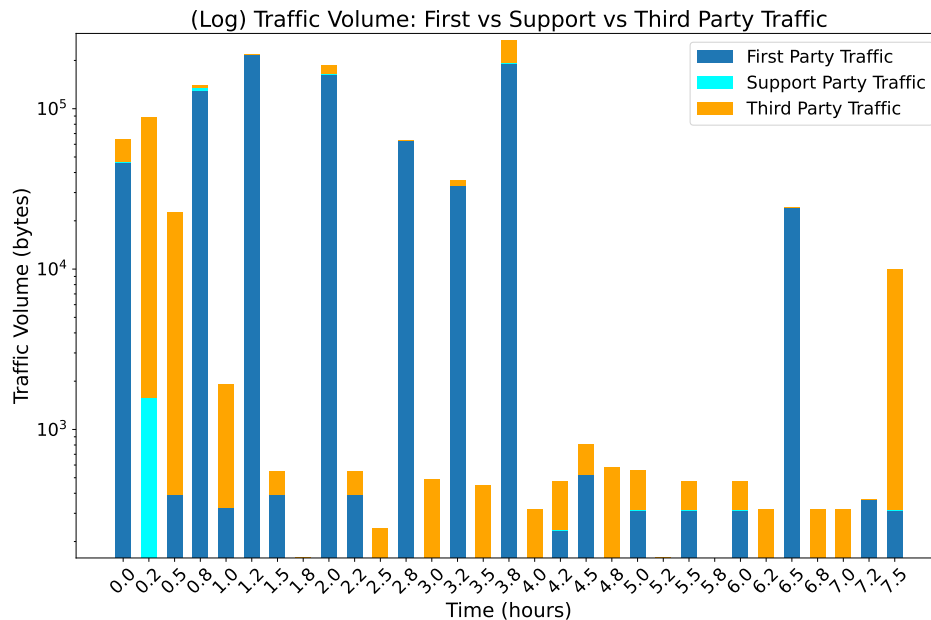


Figure 7.20: Public network traffic captured during the idle traffic experiment, with the data categorized by party type.

Figure 7.20 shows that nearly all of the traffic captured occurred in the first four hours of the capture. Compared to active traffic, idle traffic had a much higher percentage of third-party traffic, contributing slightly above 10% of the total traffic volume. There is minimal support traffic, and they appeared only during the initial half-hour of the experiment.

Figure 7.21 illustrates the traffic volume for all addresses that contributed more than 1100 bytes of volume. As depicted in the figure, most of the fluctuations arose from fluctuations in first-party traffic. The highest traffic volume appeared at the start of the capture, coinciding with when the glasses were about to be closed, placed into the charging case, and the phone switched off. Thereafter, the traffic sent to Meta servers occurs in bursts rather than in a consistent flow, although the traffic volume dropped to nearly zero after the four-hour mark. The traffic bursts may represent updates being sent or synchronizations with the glasses.

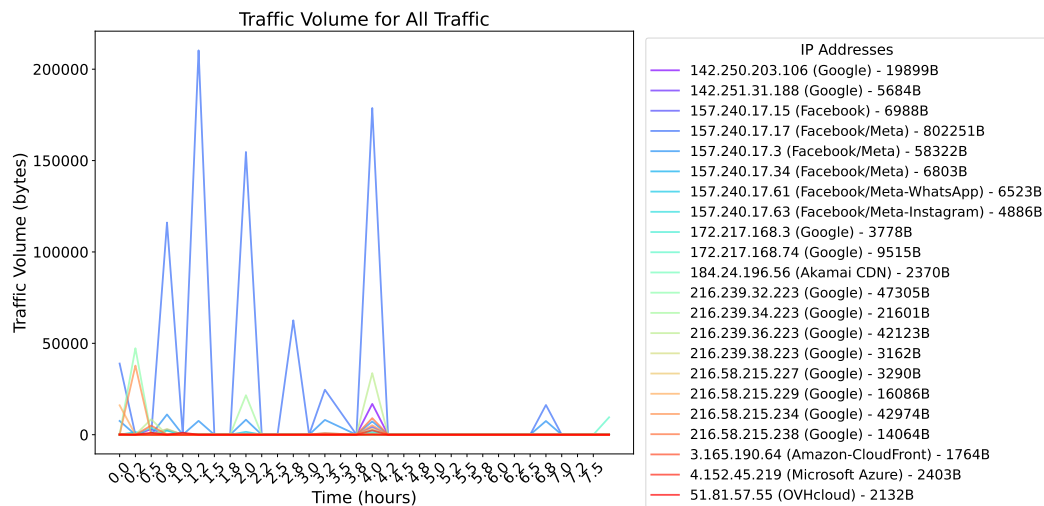


Figure 7.21: Public network traffic that contributed more than 1100 bytes captured during the idle traffic experiment, with the data categorized by IP addresses. The legend displays each IP address along with its corresponding ISP or Service if known, and the Total Number of Bytes.

Another possibility is that an app is running some unknown and possibly unnecessary background task. Due to the traffic being encrypted, the exact content being sent is unknown. Although not armed with knowledge of the content of the packets, the traffic metadata still reveals the user's geological location, as nearly all traffic is routed to Zürich, Switzerland. Although it does not pinpoint the user's exact location, it still provides a general location to determine where the user is located. The red line, shown to have the most fluctuations, also exists during active captures, which are usually more active in minute 0.5, which is when powering on the glasses and opening the Meta View app. Although the traffic volume is not as high, this address continues to have a constant presence throughout the remaining time of the active captures. This suggests that this destination address could mainly be for connection purposes. The destination for this IP address is the Meta server in Zürich, Switzerland, suggesting that all Meta traffic routed to the US is due to active interactions with the glasses.

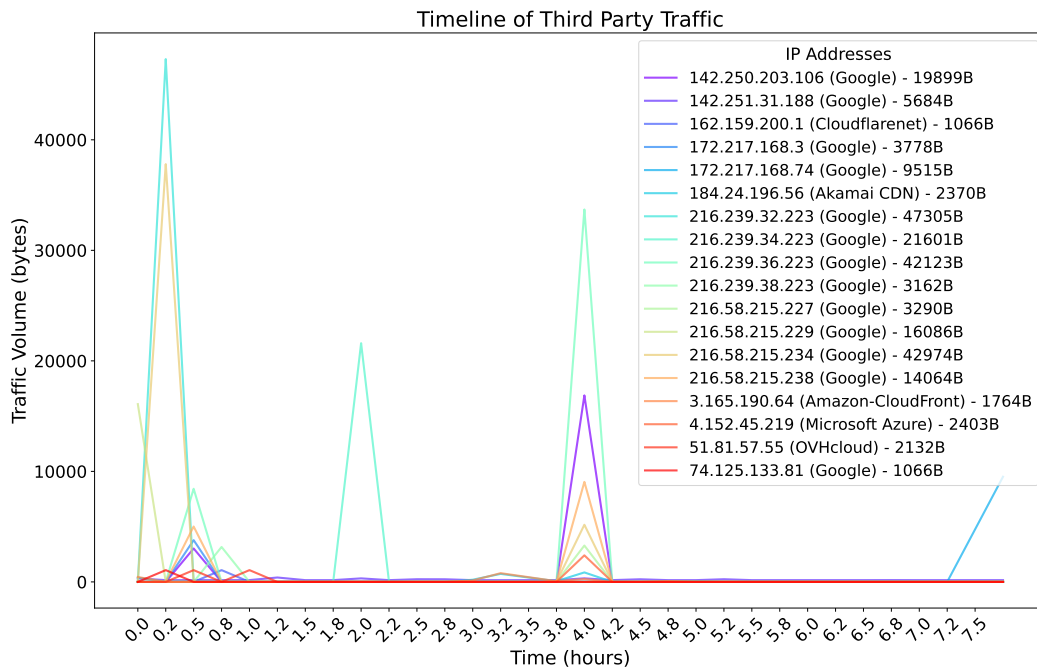


Figure 7.22: Third-party traffic that contributed more than 1000 bytes captured during the idle traffic experiment, with the data categorized by IP addresses. The legend displays each IP address along with its corresponding ISP or Service if known, and the Total Number of Bytes.

Third-party addresses that contributed more than 1000 bytes during the idle traffic experiments are illustrated in Figure 7.22. An intriguing note in the figure is that every single spike in the traffic is from a Google IP address, but each spike represents a different IP address. For idle traffic, the third-party traffic was nearly all unreadable, which was also observed for each spike, which were packets with protected payload with the QUIC protocol. The timing for the third-party traffic also aligns well with the peaks observed in first-party traffic, which implies interdependencies between first parties and third parties. As mentioned, a Firebase database was identified in the app analysis 6. It could be analytic events via Firebase that are triggered somehow, which results in these simultaneous first-party and third-party traffic occurrences. The locations to these addresses showed a similar pattern as in active traffic, with Google and, this time, Microsoft in the US, whilst the rest of the traffic was mainly routed to Zürich, Switzerland.

Finally, Figure 7.23 shows that the readability of all traffic captured during the idle traffic experiment was predominantly unreadable, with few packets being partially readable due to the exposed domain names, such as `www.googleapis.com`, in the payload. Unlike the active interaction experiments, there was no readable traffic. Another difference found was that idle traffic also captured non-readable DHCP traffic, which did not exist in the active traffic captures. However, these are DHCP ACK packets, which contain negligible information and serve only to confirm IP address assignment. The unreadable traffic and what can be understood from partially readable traffic have the same observations as in active traffic interactions.

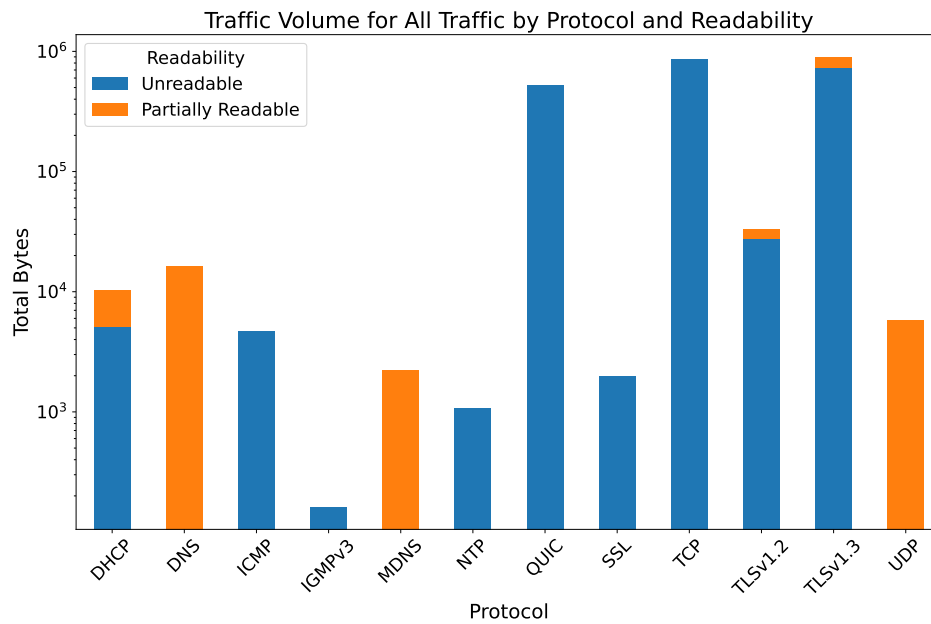


Figure 7.23: All network traffic captured during the idle traffic experiment, with the data categorized by readability and protocol.

7.4 Summary

Nearly all of the captured traffic was encrypted and, therefore, unreadable, which made the payload content inaccessible. However, findings from the experiments demonstrated that, despite the content of the packet being unreadable and without the ability to decrypt the payloads, there are still risks and concerns for user and bystander privacy. The analysis showed that purely observing the traffic patterns and using the metadata from the traffic, including packet size, timing, domain names, and destinations of IP addresses, is enough to expose application usage, user behavior, and the geological location of the user, allowing for user tracking and profiling. For the Ray-Ban Meta glasses, a few interactions stand out, including media transfer and receiving messages on WhatsApp.

The inability to know the traffic also poses a privacy concern to users who themselves are not aware of and therefore not able to give consent to what user data or activities may be tracked or collected. The traffic showed minimal interactions with third-party destinations, which assists in reducing the number of possible attack vectors and protecting user data. However, it is plausible that the service providers may process the traffic routed to Google or Spotify. Also, although the traffic captured did die down after four hours, the lack of transparency surrounding background tasks during the idle state raises questions about whether there may be continuous data capture without the user's awareness, which could potentially impact bystander privacy if audio or images are inadvertently transmitted.

8

Bluetooth Analysis

This chapter analyzes the Bluetooth communications observed between the Ray-Ban Meta glasses and the phone. The purpose of the analysis is to understand the device functionality of the Bluetooth connections of the device and to support potential decryption efforts related to the network traffic.

8.1 BLE Analysis

An initial overview of the captures reveals frequent advertising initiation (ADV_IND) packets, reflecting periodic advertising from nearby devices, as well as Non-Connectable Advertising (ADV_NONCONN_IND) packets, which are the advertising packets not intended for direct connection. Although both packet types are continually present, ADV_NONCONN_IND packets are much more consistent, whereas ADV_IND packets show large fluctuations. Although relatively consistent, the rest contributes a much smaller traffic volume. Occasionally, malformed packets were captured, which could be due to transmission errors, collisions, or incomplete captures.

Figures 8.1 and 8.2 were derived from the initial pairing experiment of BLE traffic. Figure 8.1 shows the total volume of traffic over time based on the Information column, whilst Figure 8.2 shows the comparison of the traffic of the glasses and the phone over time, which shows that traffic from the phone shows more significant fluctuations than that from the glasses. A closer comparison of the figures indicates that the green line in Figure 8.1, representing ADV_NONCONN_IND packets, closely mirrors the blue line in Figure 8.2, which depicts the phone traffic. Likewise, the orange lines in both figures show that the ADV_IND packets follow an analogous pattern to the traffic of the glasses. This observation is confirmed by examining the capture data, where most ADV_IND packets originate from the glasses, and ADV_NONCONN_IND packets originate from the phone. Neither device contributes to the other category of traffic packets, which aligns with the BLE design for energy efficiency. The similarity between the traffic volume patterns further suggests that the controlled environment used for these experiments successfully isolated other sources of interference.

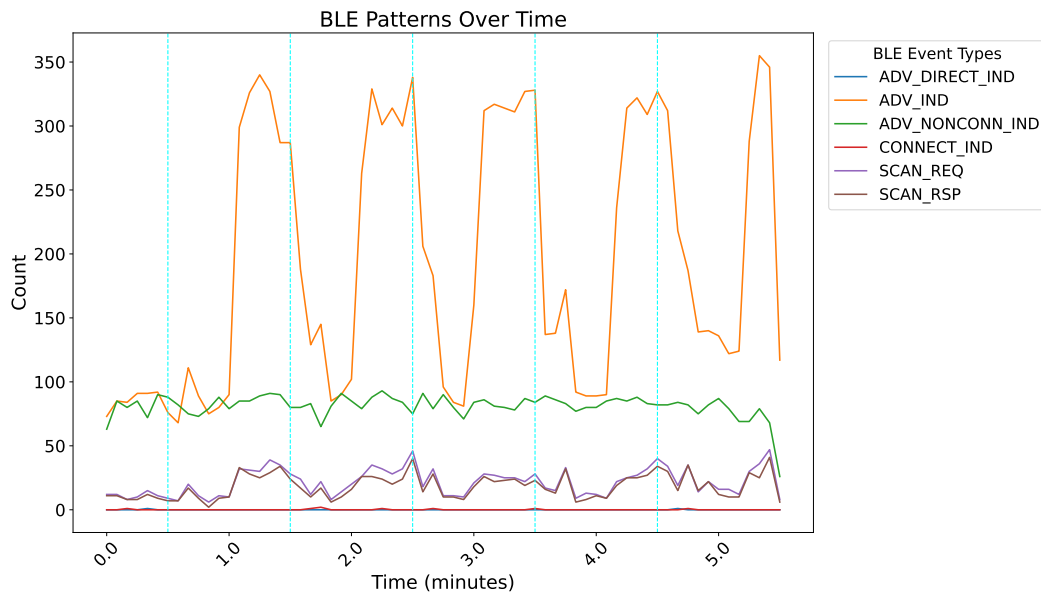


Figure 8.1: Packet Count captured during the initial pairing experiment, with the traffic categorized based on their event types.

The vertical cyan markers in Figures 8.1 and 8.2 identify the start of each pairing round. The phone shows a relatively consistent traffic volume, with no considerable fluctuations in advertising traffic throughout the pairing of the devices, reflecting its role as a central device scanning and initiating connection. The glasses traffic, on the other hand, acting as a peripheral, shows notable yet consistent fluctuations that align with the timing of the rounds of pairing performed.

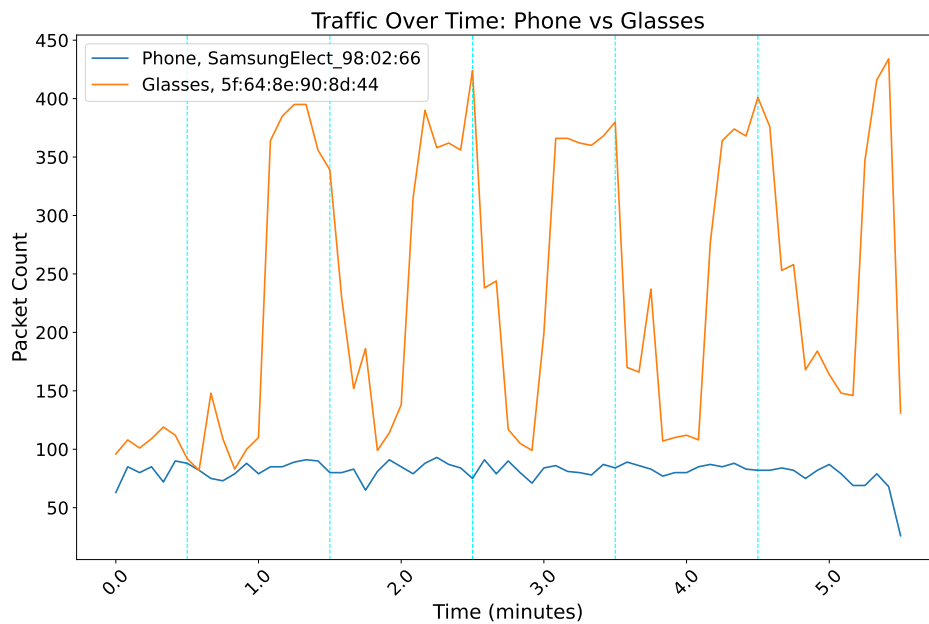


Figure 8.2: Packet Count from the Glasses or Phone during the initial pairing experiment.

At the start of each pairing round, the overall traffic volume for both devices is similar, suggesting that their normal advertising volume announcing their availability is roughly the same. When the pairing begins, the traffic volume does not immediately increase, suggesting that pairing the devices does not introduce additional advertising or scanning activity. This behavior matches the design of BLE, where unnecessary transmissions are limited to maintain efficiency. Approximately 30 seconds into the pairing process, the traffic volume increases significantly. The timing coincides with the glasses being unpaired from the phone, potentially reflecting the protocol steps needed to terminate the previous connection and resume broadcasting ADV_IND packets to signal their availability. Afterwards, the original traffic volume reduces to its original level, possibly due to the glasses reducing the frequency of their ADV_IND packets designed to conserve energy while the glasses remain discoverable.

Experiments involving touch-based and voice-based commands displayed a different situation. While the phone continued to appear in the traffic capture with the same consistent traffic volume as in the pairing experiments, the glasses did not advertise nor communicate any BLE traffic once they were connected to Bluetooth and configured in the app.

8.2 Classic Bluetooth Analysis

The classic Bluetooth experiment analysis assumes that the phone acts as the master device while the glasses serve as the worker device. The average signal strength values recorded portray the stability of the experiment environment. During the touch-based commands experiment analysis, the glasses had an average SP of -23 dBm compared to -38 dBm for the phone. Meanwhile, during the voice-based command experiments, the glasses averaged -28 dBm while the phone averaged -36 dBm.

Figures 8.3 and 8.4 illustrate the average packet count for the touch and voice commands experiments. Both figures show a pronounced spike in the first minute the glasses are powered on, where both devices send packets during pairing. This period resulted in one of

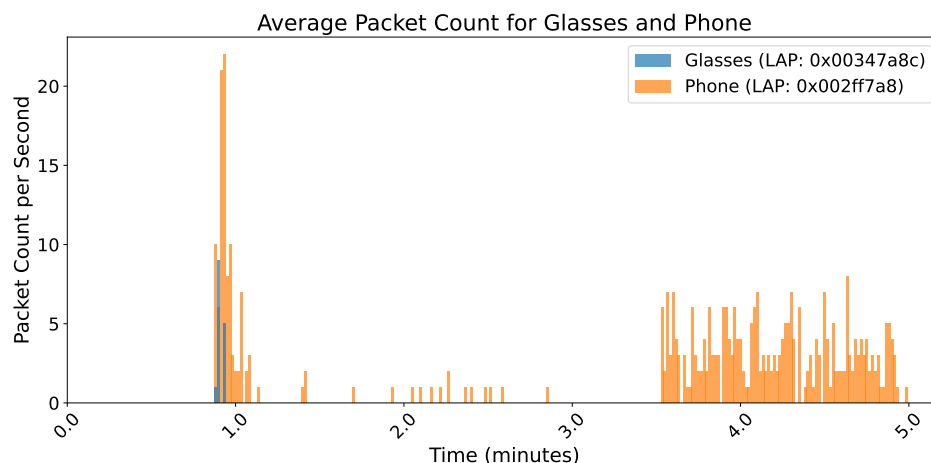


Figure 8.3: Average Packet Count for the phone and glasses in the two trials of touch-based commands experiment

the largest spikes in traffic throughout the experiments. This burst of activity then subsides, with the glasses appearing to cease further transmissions. The cease in traffic suggests that once a link is established, the glasses use the LAP (Lower Address Part) of the phone for communication rather than maintaining its own separate stream.

In the touch-command experiment, in Figure 8.3, a clear pattern arises in which commands trigger a nearly immediate packet transmission. However, no observable traffic appears during media transfer minute 3.0. This is also observed in Figure 8.4, at minute 9.0, confirming that these transfers occur over network traffic rather than through Bluetooth. Music playback, on the other hand, consistently generates traffic, suggesting that audio streaming relies on Classic Bluetooth.

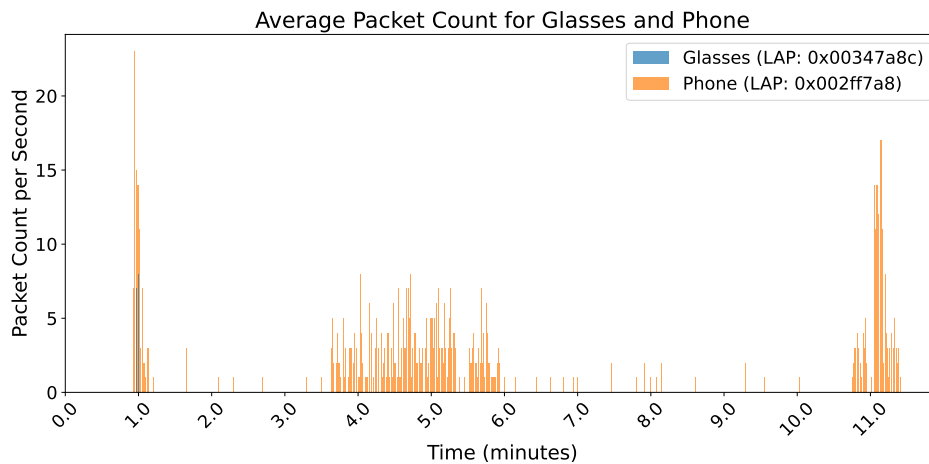


Figure 8.4: Average Packet Count for the phone and glasses in the two trials of voice-based commands experiment

There was a recurring issue during the experiments with Ubertooth One, where it would occasionally exhibit time jumps in the captured data, meaning that the timeline would sometimes advance abruptly, creating discontinuities. Although these anomalies did not appear in the final touch-based commands experiments, they are present in the voice-based commands experiments. This anomaly is evident due to the original captures in both experiment trials extending beyond fourteen minutes, whereas the actual experiment lasted only eleven and a half minutes.

Upon closer examination, both trials revealed two intervals exceeding one and a half minutes each with no recorded activity. Given that the experiment's interaction intervals were thirty seconds and the expectation that Classic Bluetooth devices typically send periodic packets for device discovery and keep-alive signals, these inactive periods were removed from the timeline. The final confirmation that these two inactive intervals were indeed the jumps in the capture was that the subsequent duration of the experiment after removing these intervals was around eleven and a half minutes, which matched the expected duration of the experiment.

Excluding these anomalies, the captures show that most user commands trigger packet transmission. Two exceptions were questions about the date and time, which did not produce

immediate traffic. This is presumed to be due to the glasses having this information locally and the phone sending updates only when a change occurs, or it is because this is being handled through network traffic. Continuous traffic was observed during music playback, ranging from 0 to 10 packets per second. Further commands related to timers and media capture resulted in little packet transmissions with none for the command to import media captures from the phone to the glasses. In the voice-based commands experiments, the command to transfer media capture, executed at minute 9.0, also resulted in no observable packets. Although there was some traffic during the thirty-second interval for this command, it could be from the periodic packets being sent, indicating that these commands are handled through the network traffic. The second significant spike in traffic occurred upon receiving a phone call, reflecting the involvement of audio streaming during such interactions.

Sending or receiving a text or voice message to a person produced almost no traffic, suggesting that voice recording and sending messages are mainly handled via network traffic. On the other hand, asking the AI to read out the received message aloud generated more noticeable traffic, although it was less intense than music playback. A possible reason for this traffic could be that the readout functionality is generated directly on the glasses and does not rely on the phone for processing, which would align with Meta's focus for the glasses to be more autonomous. Although this enhances privacy by reducing network dependency, the fact that the glasses are capable of processing sensitive data locally introduces other concerns. If the glasses are stolen, lost, or compromised by unauthorized access, this data could be directly exposed. Furthermore, users may not be fully aware of what information is being stored or how long it remains on the glasses after processing, raising transparency issues about data collection and retention. The message itself could also contain information about bystanders who are completely unaware of their data being handled and processed, increasing bystander privacy concerns.

In summary, these identifiable traffic patterns indicate that intercepting the Bluetooth streams could result in third parties inferring certain user interactions, such as music playback and phone calls. A determined observer could use the packet timing and volume to identify specific events or commands performed on the glasses without the need to decrypt the communication. Additionally, the potential capabilities of the glasses to process sensitive data locally create privacy risks and transparency concerns.

9

Decryption

Attempts were made to determine whether any personal data or raw sensor readings could potentially be leaked through network traffic between the Ray-Ban Meta glasses and the paired phone and the limitations encountered. This chapter also mentions the decryption attempts made in Bluetooth communications and the limitations encountered.

9.1 Decryption Attempts

The decryption attempts centered around intercepting and decrypting network traffic on a Xiaomi POCO X5 5G. However, there were many challenges throughout the decryption process. The following subsections detail the methods explored and problems encountered before and after rooting the phone.

Before Rooted Phone

The initial setup included installing mitmproxy and setting up the phone's Wi-fi proxy setting to route the traffic through mitmproxy. Initially, the phone was not rooted, but that did not affect the installation of the mitmproxy certificate as a user-trusted certificate on the phone. The outcome of the intercepted traffic showed that HTTP traffic was successfully intercepted. However, additional security checks, such as certificate pinning, were implemented on the app to prevent proxies such as mitmproxy from intercepting the traffic. This was evident in the proxy from the TLS handshake errors received, meaning that the certificate pinning implemented in the app was preventing the user-installed certificate from being trusted.

Different approaches were explored when attempting to bypass the certificate pinning, including modifying the APK and using runtime debugging and hooking frameworks. These efforts either tried to disable the certificate pinning checks by modifying the APK or dynamically bypassing them during app execution. However, there were multiple problems during the process, including errors caused by attempts to access or modify SELinux policies, which control system permissions and require root access to change. Additionally, recompiling errors, which were related to resource compilation and malformed XML files, occurred while

repacking the APK. In order to address these problems, rooting the phone was determined to be the most logical next step.

After Rooted Phone

The rooting process began with unlocking the phone bootloader. This further required a Xiaomi account and a physical SIM card with mobile data. After obtaining the SIM card and beginning to unlock the bootloader, it was found that Xiaomi had security measures that mandated a seven-day waiting period after initiating the unlock process, and interruptions such as logging out of the Xiaomi account would reset the timer. A Windows machine was required to unlock the device because the official Xiaomi USB drivers are only natively compatible with the Windows operating system. Following the mandatory waiting period, initial attempts to root the phone using Team Win Recovery Project (TWRP), a common method for rooting phones, were unsuccessful due to the dynamic partition scheme of the phone and the lack of a traditional recovery partition, which is mostly the case for the newer phone models. Errors such as "Unable to mount /data" and partition encryption issues made it impossible to proceed with TWRP further. Therefore, the Magisk tool was used to patch the boot image file directly as a workaround. This method removed the need for TWRP and allowed the phone to be successfully rooted.

Once the phone was rooted, the focus shifted back to decrypting the network traffic, with the previous attempts being replicated with the rooted phone. However, rooting the phone should have allowed the installation of the mitmproxy certificate as a system-trusted certificate, for example, by using Magisk or directly trying to write it as such in the system partition. They proved to be unsuccessful, as the certificate did not appear as system-trusted, which is likely due to protections on the system partition to prevent injections or modifications. Therefore, the rest of the process continued with the certificate as a user-installed and trusted certificate.

Efforts to modify the APK remained unsuccessful, mainly due to errors when compiling the resources. The most common error was an invalid resource directory name for a directory that did not exist. An attempt was made to avoid the decompilation of the resource entirely by using the '-no-res' flag. However, the app still did not rebuild properly. Different packaging tools were considered for the failed build process. However, errors related to resource compilation and malformed XML files remained.

The other approach was to dynamically bypass SSL pinning when intercepting the traffic through mitmproxy. Rooting the phone solved the initial problems when dynamically bypassing the pinning. Objection was used to disable SSL pinning. The steps included pushing and starting the Frida server on the phone, then connecting Objection to the running app and disabling SSL pinning using their built-in command. Objection confirmed in the terminal that SSL pinning was disabled for implementations such as TrustManagers. However, the Meta View app continued to reject the proxy's certificate, resulting in TLS handshake errors for domains like graph.facebook.com. This suggests that the app is likely using custom certificate pinning implementations or additional verifications.

A custom Frida script was written to try to hook into implementations to practice

the certificate pinning. The decompiled APK, taken from the app analysis, was used to identify the custom methods. The app had multiple certificate validation methods, including `checkServerTrusted` and `TrustManager`. It was found that Meta regularly changes the names of the classes, which required consistent updates on the code that bypasses methods in explicitly mentioned classes. In the end, some endpoints, like `ar.graph.meta.com`, were successfully bypassed by injecting the script into these targeted methods. However, most endpoints remained unsuccessfully bypassed. From the successfully bypassed endpoints, no personal or sensitive data was exposed. The decryption process displayed strong security and network traffic protection, reflecting its safety against MitM proxies for users. The bypass is shown in the Frida script in Figure 9.1, where the custom code was successfully hooked into the relevant classes and overloaded their security methods.

```
Starting Certificate Pinning Bypass for checkServerTrusted
Found class: X.0F1
Found class: X.0Ax
Bypassing checkServerTrusted in X.0F1 with Socket
Bypassing checkServerTrusted in X.0F1 with Socket
Bypassing checkServerTrusted in X.0F1 with Socket
Bypassing checkServerTrusted in X.0F1 with Socket
```

Figure 9.1: Screenshot of the terminal output when running Frida, showing that Frida found the targeted classes and bypassed `checkServerTrusted` methods within the class.

The bypass is confirmed in Figure 9.2, which shows a snippet of the terminal output when running the custom Frida script. The figure shows that there were no TLS handshake errors when running `ar.graph.meta.com` nor `scontent.xx.fbcdn.net`, meaning they were, indeed, successfully bypassed. The TLS handshake fails in the figure, which shows an example of the rejection of the proxy's certificate for `googleapis`, providing an example that although the certificate pinning for some endpoints was bypassed, most were not.

```
Proxy server listening at *:8080
192.168.1.194:44902: client connect
192.168.1.194:44902: server connect ar.graph.meta.com:443 ([2a03:2880:f05b:12:face:b00c:0:2]:443)
192.168.1.194:41706: client connect
192.168.1.194:41706: server connect scontent.xx.fbcdn.net:443 ([2a03:2880:f05b:10:face:b00c:0:3]:443)
192.168.1.194:38830: client connect
192.168.1.194:38830: server connect volatile-pa.googleapis.com:443 ([2a00:1450:400a:801::200a]:443)
192.168.1.194:38830: Client TLS handshake failed. The client does not trust the proxy's certificate for volatile-pa.googleapis.com (OpenSSL Error([('SSL routines', '', 'ssl3 alert certificate unknown'])))
```

Figure 9.2: Screenshot of the terminal output when running mitmproxy, showing that endpoints like `ar.graph.meta.com` did not receive a TLS handshake error, while endpoints like `pa.googleapis.com` did, meaning that not all certificate pinning was bypassed.

Decryption Efforts on Bluetooth

Apart from the possible transmission of sensitive user data over Bluetooth, the Ray-Ban Meta glasses use Bluetooth during the initial pairing, which includes exchanging encryption keys and authentication credentials to establish a secure connection over the network. Therefore, another idea was to try to decrypt Bluetooth traffic to obtain these keys. However, in the Bluetooth captures, no information, including keys, could be identified in the

captures due to the keys transmitted being encrypted. The setup for BLE pairing uses Elliptic Curve Diffie-Hellman (ECDH) for key generations, and the keys needed to decrypt the BLE traffic included the Link Key, the identity resolving key, and the signature resolving keys. However, to be able to read the data, keys like the Link Keys or Long-Term Keys need to be extracted during the pairing process. The rooted phone has the ability to log all Bluetooth traffic. Therefore, this log file was pulled to try and find the necessary keys for decryption. The keys found on the file included all the necessary keys. However, although these keys were found, applying the keys to the captures for decryption was unsuccessful.

9.2 Decryption Analysis

A couple of domains were successfully bypassed in mitmproxy using custom Frida scripts. The full list of domains can be found in A.6. There were a few endpoints that were less relevant to the privacy concerns of the glasses, for instance, the Android connectivity checks, <http://clients3.google.com/generate> 204, which are simply to confirm internet access. The traffic captured that is relevant to the glasses falls into the following three categories: Reliability Event Logging, Meta Media services, and Federated Learning.

Reliability Event Logging

There were multiple POST requests to https://www.facebook.com/mobile/reliability_event_log_upload/. The data, were typically sent as multipart/form-data, which allows multiple fields and binary chunks to be included in one request body. These requests most likely are part of a mechanism that regularly sends diagnostic, crash logs, or usage data from the app back to Meta servers. The purpose of these requests is to upload events about app usage or crashes to Meta servers for debugging or performance statistics.

Although they are primarily used for debugging, these logs can contain usage patterns, such as how frequently a user opens certain features or the device states that lead to the errors. The brackets for the following information show what was written in an example flow in the capture. For instance, data fields, including the `device_id`, `device_brand`, `user_id`, and `has_wearable_device_paired(=True)`, could tie the companion app's crash or usage logs back to a specific user. Data such as `exit_info_reason (= 10)` and `exit_info_subreason (= FORCE STOP)` confirm that the app was forcibly closed, while fields like `attempt_count` revealed repeated attempts to launch or execute an action. Other fields, such as `previous_build_id`, `report_build_id`, `device_brand (= POCO)`, or `consent_choice`, can also reveal information about the crashes, environment details, and user preferences. Although the logs do not contain direct raw data, the repeated references allow Meta's servers to correlate sessions with the application usage, potentially accumulating a timeline of user activity profiling.

Meta Media Services

The decrypted traffic reveals instances in which the companion app is requesting and displaying official resources from Meta's servers, primarily through the endpoint side.facebook.com/ras/v2?id=. The examination of these GET requests shows images or

application/octet-stream responses. The response type application/octet-stream combined with the size of these responses (an instance showed to be 9.35 MB), suggests that the downloaded file could be a video or a high-resolution image. Figure 9.3 provides examples of exposed images. The left side is the PNG image preview in the response that was triggered from one of the available in-app guides on how to use the glasses on the home page, whereas the right side is a full response to asking for the image of the glasses.

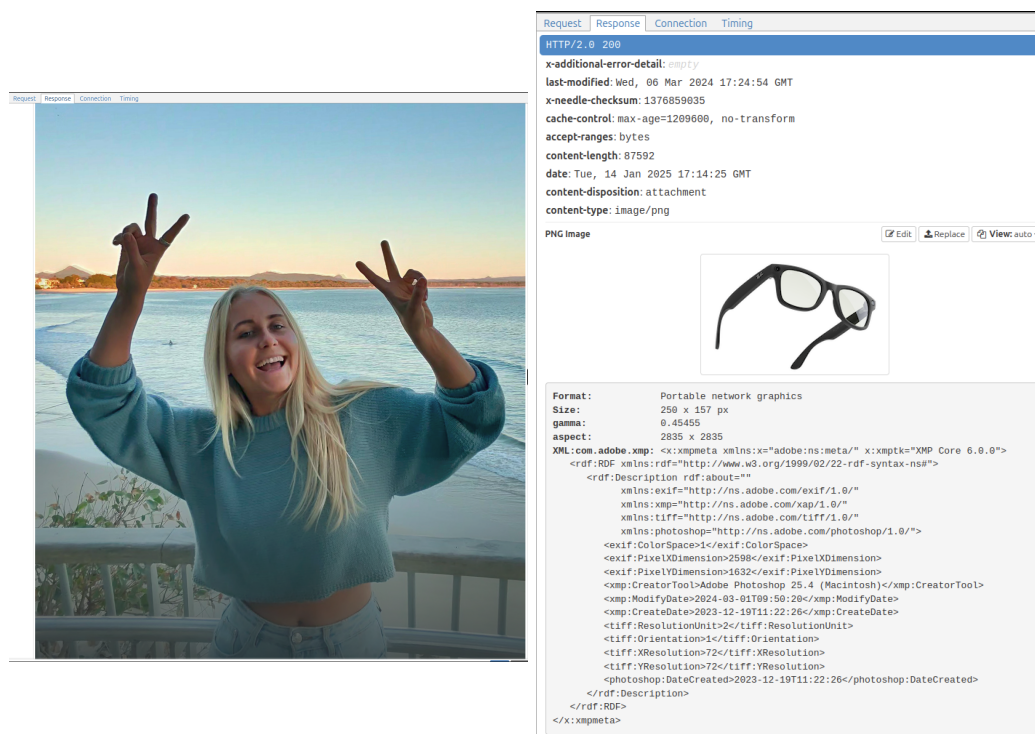


Figure 9.3: Responses to lookaside requests for images.

The GET request shows how the app uses lookaside.facebook.com to retrieve an image resource. The request for the flow on the right side of the figure only had one parameter, `id=AQADK1nKSW9`, which is likely a unique resource identifier on Meta's servers. The response, already shown in the figure, displays the header, which contains the content type, length, and other information. The body of the response contains the actual PNG image, which is a picture of the Ray-Ban Meta glasses. Below that is the metadata, in the form of XMP tags, of which the XMP data located in the `XML.com.adobe.smp` tag and information in EXIF data, under the EXIF namespace, can be found. These revealed the date of when the image was created (2023-12-19T11:22:26) and last modified (2024-03-01T09:50:20), the image resolutions (2598 x 1632 px), and the software used to create or edit the image (Adobe Photoshop 25.4).

However, it is unclear whether the companion app relies on similar mechanisms for images captured by the user, as they were not part of the decrypted traffic. The traffic captures only reveal the retrieval of standard resources. It was noted that the size and ID of these images were shown to be the same after repeated captures, suggesting a certain mapping between an ID and their respective media in the servers. For example, the image

for the glasses on the right side of the figure has an ID of AQADK1nKSW9 and a size of 85.8 KB. This means that, in theory, if images created by the user were also using the same mechanism, anyone aware of the relevant IDs could request the corresponding media. However, it is important to keep in mind that there was no direct evidence in these captures that personal images are shared in the same way. The presence of metadata, such as timestamps, editor tags, and file properties, can expose sensitive information such as the creation dates, camera or image information, or geological location details. Together, although seeing an official product image is relatively harmless, these aspects would pose a significant privacy concern if user photos are stored in a similar manner, as then the data would include camera details and the location of a user's device.

Federated Learning

Throughout the traffic capture, there were multiple sub-endpoints to `ar.graph.meta.com`, including `fl_acs.checkin` and `fl_acs.report`. In the request body, the `clientAssignment` block has attributes, such as `taskAssignment` and `clientRoundId`, which suggest that the app is participating in some form of federated learning. In general, Federal learning is a user-friendly concept that allows the phone to perform computations or data processing tasks locally in multiple generations or rounds without sending raw data to a server, hence a privacy-preserving local ML. Then, the client communicates the partial result to a central server, where the server then aggregates the partial result to refine a global model.

Specifically looking into a POST request of `ar.graph.meta.com/federated.learning/fl_acs_download` as an example, the query parameters include task ID, client round ID (`=3873906`), generation number (`=1048313`), and sequence number (`=8406`). These parameters indicate that the companion app is well into the training or update cycle, whereas the `shardID` (`=0`) implies that if the model is split into multiple segments, then this is dealing with shard number 0. The `chunkInfo` also provides the offset (`=21606`), which indicates where the next piece of data would begin. The server's response to this includes two main components, a `client_exception` and a response string. The response includes another chunk, which includes the offset as seen in the request, indicating that the chunk is recognized, but the size (`=0`) shows that there is no new payload at that time. There was a trace block that enumerated several stages, including 100, 110, 200, 210, ..., with their corresponding timestamps, such as 1736938938240614. Each stage likely denotes a step in the server's internal pipeline, which could be validating the client request.

Essentially, this flow shows that the companion app is telling the server that it is on round number 3873906 for generation number 1048313, dealing with a training or data-shard assignment of task ID 4931, and currently retrieving a chunk at offset 21606. The server then replied that, for now, there is no chunk data to download, as the size is zero. This suggests that the app could either be waiting for further instructions or supply the server with local updates from the user's device computations.

Federated learning is often considered more protective of user privacy, especially in comparison to sending raw user data, as the raw data theoretically never leaves the device as it transfers model updates. However, from the capture, there appears to be a cyclical

process of uploading partial results, reporting progress, and then checking in with the central server. These repeated calls raise questions about exactly what data is traveling to Meta's servers, as they could be aggregated to produce a detailed timeline of when and how the device is used or what local data it may be processing. The example flow shows that the device is an active participant in a large-scale iterative process. And although the content in this instance is presumably an empty chunk, future requests could carry partial model or personal user information. Furthermore, some federated learning implementations could unintentionally gather partial personal user data or aggregate user statistics, which, if combined with other fields, allow the server to identify each individual app's progress in their tasks, and the server would be able to link an activity to a specific user aiding in user profiling.

10

Discussion

This chapter discusses the findings derived from examining the Ray-Ban Meta glasses. The examination comprised an overview of the device's hardware and software (Chapter 4), an assessment of the companion app (Chapter 6), analyses of network and Bluetooth traffic (Chapters 7 and 8), and an overview of the Decryption attempts and analysis (Chapter 9). By combining these perspectives, a clearer image of the privacy implications of the glasses and the concerns of data collection and retention, user control, and bystander consent can be painted. The discussion is organized around key topics, metadata exposure and user activity inference, companion app dependencies, trade-offs between functionalities and privacy, and policy-oriented reflections, including bystander privacy.

Metadata Exposure and User Activity Inference

The network traffic analysis reveals that, although end-to-end encryption was used for the traffic, mainly via TLS and QUIC protocols, users are still vulnerable through metadata exposure, which allows for inference of application usage and user activities.

Large packet transfers (more than 10^5 bytes when grouped in ten-second intervals) usually coincide with interactions such as media transfer or issuing voice commands. Media transfer has the largest traffic volume spike compared to all other interactions. Media transfer also has a distinct sequence of packets that starts with four ICMPv6 packets, which appeared exclusively during media transfer. Other voice commands also have different sequences of packets. For instance, sending a message to a person involves a back-and-forth exchange of around six to eight QUIC packets with payload, followed by roughly four packets of empty QUIC packets, whereas asking for the weather typically would have roughly six consecutive QUIC packets that contain payload followed by four empty ones, which was then continued with around eight QUIC packets with payload. These traffic patterns allow an adversary to intercept the communication to reasonably deduce user actions without the need to decrypt the content.

A similar risk exists in Bluetooth communications as the glasses rely on Bluetooth for pairing, periodic advertisements, and device management. An adversary capable of sniffing BLE advertisements or tracking the device's Lower Address Part (LAP) in Classic Bluetooth

would have a stronger basis for user profiling or inferring specific user interactions based on traffic volume. BLE's use of advertising data, including a Company ID (0x01ab for Meta), simplifies the identification of the glasses and correlates transmission to specific features, such as pairing events or periodic advertisements. On the other hand, Classic Bluetooth captures showed traffic bursts that revealed active usage windows, with most traffic volume during music playback or voice-based audio streaming, including asking the AI to read received messages. The glasses with the potential ability to locally process messages received could be vulnerable if the glasses are stolen or compromised. Furthermore, the lack of user awareness about what data is stored and for how long it is stored raises concerns about transparency and bystander privacy.

Most network transmissions were well-protected using TLS or QUIC protocols. However, the partial success in bypassing certificate pinning revealed additional insights into the data flow. For instance, a decrypted response from `ar.graph.meta.com` revealed that the app actively transmits details such as device IDs and session states to the server. These identifiers allow Meta's servers to monitor app activity and, when combined with other logs, could be aggregated to infer user behavior. Additionally, the repeated requests to `lookaside.facebook.com` demonstrated how the app fetches image resources. It is harmless for official product images but raises concerns about how images captured by the user may be processed or stored. These findings reinforce the notion that even with encryption, determined adversaries can extract not only metadata but also content-level information, potentially revealing user activities and exposing bystander information. Although the stronger certificate pinning prevented the decryption for the other endpoints, it leaves the question of the exact nature of the data being collected and retained within the more protected transmissions.

Companion App and Data Retention Concerns

The static analysis of the companion app for the Ray-Ban Meta glasses showed that it does not request excessive permissions nor exhibit excessive reliance on external third-party services. By targeting specific permissions, an adversary could obtain location coordinates, social relationships, and behavioral patterns. This information could not only be used by adversaries but also aggregated into Meta's data collection, allowing for extensive user profiling that extends beyond the app itself.

Although no explicit references to dedicated advertising SDKs were detected, the heavy usage of domains like `graph.facebook.com` or `spclient-based` Spotify APIs and the lack of transparency of Meta's data collection practices raises questions about long-term data retention and user profiling. Additionally, the app's usage of WhatsApp during physical interactions, such as taking photos and recording videos in the network traffic analysis, stresses the questionable practices of data collection by Meta, prompting further concerns about how they might use or for how long they may retain the data.

Trade-offs between Device Functionality and Privacy

Meta has been continuously working to improve the privacy of its users by implementing measures to prevent unauthorized access to the glasses. As detailed in the device overview, one such measure introduced is to lock certain functionalities, such as having the AI read a message aloud, unless the paired phone is unlocked. Although this measurement improves user privacy, it diminishes the glasses' selling point of hands-free convenience. The requirement to unlock the phone before reading text messages aloud, for instance, negates the ideal hands-free interaction initially offered by the device, as the user would already have access to the message contents when unlocking the phone.

This trade-off represents a broader challenge where enhancing security and privacy controls comes at the cost of usability while optimizing ease of use can weaken protective measures. Even when some level of compromise may be needed, users would benefit more when these design decisions and their associated trade-offs are transparently communicated and available for users to decide.

Policy and Ethical Considerations

From a policy point of view, the observations from the network, Bluetooth, and app analyses show the lack of a clear regulatory framework and transparency surrounding the collection, storage, and sharing of data by wearable AR devices. Therefore, it is important to have clear guidelines on the retention periods of the data, the parties who have access to the data, and the specific purposes for processing the data. The encryption of the content prevents the interception of raw sensor data, addressing data confidentiality. However, the metadata analysis reveals substantial details about user activity and application usage, necessitating further technical solutions to address these exposures.

In addition, measures for bystander privacy remain underexplored, and there is no guarantee nor verification mechanism for bystander consent. Even though an LED notification is in place for bystanders on the Meta glasses, they can be easily missed or obstructed, as pointed out in the device overview. Users could bypass the security measures implemented in the glasses during video recording by blocking the LED indicator after starting a video recording, limiting bystander awareness. As a result, the glasses can be used as a discrete recording device, meaning that bystanders' data can still be easily and carelessly gathered, transmitted, and analyzed without their awareness or consent. This raises ethical questions about the transparency of AR devices in public spaces. Therefore, the risk from Meta's data collection, combined with the fact that bystanders, who have no direct relationship with the device or wearer, could be unknowingly captured in video and audio recordings, can lead to bystanders becoming part of a long-term analytic dataset.

The findings, emphasized by the device's heavy dependence on cloud services and the associated risks to bystanders, call for the need for stronger policies and strategies aimed at preserving the ability of users and bystanders to give informed consent. Device manufacturers and the wearers are both responsible for mitigating these concerns.

Conclusion and Future Work

Conclusion

The analyses presented in this thesis demonstrate that the Ray-Ban Meta glasses can reveal substantial user and bystander information, even when transmissions are protected through encryption protocols such as TLS or QUIC. The network analysis revealed that repeated domain lookups, distinctive packet sizes, and recognizable traffic volume patterns allow for inference of application usages and user activity, such as certain voice commands or media transfers. A possible countermeasure could be to obfuscate traffic volume by padding the packets. This would hide the user's usage patterns as the sizes of the packets being transmitted are similar to one another. However, if implemented, it raises other potential concerns, including the efficiency of the transmission, trust from the user, and transparency of the Meta servers regarding the transmitted content.

Similarly, the Bluetooth analysis reveals potential identifiers, including the manufacturer, in advertisements or user interactions, such as music playback. An advertiser that is able to monitor the network and Bluetooth communications may be able to infer user actions and gather information for user profiling. Furthermore, the potential abilities of the glasses to directly process information locally could lead to the possibility of data collection and retention, raising concerns about user and bystander privacy due to the possibly sensitive nature of the messages transmitted.

The companion app inspection shows that the glasses rely heavily on first-party infrastructure, such as data synchronization. These dependencies potentially enable the aggregation of location data, communication patterns, and media captures over a long period, not only to Meta but also to adversaries. This aggregation risk exists not only for users but also for bystanders. By continuously capturing audio or video in public environments, AR glasses can unintentionally record individuals who have neither been made aware of the device's data collection capabilities nor provided informed consent.

These findings are further reinforced by the partial success in the bypassed traffic, which also shows that determined adversaries could obtain certain encrypted content. Exposed data flow, such as usage metrics, including device IDs and session states, that are sent to Meta's servers could be aggregated to infer user behavior. Other requests revealed the app's method of fetching image resources, raising concerns about how images captured by

the user may be handled as it could also affect bystander privacy. Although encryption protects most transmission, the data being collected and retained in the more secure encrypted endpoints remains a question. Moreover, combined with the questionable practices mentioned in the network traffic, these findings reflect the limited transparency and ambiguous data retention policies available to users and bystanders.

Implemented designs, such as the phone-unlock requirement for certain AI features, enhance security, but they degrade the promised hands-free experience. Other mitigations, such as more visible recording indicators for different environments or continuous checks for no obstructions in the LED indicator, may improve public awareness. However, it does not fully address the deeper ethical concerns surrounding bystander privacy and the obligations of device manufacturers.

In conclusion, the Ray-Ban Meta illustrates that while AR wearables offer convenience to daily functions, they pose privacy challenges that require further exploration. This thesis shows that the metadata captured in the traffic is able to reveal user activity and behaviors even when payloads remain encrypted and hidden. Therefore, with the continuous expansion of AR technologies, these findings demonstrate the crucial need for regulatory guidelines, privacy measures, and transparency to protect users and the broader community affected by data collection.

Future Work

Future work could continue in several directions. The analysis did not fully decrypt network traffic, particularly when facing the app's certificate-pinning techniques. Future efforts could use more sophisticated certificate-pinning bypass methods or directly collaborate with Meta to better understand how data are collected, how long they are retained, and to whom they are shared.

Additional research could apply similar static and dynamic analysis methods to other AR devices, such as the Apple Vision Pro or Microsoft HoloLens. Comparative studies could identify common design patterns, vulnerabilities, or privacy features across multiple manufacturers, which may pinpoint better practices or device-specific shortcomings that require further solutions.

Bridging the gap between convenience and security while also ensuring the ethical treatment of non-users is a challenge that future AR devices should address. Future work might focus on filtering and transforming the data captured by AR devices so that unique personal identifiers are removed or masked to protect user privacy while sacrificing the least user convenience. For user privacy, the research could focus on traffic padding that reduces the identifiability of user activities or stronger privacy controls, such as providing explicit acknowledgments when cameras or microphones are activated through small audio cues or vibrations. For bystander privacy, the research could create methods to alert individuals nearby when AR glasses begin to take photos or record videos or implement opt-out features for bystanders, including blurring images to preserve bystander privacy in video captures.

Although research exists that introduces solutions to some privacy concerns, with the development of more AI-centered features on the Ray-Ban Meta, such as object recognition

or translation, more efforts could be made to introduce privacy-centered guidelines for AR wearables that protect both users and bystanders.

Bibliography

- [1] Python api for bluefruit le sniffer. https://github.com/adafruit/Adafruit_BLESniffer_Python.
- [2] Adafruit bluefruit le sniffer. <https://www.adafruit.com/product/2269>.
- [3] Meta privacy policy for additional data. <https://www.meta.com/en-gb/help/quest/articles/accounts/privacy-information-and-settings/additional-data-meta-quest/>.
- [4] Meta privacy policy for the activities and information provided by the user. <https://www.facebook.com/privacy/policy?subpage=1.subpage.1-YourActivityAndInformation>, .
- [5] Ray-ban — meta wayfarer smart glasses. <https://www.meta.com/ch/en/ai-glasses/wayfarer/>, .
- [6] Micro:bit. <https://microbit.org/>.
- [7] Ray-ban — meta wayfarer smart glasses - black. <https://www.ray-ban.com/usa/electronics/RW4006ray-ban%20|%20meta%20wayfarer-black/8056262326787>.
- [8] Ubertooth one. <https://greatscottgadgets.com/ubertoothone/>.
- [9] Wireshark. <https://www.wireshark.org/>.
- [10] nrf52840 dongle. <https://www.nordicsemi.com/Products/Development-hardware/nrf52840-dongle>, .
- [11] nrf52840 dongle hardware files. <https://www.nordicsemi.com/Products/Development-hardware/nRF52840-Dongle/Download>, .
- [12] Ftc staff report finds large social media and video streaming companies have engaged in vast surveillance. <https://www.ftc.gov/news-events/news/press-releases/2024/09/ftc-staff-report-finds-large-social-media-video-streaming-companies-have-engaged-vast-surveillance>, 2024.
- [13] Abbas Acar, Hossein Fereidooni, Tigist Abera, Amit Kumar Sikder, Markus Miettinen, Hidayet Aksu, Mauro Conti, Ahmad-Reza Sadeghi, and Selcuk Uluagac. Peek-a-boo: i see your smart home activities, even encrypted! In *Proceedings of the 13th ACM Conference on Security and Privacy in Wireless and Mobile Networks*, WiSec '20. ACM, July 2020. doi: 10.1145/3395351.3399421. URL <http://dx.doi.org/10.1145/3395351.3399421>.

- [14] Ajin Abraham. Mobile security framework (mobsf). <https://github.com/MobSF/Mobile-Security-Framework-MobSF>.
- [15] Damien Cauquil. Btlejack: a new bluetooth low energy swiss-army knife. <https://github.com/virtualabs/btlejack>.
- [16] Aldo Cortesi, Maximilian Hils, and Thomas Kriechbaumer. mitmproxy. <https://mitmproxy.org/>.
- [17] Jaybie A. De Guzman, Kanchana Thilakarathna, and Aruna Seneviratne. Security and privacy approaches in mixed reality: A literature survey. *ACM Computing Surveys*, 52(6):1–37, October 2019. ISSN 1557-7341. doi: 10.1145/3359626. URL <http://dx.doi.org/10.1145/3359626>.
- [18] Tamara Denning, Zakariya Dehlawi, and Tadayoshi Kohno. In situ with bystanders of augmented reality glasses: Perspectives on recording and privacy-mediating technologies. *Conference on Human Factors in Computing Systems - Proceedings*, 04 2014. doi: 10.1145/2556288.2557352. URL <https://ar-sec.cs.washington.edu/files/ar-chi2014.pdf>.
- [19] Onyeka D’Mello, Mathilde Gelin, Fatma Ben Khelil, Rojen Erik Surek, and Huihui Chi. Wearable iot security and privacy: A review from technology and policy perspective. In *International Conference on Future Network Systems and Security*, 2018. URL <https://api.semanticscholar.org/CorpusID:49412926>.
- [20] Hamidreza Fereidouni, Olga Fadeitseva, and Mehdi Zalai. Iot and man-in-the-middle attacks, 2023. URL <https://arxiv.org/abs/2308.02479>.
- [21] Andrea Gallardo, Chris T.L. Choy, Jaideep Juneja, Efe Bozkir, Camille Cobb, Lujo Bauer, and Lorrie Faith Cranor. Speculative privacy concerns about ar glasses data collection. *Proc. Priv. Enhancing Technol.*, 2023:416–435, 2023. URL <https://api.semanticscholar.org/CorpusID:259953156>.
- [22] Hilda Hadan, Derrick Wang, Lennart Nacke, and Leah Zhang-Kennedy. Privacy in immersive extended reality: Exploring user perceptions, concerns, and coping strategies. 05 2024. URL https://www.researchgate.net/publication/378206872.Privacy_in_Immersive_Extended_Reality_Exploring_User_Perceptions_Concerns_and_Coping_Strategies.
- [23] Andreas Kotsios. Privacy in an augmented reality. *International Journal of Law and Information Technology*, 23:157–185, 06 2015. doi: 10.1093/ijlit/eav003. URL https://www.researchgate.net/publication/276499565_Privacy_in_an_augmented_reality.
- [24] Tushar Nagraire, Parul Sindhwad, and Faruk Kazi. Ble protocol in iot devices and smart wearable devices: Security and privacy threats, 2023. URL <https://arxiv.org/abs/2301.03852>.
- [25] Yuhong Nan, Xueqiang Wang, Luyi Xing, Xiaojing Liao, Ruoyu Wu, Jianliang Wu, Yifan Zhang, and XiaoFeng Wang. Are you spying on me? Large-Scale analysis

- on IoT data exposure through companion apps. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 6665–6682, Anaheim, CA, August 2023. USENIX Association. ISBN 978-1-939133-37-3. URL <https://www.usenix.org/conference/usenixsecurity23/presentation/nan>.
- [26] AnhPhu Nguyen and Caine Ardayfio. I-XRAY: Raising Awareness About the Privacy Implications of Ray-Ban Meta Glasses. https://docs.google.com/document/d/1iWCqmaOUKhKjcKSktIwC3NNANoFP7vPsRvcbOIup_BA.
- [27] OpenAI. ChatGPT (Version 4), 2024. URL <https://openai.com/chatgpt>. ChatGPT was used for inspiration and optimization of code used in this thesis.
- [28] Jingjing Ren, Daniel J. Dubois, David R. Choffnes, Anna Maria Mandalari, Roman Kolcun, and Hamed Haddadi. Information exposure from consumer iot devices: A multidimensional, network-informed measurement approach. *Proceedings of the Internet Measurement Conference*, 2019. URL <https://api.semanticscholar.org/CorpusID:203582354>.
- [29] Franziska Roesner, Tadayoshi Kohno, and David Molnar. Augmented reality: Challenges and opportunities for security and privacy. <https://ar-sec.cs.washington.edu/files/ARSec2011.pdf>.
- [30] Seyedmostafa Safavi and Zarina Shukur. Improving google glass security and privacy by changing the physical and software structure. *Life Sciences*, 11, 05 2014. URL https://www.researchgate.net/publication/265867348_Improving_Google_glass_security_and_privacy_by_changing_the_physical_and_software_structure/link/5a94ee2a0f7e9ba42970ddb2/download?_tp=eyJjb250ZXh0Ijp7ImZpcnN0UGFnZSI6InB1YmxpY2F0aW9uIiwicGFnZSI6InB1YmxpY2F0aW9uIn19.
- [31] Daniel Schwendner. Now I See You: Uncovering Security Vulnerabilities in Camera Sunglasses. <https://code-byter.com/2023/09/24/rayban-hacking.html>, September 2023.



Appendices

Detailed experimental setup, scripts, or tools used. Raw data samples or logs (if permissible).

	Malware	Common
Dangerous	Access to Fine location Access to Coarse location Get Accounts Read to External Storage Write to External Storage Receive SMS Read SMS Send SMS Read Call log Read Contacts Read Phone State	Access Background location
Normal	Access Network State Access Wi-Fi State Internet Wake Lock Receive Boot Completed	Change Network State Change Wi-Fi State Bluetooth Bluetooth Admin Modify Audio Settings Foreground Service Receive

Table A.1: Summary of abused permissions

Shared Object	What is it	Origin	Purpose	Party
libbreakpad	A crash-reporting library	Google	Used for collecting crash data and debugging issues	Third-Party
libandroidx.graphics.path	Related to the Android Jetpack ‘androidx.graphics’ package	Android Jetpack	Used for rendering graphics	Platform-Provided
libforce_dlopen	Dynamic linking library	In-house (Meta / Facebook)	Loads the dynamic shared object and handles runtime linking	First-Party
libbreakpad_cpp_helper	A crash-reporting library	Google	Helper for collecting crash data and debugging issues	Third-Party
libandroid_aware_dlopen	Dynamic linking library	In-house (Meta / Facebook)	Handles dynamic linking	First-Party
liblinkerutils	Dynamic linking utility library	In-house (Meta / Facebook)	Assists with dynamic linking	First-Party
libsigmux	Signal multiplexing library	In-house (Meta / Facebook)	Manages execution flow and signal handling	First-Party
libfbunwindstack	Related to stack unwinding for crash analysis and debugging	In-house (Meta / Facebook)	Provides stack unwinding for debugging and crash reporting	First-Party
libsigx	Related to signal handling	In-house (Meta / Facebook)	Handles signal-related functionality	First-Party
libdistract	Runtime instrumentation library	In-house (Meta / Facebook)	Implements instrumentation features such as function hooking and memory management	First-Party
libc++_shared	Standard shared library	Android	Provides C++ runtime support	Platform-Provided
libsuperpack-jni	JNI-based compression utility	In-house (Meta / Facebook)	Used to reduce the size of APKs and optimize storage	First-Party
libunwindstack_binary	Stack unwinding library	In-house (Meta / Facebook)	Supports stack tracing for debugging purposes	First-Party

Table A.2: Classification of native libraries in the Meta View app

Type			
Component Name	Purpose	Protection	Party
Broadcast Receivers			
com.facebook.stella.util.timechange.TimeChangeReceiver	Listens for system time change events	None	First-Party
androidx.work.impl.diagnostics.DiagnosticsReceiver	Handles diagnostic logging	android.permission.DUMP	Platform-Provided
androidx.profileinstaller.ProfileInstallerReceiver	Installs optimized runtime profiles	android.permission.DUMP	Platform-Provided
com.google.firebase.iid.FirebaseInstanceIdReceiver	Handles Firebase push notifications	com.google.android.c2dm.permission.SEND	Third-Party
Services			
com.facebook.analytics2.logger.GooglePlayUploadService	Uploads analytics data to Facebook servers	com.google.android.gms.permission.BIND_NETWORK_TASK_SERVICE	First-Party
androidx.work.impl.background.systemjob.SystemJobService	Schedules background tasks via JobScheduler	android.permission.BIND_JOB_SERVICE	First-Party
com.facebook.pushlite.PushLiteGCMJobService	Handles lightweight push notifications via Google Cloud Messaging	com.google.android.gms.permission.BIND_NETWORK_TASK_SERVICE	Third-Party
com.meta.wearable.acdc.service.ACDCService	Provides services for the Meta View app	None	First-Party
com.facebook.assistant.stella.ipc.instagram.service.InstagramIpcDirectMessagingServiceServer	Handles Instagram direct messaging services	None	Support-Party
com.facebook.stella.assistant.services.StellaMessengerIpcIntentService	Handles messaging services for the Meta Assistant	None	Support-Party
com.facebook.stella.assistant.services.StellaInstagramIpcIntentService	Handles Instagram-related messaging intents	None	Support-Party
com.facebook.stella.assistant.services.StellaWhatsAppCallbackService	Handles callbacks for WhatsApp integration	com.whatsapp.permission.BROADCAST	Support-Party
com.meta.wearable.applinks.AppLinkService	Handles app linking functionality	None	First-Party
com.meta.wearable.applinks.InsecureAppLinkService	Handles unsecured app linking functionality	None	First-Party
com.facebook.wearable.assistant.ipc.MwaTunnelService	Facilitates tunneling for Meta Wearable Assistant services	None	First-Party
com.facebook.wearable.companion.notification.MWANotificationListenerService	Listens for wearable notifications	android.permission.BIND_NOTIFICATION_LISTENER_SERVICE	First-Party

com.google.android.gms.auth.api.signin.RevocationBoundService	Handles Google account sign-in revocation notifications	com.google.android.gms.auth.api.signin.permission.REVOCATION_NOTIFICATION	Third-Party
Activities and Activity Aliases			
libraries.fxmetalogin.webauth. MetaWebAuthenticationActivity	Handles Meta login authentication via the web	None	First-Party
com.spotify.sdk.android.auth.browser. RedirectUriReceiverActivity	Manages Open Authorization redirects for Spotify authentication	None	Third-Party
com.facebook.secure.packagefinder.PackageFinderActivity	Finds secure packages for Meta services	None	First-Party

Table A.3: Analysis of Manifest Components in the Meta View App

Context	
Action	What does it do
Media Capture	
Short click capture button	Take a photo or to stop a video recording
Long click capture button	Start video recording
Music Playback	
Tap (and hold) on the touchpad	Play music
Swipe forward on the touchpad	Increase volume
Swipe backwards on the touchpad	Decrease volume
Double tap on the touchpad	Skip music
Tap on the touchpad	If music is playing to pause it

Table A.4: List of all the touch commands performed on the glasses in the experiments

Context	Action
General	"Hey Meta, how is the weather in [place] today?"
	"Hey Meta, what time is it?"
	"Hey Meta, what date is today?"
	"Hey Meta, what is the battery percentage?"
Music Playback	"Hey Meta, play music."
	"Hey Meta, increase the volume."
	"Hey Meta, lower the volume."
	"Hey Meta, skip track."
	"Hey Meta, pause music."
Timer	"Hey Meta, set a timer for 3 minutes."
	"Hey Meta, how much time is left in my timer?"
	"Hey Meta, cancel my timer for 3 minutes."
Media Capture	"Hey Meta, take a photo."
	"Hey Meta, take a video."
	"Hey Meta, stop recording."
Messaging	"Hey Meta, send a message to [name]."
	"Hey Meta, send a voice message to [name]."
	"Hey Meta, read my messages."
Phone Call	"Hey Meta, decline the call."

Table A.5: List of all the voice commands performed on the glasses for the voice command experiments

Domains
http://clients3.google.com/generate_204
http://findapi.micloud.xiaomi.net/mic/find/v3/anonymous/time?ckey=...
https://lookaside.facebook.com/ras/v2/id=...
https://lookaside.bfsbx.com/elementpath/media?media_id=...
https://b-www.facebook.com/mobile/reliability_event_log_upload/
https://accounts.google.com/ListAccounts?gpsia=1&source=ChromiunBrowser&json=standard
https://clients4.google.com/chrome-sync/command/?client=Google+Chrome&client_id=...
https://peoplestack-pa.googleapis.com/peoplestack.PeopleStackAutocompleteService/Autocomplete
https://taskassist-pa.googleapis.com/v2/taskassist:compose?alt=photo
https://ar.graph.meta.com/federated_learning/fl_acs_match?acs_config_id=...
https://ar.graph.meta.com/federated_learning/fl_acs_checkin?acs_config_id=...
https://ar.graph.meta.com/federated_learning/fl_acs_download?acs_config_id=...
https://ar.graph.meta.com/federated_learning/fl_acs_report?acs_config_id=...
https://ar.graph.meta.com/federated_learning/fl_acs_upload?acs_config_id=...
https://ar.graph.meta.com/federated_learning/fl_acs_config
https://ar.graph.meta.com/federated_learning/fl_acs_auth
https://scontent.xx.fbcdn.net/m1/v/t39.22532-6/...
https://static.xx.fbcdn.net/m1/rsrc.php/v4...
https://about.facebook.com/ray-ban-stories-privacy/
https://www.facebook.com/tr/?id=...
https://www.meta.com/smart-glasses/privacy/...
https://www.meta.com/ajax/bootloader-endpoint/...
https://www.meta.com/ajax/bulk-route-definitions
https://www.meta.com/api/graphql
https://www.meta.com/a/fl?_a=1&_ccgEXCELLENT&_comet_req=18&_hs=...
https://connect.facebook.net/signals/config/...

Table A.6: List of the domains found in mitmproxy

B

Code for Bypassing Certificate

Code B.1: Code to Bypass Certificate Pinning that hooks into specific classes in the app and bypasses the methods. ChatGPT [27] was used for inspiration and optimization of code for bypassing certificate pinning and visualizations.

```
1 Java.perform(function () {
2     console.log("Starting_Certificate_Pinning_Bypass_for_
      ↳ checkServerTrusted");
3
4     try {
5         // Hooking into X.0Et
6         // How to find class:
7         // Search for '.super Ljavax/net/ssl/X509ExtendedTrustManager;'
      ↳ in decoded apk
8         let class_name = 'X.0F1';
9         var EtClass = Java.use(class_name);
10        console.log('Found class: ${class_name}');
11
12        EtClass.checkServerTrusted.overload('[Ljava.security.cert.
      ↳ X509Certificate;', 'java.lang.String').implementation =
      ↳ function (certs, authType) {
13            console.log('Bypassing checkServerTrusted in ${class_name}
      ↳ with String');
14            return;
15        };
16
17        EtClass.checkServerTrusted.overload('[Ljava.security.cert.
      ↳ X509Certificate;', 'java.lang.String', 'java.net.Socket').
      ↳ implementation = function (certs, authType, socket) {
18            console.log('Bypassing checkServerTrusted in ${class_name}
      ↳ with Socket');
19            return;
20        };
21    }
```

```
22      EtClass.checkServerTrusted.overload('[Ljava.security.cert.  
    ↪ X509Certificate;', 'java.lang.String', 'javax.net.ssl.  
    ↪ SSLEngine').implementation = function (certs, authType,  
    ↪ sslEngine) {  
23          console.log('Bypassing checkServerTrusted in ${class_name}  
    ↪ with SSLEngine');  
24          return;  
25      };  
26  
27      // Hooking into X.OAM  
28      // How to find class:  
29      // Search for '.implements Ljavax/net/ssl/X509TrustManager;'  
30      // Then check which has an annotation with value '"Ljava/util/  
    ↪ Set<", "Ljava/nio/ByteBuffer;",' in instance fields in  
    ↪ code  
31      let class_name2 = 'X.OAx';  
32      var AMClass = Java.use(class_name2);  
33      console.log('Found class: ${class_name2}');  
34  
35      AMClass.checkServerTrusted.overload('[Ljava.security.cert.  
    ↪ X509Certificate;', 'java.lang.String').implementation =  
    ↪ function (certs, authType) {  
36          console.log('Bypassing checkServerTrusted in ${class_name2}  
    ↪ with String');  
37          return;  
38      };  
39  
40      } catch (err) {  
41          console.log("Error_hooking_into_SSL_methods:_ " + err);  
42      }  
43      console.log('SSL_Pinning_Bypass_done');  
44  
45  });
```

Declaration on Scientific Integrity



University
of Basel

Faculty of Science



Declaration on Scientific Integrity

(including a Declaration on Plagiarism and Fraud)

Translation from German original

Title of Thesis: Privacy Analysis of Augmented Reality Glasses

Name Assessor: Prof. Dr. Isabel Wagner

Name Student: Michelle Jessica Bösigler

Matriculation No.: 21-061-528

I attest with my signature that I have written this work independently and without outside help. I also attest that the information concerning the sources used in this work is true and complete in every respect. All sources that have been quoted or paraphrased have been marked accordingly.

Additionally, I affirm that any text passages written with the help of AI-supported technology are marked as such, including a reference to the AI-supported program used. This paper may be checked for plagiarism and use of AI-supported technology using the appropriate software. I understand that unethical conduct may lead to a grade of 1 or "fail" or expulsion from the study program.

Place, Date: Basel, 23.01.2025 Student: Michelle J. Bösigler

Will this work, or parts of it, be published?

☐ No

☒ Yes. With my signature I confirm that I agree to a publication of the work (print/digital) in the library, on the research database of the University of Basel and/or on the document server of the department. Likewise, I agree to the bibliographic reference in the catalog SLSP (Swiss Library Service Platform). (cross out as applicable)

Publication as of: _____

Place, Date: Basel, 23.01.2025 Student: Michelle J. Bösigler

Place, Date: _____ Assessor: _____

Please enclose a completed and signed copy of this declaration in your Bachelor's or Master's thesis.